

重庆邮电大学

学生实验实习报告册

学年学期： 2025-2026学年 ☒春 ☐秋学期

课 程 名 称： A2013090-嵌入式及其应用开发综合实验

学 生 学 院： 通信与信息工程学院

专 业 班 级： 通信与信息类

划 分 组 号： 1

	2024210241	张健豪
--	------------	-----

学生学号：	2024210232	黄浩睿
-------	------------	-----

学生姓名：	2024210236	杨洪川
-------	------------	-----

	2024210220	姚家宝
--	------------	-----

联 系 电 话
(组 长)： 18696866652

重庆邮电大学教务处制

课程名称	嵌入式及其应用开发综合实验	课程编号	A2013090
实验地点	YF312 □, YF311 □	实验时间	第 19-20 周
校外指导教师		校内指导教师	梁燕
实验名称	专题 2&专题 3 物流分拣平台设计与实现		
评阅人签字		成绩	

摘要（中文）

随着智能制造与智慧物流的快速发展，自动化分拣系统已成为提升物流效率、降低运营成本的关键环节。本实训项目基于 STM32F103C8T6 微控制器，设计并实现了一套完整的物流分拣平台仿真系统。系统由三大功能模块构成：光电计数器通过外部中断按键实现计数值的加、减与清零，并支持串口远程设定与长按连加连减功能；传送带控制器通过 ADC 采集电位器实现模拟调速，同时支持 UART 数字指令调速，采用 10 档位精细控制与缓增缓减平滑变速机制，并集成正反转安全换向保护；分拣器控制器通过 USART 接收视觉识别结果，在 OLED 显示屏上实时显示货物类别，支持多货物分类统计、分拣动画进度条以及含 CRC 校验的标准化通信协议。在物联网接入层面，系统部署了 EMQX 本地 MQTT 服务器，编写 Python 网关脚本实现虚拟串口数据与 MQTT 消息的双向转换与设备 ID 路由，完成 STM32→Python→EMQX→Qt 上位机的全链路数据透传与远程控制。经 Proteus 仿真与全链路联调测试，三大模块基础功能与拓展功能均运行稳定，上下行数据解析与分发准确无误，实现了设备状态上报与远程指令下发的完整闭环。本系统将分散的单一实验集成为协同工作的物流分拣平台，从仿真到上云完整覆盖了嵌入式系统开发的工程流程，为实训教学提供了可验证、可扩展的技术方案。

关键词（中文）

STM32；物流分拣；PWM 调速；MQTT；物联网

第1章 引言

1.1 项目背景与意义

一、物流行业智能化、自动化的发展趋势

物流是实体经济的“筋络”，连接生产与消费的各个环节。近年来，随着 AI 大模型、物联网、云计算等新一代信息技术的深度渗透，以及电商经济持续增长对物流效率提出的更高要求，中国物流行业正经历从“人力密集型”向“技术密集型”的关键跃迁。

市场规模持续扩张。2025 年，全国社会物流总额达 368.2 万亿元，同比增长 5.1%，连续多年保持高于 GDP 的增速。中国智慧物流市场规模已达约 9655 亿元至 1.2 万亿元，预计 2026 年将突破 1.4 万亿元。在智能仓储领域，市场规模已从 2019 年的 882.9 亿元增长至 2024 年的 1760.5 亿元，实现了体量的显著跃升。

技术驱动深刻变革。当前，人工智能在物流供应链领域的渗透率已超过 37%，物流科技领域呈现出 AIoT、智能机器人、自动驾驶等技术多点突破的态势。AI 大模型、机器视觉、自动驾驶、智能调度和数字孪生等技术共同作用，使物流系统开始深度参与路径规划、库存管理、设备调度和异常响应。仓储物流技术已从“适度自动化+高度信息化”阶段过渡到“高度自动化+高度信息化”阶段，并开始向智能化全面演变。

新型装备规模化应用。随着移动协作机器人、智能分拣设备、无人配送等新型物流装备的规模化应用，物流效率持续改善。具身机器人已走进仓库和厂房，完成了从“自动导引车”向“能感知、会思考、可操作”的智能体的阶段性跨越。在分拣中心和仓库里，搬运和拣选机器人功能持续改进，既可以服务包裹搬运，也能够快速精准处理复杂单品。无人配送车加快应用落地，行业累计应用无人车超千辆。智能化已成为贯穿行业干线运输、末端配送、冷链管理等环节的关键力量。

降本增效成效显著。2025 年，社会物流总费用与国内生产总值的比率降至 13.9%，为有记录以来的最低水平。智能化技术不仅提升了物流效率、降低了成本，更通过预测分析、智能调度等创新应用，增强了供应链韧性。智慧物流正成为实体经济降本增效的核心引擎。

展望未来，物流行业将加速从“规模竞赛”转向“价值深耕”，由人工智能驱动的业务预测、数字孪生技术实现的预测性维护等，将成为物流服务运营商日常运营的标配功能。自动化分拣系统作为智能物流的关键一环，其重要性日益凸显。

二、分拣系统在物流中的关键作用

在物流配送中心的全部作业环节中，分拣是非常重要的环节，分拣成本占物流搬运成本的 40%，是整个配送中心作业系统的核心。自动分拣系统是现代物流配送中心所必需的设施设备之一，其关键作用体现在以下几个方面：

大幅提升分拣效率。与传统人工分拣相比，自动化分拣系统可实现质的飞跃。在技术应用层面，一套智能分拣系统可使转运中心分拣效率达到每小时 8 万袋件，生产效率提升 90%。在实际案例中，有快递企业引入智能分拣系统后，日处理能力从人工分拣的 1 万多件跃升至 9 到 10 万件；自动化分拣线可使整体作业效率提升 4 至 5 倍；大件自动分拣线效率较人工提升 5 倍。

显著提高分拣准确率。自动化分拣系统搭载高精度扫描与识别技术，可自动完成包裹称重、扫码、分拣全流程操作。智能分拣设备的准确率可超过 99.99%，错分率可控制在万分之一以下。部分先进系统的错分率甚至低于 0.01%，实现了低人力投入、高生产收益的目标。

有效降低人力成本。自动化分拣系统能够完全替代传统人工分拣中的大量重复性劳动。在智能化程度超过 90%的分拣系统中，人力成本可节省约 20%。机器人可以全天不停工，再加上 AI 调度系统，仓储周转速度明显加快，库存成本也随之下降。

增强物流系统韧性。自动化分拣系统可全天连续运转，能够有效应对电商促销季等业务高峰期的订单压力。通过 AI 驱动的动态优化，系统能实时适应不断变化的需求，最大限度地减少处理时间。

综上所述，自动分拣系统作为物流配送中心的核心功能环节，对于企业提高物流效率、降低运营成本具有不可替代的作用。随着物流行业智能化、自动化趋势的持续深化，分拣系统的技术升级与应用拓展将成为推动物流业高质量发展的重要引擎。

1.2 国内外研究现状

当前物流分拣设备主要分为大型快递自动分拣机和工业生产线分拣器两大类。快递自动分拣机普遍采用条码/二维码识别技术,通过高速扫码相机获取包裹面单信息,配合摆轮式或交叉带式分拣机构实现高效分拣,分拣效率可达每小时数万件,准确率超过 99.9%。工业生产线分拣器则多依赖光电传感器、视觉传感器或 RFID 识别技术,由控制器根据检测结果驱动气动或电动执行机构完成物料分类。

然而,现有系统在实训教学场景中存在明显局限:一是系统高度集成、封闭性强,学生难以深入理解底层硬件驱动与通信协议细节;二是多数设备采用专用控制器或 PLC 方案,编程门槛高、开发周期长,不利于嵌入式系统教学;三是缺乏从硬件设计、驱动开发到云端接入的完整实训链条,学生难以建立系统级工程思维。

针对上述问题,本项目以 STM32F103C8T6 为核心控制器,从硬件电路设计、外设驱动开发、串口通信协议设计到 MQTT 物联网接入,构建了一套完整的物流分拣平台实训方案。系统涵盖光电计数器、传送带控制器、分拣器控制器三大功能模块,并通过 Python 网关实现串口数据与 MQTT 消息的双向转换,最终接入 Qt 上位机实现远程监控与控制。

1.3 本文主要工作与章节安排

• 本文围绕物流分拣平台的设计与实现展开,各章节安排如下:

• **第 1 章 引言:** 阐述项目背景、物流分拣在智能制造中的重要性,调研国内外同类产品现状,指出现有系统在实训教学中的局限性,提出本项目的改进思路。

• **第 2 章 系统总体设计:** 进行系统需求分析,设计系统总体架构,明确技术路线与开发流程,划分三大功能模块并定义各模块的输入输出接口。

• **第 3 章 硬件设计:** 分别设计光电计数器、传送带控制器、分拣器控制器的电路原理图,完成引脚规划、元器件选型,并详细描述各电路的基础功能与拓展功能。

• **第 4 章 软件设计:** 分别阐述三大模块的软件架构、程序流程、模块化编程设计、关键函数接口,设计组件串口通信协议,并完成 MQTT 物联网部署与 Python 网关脚本设计。

• **第 5 章 系统测试与结果分析:** 制定测试环境与方案,分别对三大模块进行功能测试,完成 MQTT 全链路联调测试,分析测试中遇到的问题及解决方法。

• **第 6 章 总结与展望:** 总结已完成的工作与成果,指出当前系统的不足,并提出未来拓展方向。

本项目的创新点主要体现在三个方面:一是将分散的单一功能实验(计数器、传送带、分拣器)集成为完整的物流分拣系统,实现了功能模块间的协同工作;二是通过 Python 网关实现串口到 MQTT 的协议转换,完成了从本地仿真到物联网云端的全链路接入;三是在基础功能之上扩展了长按连加连减、缓增缓减、CRC 校验协议等实用功能,提升了系统的交互性与可靠性。

第2章 系统总体设计

2.1 系统需求分析

本系统面向智能物流分拣场景，需满足以下功能需求：

- (1) **计数功能：**光电计数器应能通过按键实现加一、减一和清零操作，计数值在两位数码管上实时显示，并支持通过串口远程设定任意计数值。
- (2) **调速功能：**传送带控制器应支持 ADC 模拟调速（通过滑动变阻器）和 UART 数字指令调速两种模式，速度分为 10 个档位，电机转速随档位变化平滑过渡。
- (3) **分拣功能：**分拣器控制器应能通过 USART 接收分拣指令，在 OLED 显示屏上实时显示货物类别与分拣状态，支持多货物分类统计。
- (4) **远程监控功能：**系统应通过 MQTT 协议将设备状态上报至云端，并支持上位机远程下发控制指令，实现设备状态实时监控与远程控制。
- (5) **多设备协同：**三类设备应能并行接入同一物联网系统，通过设备 ID 实现消息精准路由，互不干扰。

2.2 系统总体架构

- **系统采用分层架构设计，自下而上分为硬件层、驱动层、应用层、通信层和云端层五个层次：**
 - **硬件层：**以 STM32F103C8T6 微控制器为核心，包含光电计数器（数码管、按键）、传送带控制器（L298N 电机驱动、电位器、OLED）、分拣器控制器（OLED、LED、蜂鸣器）三类硬件设备。
 - **驱动层：**基于 STM32 HAL 库实现各外设的底层驱动，包括 GPIO、EXTI、ADC、TIM（PWM）、USART、I²C（模拟）等模块的初始化与操作函数。
 - **应用层：**实现各设备的核心业务逻辑，包括计数器的计数更新与显示、传送带的档位管理与平滑调速、分拣器的指令解析与分类统计。
 - **通信层：**包含串口通信与 MQTT 通信两个子层。串口层实现 STM32 与 Python 网关之间的数据收发；MQTT 层实现 Python 网关与 EMQX 服务器之间的消息发布/订阅。
 - **云端层：**EMQX 作为 MQTT Broker 负责消息中转，Qt 上位机作为应用端实现设备状态显示与远程控制。
- 数据流：上行方向，设备状态数据经 STM32 串口→Python 网关（协议转换）→EMQX→Qt 上位机；下行方向，控制指令经 Qt 上位机→EMQX→Python 网关（协议转换）→STM32 串口→设备执行。
- 三大子系统的协同关系为：**传送带负责物料的输送与速度调节，光电计数器负责物料数量的统计与上报，分拣器控制器根据视觉识别结果完成物料分类决策，三者通过统一的 MQTT 主题体系实现数据互通与状态同步。

2.3 技术路线与开发流程

本项目的开发工具链包括：

STM32CubeMX：用于芯片引脚配置、外设初始化与 Keil 工程生成

Keil MDK-ARM (uVision 5)：用于 STM32 固件代码的编写、编译与调试

Proteus 8 Professional：用于电路原理图绘制与系统仿真验证

VSPD 虚拟串口软件：用于创建虚拟串口对，实现 Proteus 与上位机的串口通信

Python 3.8+：编写网关脚本，实现串口数据读取与 MQTT 消息转换

EMQX 5.3.2：本地 MQTT 服务器部署，实现消息中转

Qt 上位机：实现设备状态显示与远程控制

开发流程遵循“配置生成→驱动编写→仿真验证→物联网接入→联调测试”的闭环路径：

- (1) 使用 STM32CubeMX 完成引脚配置、时钟树设置与外设初始化，生成 Keil 工程框架。
- (2) 在 Keil 中编写各模块的驱动代码与应用逻辑，实现基础功能。

- (3) 在 Proteus 中绘制电路原理图，加载 HEX 文件进行仿真验证。
- (4) 部署 EMQX 服务器，编写 Python 网关脚本，完成串口到 MQTT 的协议转换。
- (5) 启动全链路联调，验证 STM32→Python→EMQX→Qt 的双向通信。

软件开发采用模块化思路，各功能模块独立封装（如 display.c、oled.c、motor.c），通过清晰的接口函数实现模块间调用，便于代码复用与功能扩展。

2.4 功能模块划分

系统按子系统划分为三大功能模块：

（1）光电计数器模块

主要功能：通过外部中断检测按键实现加一、减一和清零；计数值在两位共阳极数码管上动态扫描显示；通过 USART 上报按键事件并接收远程设定值。

输入接口：KEY1（PA1，加计数）、KEY2（PA2，减计数）、KEY3（PA3，清零），均为上升沿触发外部中断；USART1_RX（PA10）接收远程指令。

输出接口：段选 PB0~PB7、位选 PB8~PB9 驱动数码管；USART1_TX（PA9）发送按键事件。

创新功能：长按按键连加/连减（按住超过 500ms 后每 200ms 自动执行一次）；串口远程设置任意两位数值。

（2）传送带控制器模块

主要功能：通过 ADC 采集滑动变阻器模拟电压，映射为 PWM 占空比控制电机转速；支持 UART 远程调速指令；OLED 实时显示模式、档位、占空比和方向。

输入接口：ADC1_IN5（PA5）采集电位器分压；USART1_RX（PA10）接收远程调速指令。

输出接口：TIM2_CH1（PA0）输出 PWM 信号至 L298N 的 ENA；PA1（IN1）、PA2（IN2）控制电机方向；I²C（PA3/PA4）驱动 OLED 显示。

创新功能：10 档位精细控制；缓增缓减平滑变速（每 50ms 步进 1%）；正反转控制与安全换向保护。

（3）分拣器控制器模块

主要功能：通过 USART 接收视觉识别结果，在 OLED 上显示货物类别；支持多货物分类统计；具备 CRC 校验的标准化通信协议。

输入接口：USART1_RX（PA10）接收分拣指令。

输出接口：I²C（PA6/PA7）驱动 OLED 显示；PB0（绿色 LED）、PB1（红色 LED）状态指示；PB10 控制蜂鸣器。

创新功能：多货物分类与分拣计数统计；分拣动画与进度条显示；帧头+CRC+帧尾的标准化通信协议；声光反馈联动。

第3章 硬件设计

3.1 主控芯片与外围电路总体设计

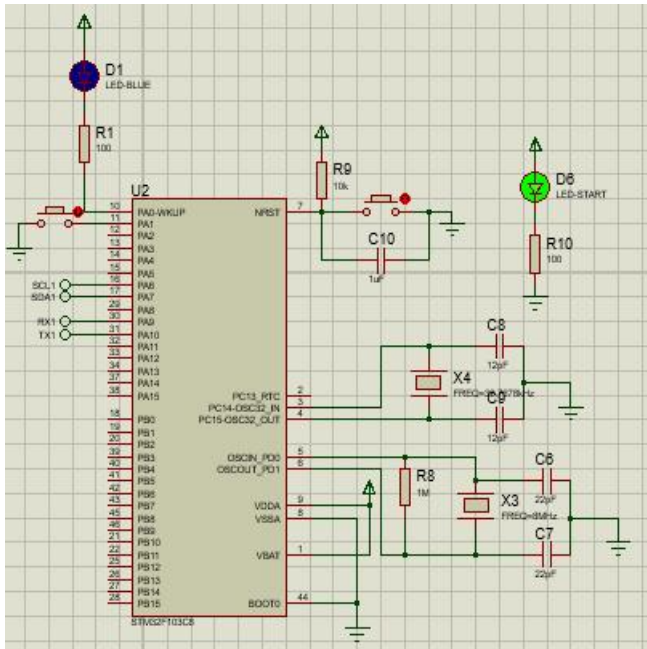


图 1.主控最小系统电路（晶振、复位、供电）

3.2 光电计数器电路设计

3.2.1 光电计数器电路设计概述

光电计数器是物流分拣平台中实现物料数量统计的核心功能模块。本系统采用红外对射式光电传感器作为检测元件，以 STM32F103C8T6 微控制器为核心处理单元，通过外部中断方式实时捕获物料经过信号，实现非接触式、高精度的自动计数功能。

3.2.2 引脚规划和功能分配

引脚编号	功能名称	方向	说明
PB0~PB7	SEG_A~SEG_G	输出	数码管段选信号（共阳极，低电平点亮）
PB8	DIG_TEN	输出	十位位选控制（高电平选中）
PB9	DIG_UNIT	输出	个位位选控制（高电平选中）
PA1	KEY_ADD	输入	加计数按键（上升沿触发外部中断）
PA2	KEY_SUB	输入	减计数按键（上升沿触发外部中断）
PA3	KEY_CLR	输入	清零按键（上升沿触发外部中断）
PA9	USART1_TX	输出	串口发送，与上位机及分拣控制器通信
PA10	USART1_RX	输入	串口接收，接收上位机远程设定值

表 1. 光电计数器电路引脚规划和功能分配

3.2.3 元器件选型

(1) STM32F103C8T6 微控制器× 1: 基于 ARM Cortex-M3 内核, 主频最高 72MHz, 64KB Flash, 20KB SRAM, LQFP48 封装, 作为系统的核心控制单元。

(2) 7SEG-MPX2-CA 2 位共阳极数码管× 1: 段选引脚 A~G、DP 经 100 Ω 排阻连接至 PB0~PB7, 公共阳极经 NPN 三极管接 VCC, 位选引脚 PB8 (十位)、PB9 (个位) 控制三极管基极, 段选低电平点亮, 位选高电平选中, 通过动态扫描实现两位数字显示。

(3) NPN 三极管× 2: 基极分别连接 PB8 和 PB9, 集电极接数码管公共阳极, 发射极接 3.3V 电源, 用于位选通断控制。

(4) 排阻 RES16DIPIS (100 Ω) × 1: 连接在 PB0~PB7 与数码管段选引脚之间, 起限流保护作用。

(5) BUTTON 按键开关× 3: KEY1 (PA1) 加计数、KEY2 (PA2) 减计数、KEY3 (PA3) 清零复位, 按键一端接 STM32 GPIO 引脚, 另一端接 VCC (高电平有效), 引脚配置为内部下拉+上升沿触发外部中断。

(6) COMPIM 虚拟串口组件× 1: Proteus 中的物理串口接口模型, 将仿真电路中的 UART 信号连接到计算机的虚拟 COM 口, 实现 Proteus 仿真与上位机串口助手的数据交互。

(7) 外部晶振 (8MHz) × 1: 连接在 OSC_IN (PD0) 和 OSC_OUT (PD1) 之间, 为 STM32 提供外部高速时钟源 (HSE), 通过 PLL 倍频至 72MHz 系统时钟。

(8) 晶振负载电容 (22pF) × 2: 分别连接在晶振两端与 GND 之间, 构成振荡电路。

(9) 复位电容 (1 μ F) × 1: 并联在 NRST 与 GND 之间, 配合 10k Ω 上拉电阻实现上电自动复位。

(10) 去耦电容 (100nF) × 若干: 连接在 VDD 与 VSS 引脚之间, 滤除电源高频噪声, 保证 MCU 稳定工作。

(11) VSPD 虚拟串口软件 × 1: 在计算机上创建虚拟串口对 (如 COM3↔ COM4), 实现上位机与 Proteus 仿真之间的串口数据通道。

(12) 串口调试助手× 1: 运行于上位机, 通过虚拟串口与 Proteus 的下位机进行串口数据收发和显示。

3.2.4 电路原理图

计数器电路原理图如图所示, 数码管采用动态扫描方式, 两个 NPN 三极管分别受 PB8 和 PB9 控制, 当位选信号为高电平时, 对应位的公共阳极接至 VCC, 段选低电平则点亮相应笔段。按键采用外部中断触发, 无需在主循环中轮询, 提高了响应效率。USART1 通过 COMPIM 映射到 PC 的虚拟串口, 便于与上位机调试。

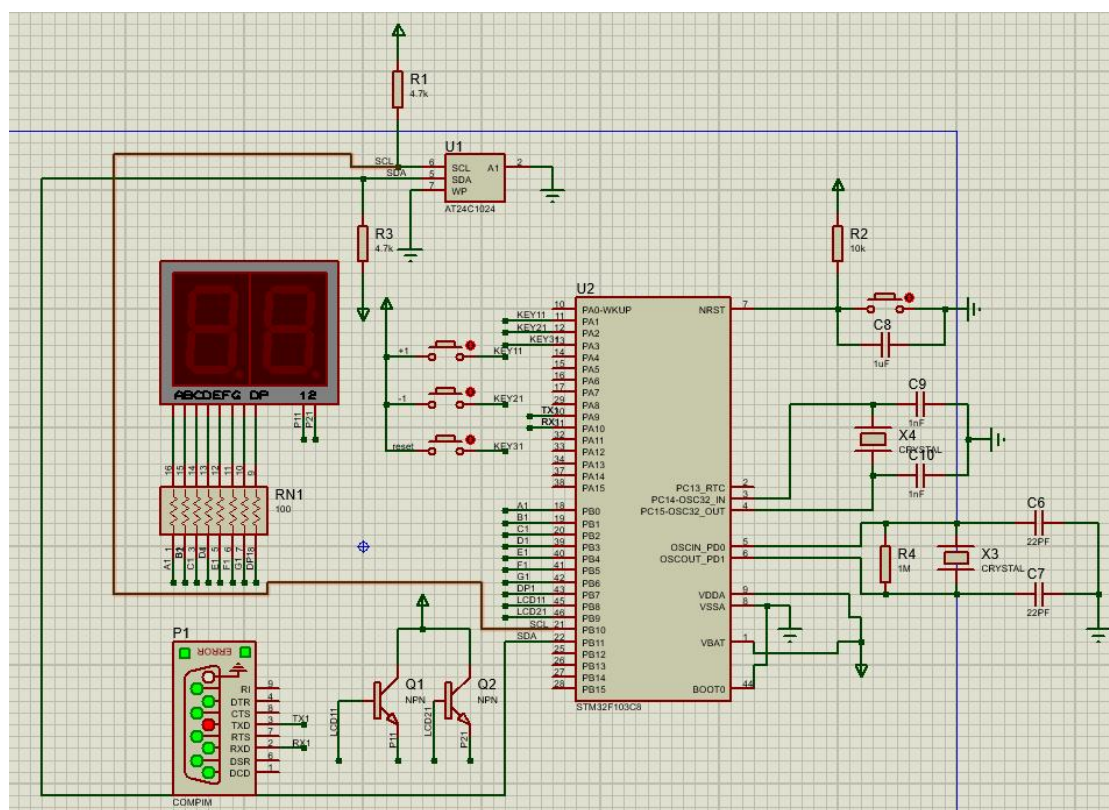


图 2. 实验 7 电路原理图

3.2.5 电路基础功能描述

任务一：LED 数码管显示驱动。

使用 STM32CubeMX 配置 2 位共阳极数码管所需的 GPIO 引脚（段选数据线 PB0~PB7，位选控制线 PB8、PB9），均配置为推挽输出模式。自主编写 display 驱动模块，包含共阳极数字段码表、初始化函数、写数据函数、清屏函数，实现 0~99 数字显示与前导零抑制功能。在 Proteus 中搭建数码管电路，通过两个 NPN 三极管分别控制十位和个位的位选通断，实现动态扫描显示。

任务二：按键控制代码编写。

通过外部中断检测三个按键，实现计数加、减和清零功能。三个按键分别配置为 PA1（加计数，上升沿触发）、PA2（减计数，上升沿触发）、PA3（清零，上升沿触发），均采用内部下拉输入模式。在 EXTI 中断回调函数中更新计数值，并调用数码管显示函数实时刷新。按键按下时，计数数值变化，数码管同步更新显示，实现实时计数控制。

任务三：USART 通信与仿真联调。

配置 USART1 异步通信模式（波特率 9600、8 位数据位、1 位停止位、无校验），编写串口发送函数，当触发加、减、清零按键时，分别发送字符串“ADD\r\n”、“SUB\r\n”、“CLR\r\n”至上位机。编写串口接收中断函数，接收上位机发送的数字（如“5”），解析后将数码管显示为对应数字。使用 Virtual Serial Port Driver 创建虚拟串口对，在 Proteus 中添加 COMPIM 组件与单片机交叉连接，配合 XCOM 串口助手完成收发测试，验证串口远程控制计数器的功能。

3.2.6 电路拓展功能描述

在基础功能（双 MCU 串口通信、数码管动态显示、OLED 同步显示）之上，新增两项核心拓展功能，显著提升系统交互性与易用性：

串口远程设置计数值：上位机通过串口调试助手发送任意两位数字（如 05、23、99），MCU#1 接收后解析并将该数字直接显示在数码管上，同时通过 UART 同步发送给 MCU2，使 OLED 显示同步更新。该功能弥补了仅能通过按键逐次加减的局限，可实现任意数值的快速跳转设定。

长按按键连加/连减：对 KEY1（加）和 KEY2（减）增加长按检测机制。当按键按下时间超过 500ms 后，进入连发模式，每 200ms 自动执行一次加/减操作，直至松开按键。连加/连减过程中数码管和 OLED 同步刷新，且速度适中（约 5 次/秒），兼顾操作效率与显示可读性。

拓展功能无需新增硬件，完全通过软件逻辑实现，充分利用现有按键和串口外设，体现了嵌入式系统“软硬件协同”的设计思想。

3.3 传送带控制器电路设计

3.3.1 传送带控制器电路设计概述

传送带控制器是物流分拣平台中实现物料输送速度动态调节的核心模块。本系统以 STM32F103C8T6 微控制器为控制核心，通过 ADC 采集滑动变阻器的模拟电压，经线性映射后生成对应占空比的 PWM 信号，驱动 L298N 电机驱动器实现对直流电机转速的精确控制。同时，系统集成 USART 串口通信，支持上位机指令远程调速与模式切换，并配备 OLED 显示屏实时反馈运行状态。电路设计兼顾模拟量采集、功率驱动、人机交互与通信接口，形成完整的闭环调速系统。

3.3.2 引脚规划和功能分配

引脚编号	功能名称	方向	说明
PA5	ADC1_IN5	输入	滑动变阻器分压输入，0~3.3V 模拟量
PA0	TIM2_CH1 (PWM)	输出	PWM 信号输出至 L298N 的 ENA 使能端
PA1	IN1	输出	L298N 方向控制引脚 1
PA2	IN2	输出	L298N 方向控制引脚 2
PA3	OLED_SCL	输出	I2C 时钟线（模拟 I2C）
PA4	OLED_SDA	输出	I2C 数据线（模拟 I2C）
PA9	USART1_TX	输出	串口发送，上报状态信息
PA10	USART1_RX	输入	串口接收，接收远程控制指令

表 2. 传送带控制电路引脚规划和功能分配

3.3.3 元器件选型

（1）STM32F103C8T6 微控制器 × 1：基于 ARM Cortex-M3 内核，主频最高 72MHz，64KB Flash，20KB SRAM，LQFP48 封装，作为系统的核心控制单元。

（2）L298N 电机驱动芯片 × 1：双 H 桥直流电机驱动器，ENA 使能端接收 PWM 信号控制电机转速，OUT1/OUT2 连接直流电机，内置续流二极管保护电路。

（3）直流调速电机（MOTOR） × 1：通过 L298N 电机驱动芯片控制转速。

（4）OLED12864I2C 显示屏 × 1：128×64 像素分辨率，SSD1306 控制器，I2C 通信接口，SCL 接 PA3、SDA 接 PA4，用于实时显示电机速度信息。

（5）可调电位器（POT-HG，1kΩ） × 1：一端接 3.3V 电源、一端接地，滑动端连接 PA5（ADC1_IN5），通过分压原理为 ADC 提供 0~3.3V 可调模拟输入。

（6）1N4148 续流二极管 × 4：并联在 L298N 输出端与电机之间，用于抑制电机绕组反电动势，保护驱动芯片。

（7）电阻（100Ω、10kΩ）、电容（22pF、12pF、1μF）、8MHz 晶振、32.768kHz 晶振若干：构成 STM32 最小系统外围电路。

（8）COMPIM 虚拟串口组件 × 1：Proteus 中的串口接口模型，将仿真电路中的 UART 信号连接到计算机的虚拟 COM 口。

（9）VSPD 虚拟串口软件 × 1：在计算机上创建虚拟串口对，实现上位机与 Proteus 仿真之间的串口数据通道。

（10）串口调试助手（XCOM） × 1：运行于上位机，通过虚拟串口与 Proteus 中的下位机进行串口数据收发。

3.3.4 电路原理图

传送带控制器电路原理如图所示。STM32F103 的 PA5 引脚连接电位器滑动端，电位器固定端分别接 3.3V 和 GND，构成模拟分压电路。PA0 输出 PWM 信号连接至 L298N 的 ENA 引脚，PA1 和 PA2 分别连接 IN1 和 IN2，控制电机正反转。L298N 的 OUT1 和 OUT2 连接直流电机，并在输出端并联四个续流二极管。OLED 显示屏通过 PA3（SCL）和 PA4（SDA）与 MCU 连接，采用 I2C 通信。USART1 通过 COMPIM 连接至 PC 虚拟串口，供上位机调试。

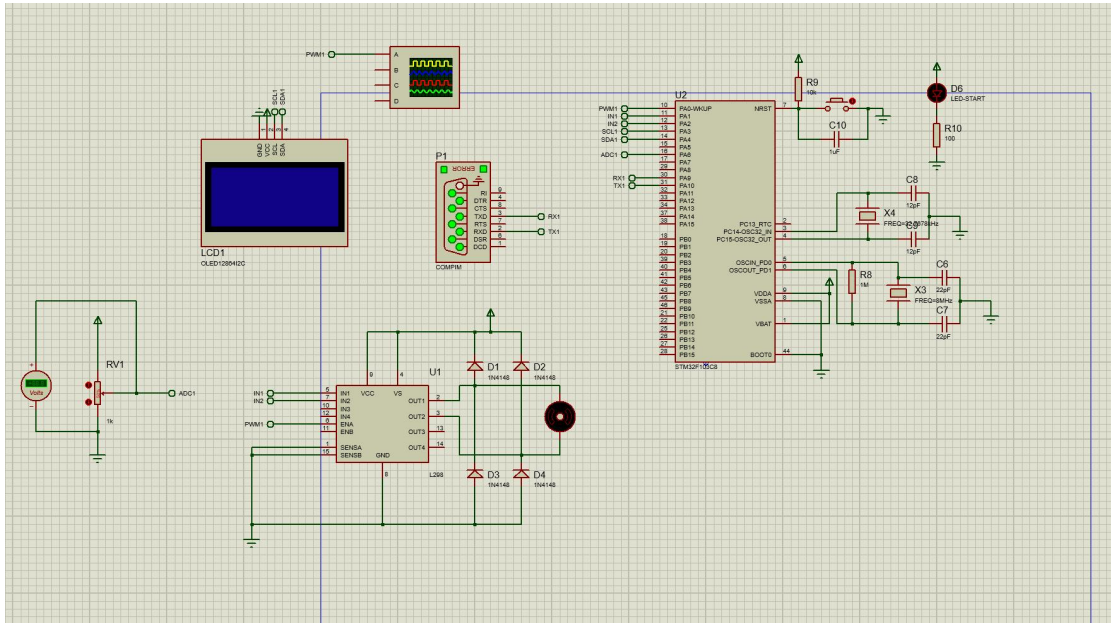


图 3. 实验 8 电路原理图

3.3.5 电路基础功能描述

任务一：ADC 采集与 PWM 控制代码编写

使用 STM32CubeMX 配置系统时钟为 72MHz (HSE)，配置 ADC1 选择 PA5 模拟输入引脚，设置为单次转换模式，分辨率 12 位，采样周期 55.5 周期。配置定时器 2 的通道 1 为 PWM 输出模式，设置预分频器和自动重装值，使 PWM 频率为 1kHz。在生成的代码基础上，在 main 函数的主循环中添加 ADC 采集和 PWM 占空比更新逻辑，将 ADC 采样值 (0~4095) 线性映射为 PWM 占空比 (0~100%)。

任务二：电机驱动电路搭建与仿真测试

在 Proteus 中使用 L298N 构建电机驱动电路，通过滑动变阻器改变 ADC 采样值以改变 PWM 占空比，从而改变电机转速。传送带电机速度信息在 OLED 显示屏上实时显示。观察虚拟示波器显示的 PWM 波形，调节滑动变阻器，验证 PWM 占空比是否随输入电压线性变化。

任务三：USART 通信与双模式控制

编写串口发送函数，通过 USART 发送电机速度信息。编写串口接收函数，通过 USART 接收串口助手发送的控制信息。接收到信息后，ADC 数据被暂时屏蔽，电机速度控制依赖于串口助手发送的数据；当拨动滑动变阻器时，ADC 数值改变，电机速度控制切换回滑动变阻器的 ADC 采样数值。传送带电机速度信息在 OLED 显示屏上实时显示。在 Virtual Serial Port Driver 配置虚拟串口，使用串口助手查看发送和接收的信息。

3.3.6 电路拓展功能描述

在基础 ADC 调速与串口指令调速之上，新增三项核心拓展功能，提升系统可控性与用户体验：

(1) **10 档位精细转速控制**：将 PWM 占空比 (0~100%) 均匀划分为 10 个档位 (每档 10%)，可通过串口发送 SPD+ 或 SPD- 指令实现档位步进 (加档/减档)，亦可发送 SPD X (X=0~10) 直接跳转到指定档位。ADC 调速模式下，采样值同样映射为 10 档，并平滑过渡。

(2) **OLED 实时显示转速档位与百分比**：OLED 屏幕第一行固定显示“传送带控制器”，第二行显示当前控制模式 (ADC 或 UART)，第三行显示当前档位 (GEAR: 5/10) 和对应的 PWM 占空比 (DUTY: 50%)，第四行显示电机转向 (DIR: F 或 DIR: R)。所有信息随调速或换向操作实时刷新。

(3) **缓增缓减 (平滑变速) 与正反转控制**：当档位或占空比目标值发生变化时，电机转速不直接跳变，而是以每 50ms 步进 1% 的速度逐步逼近目标值，实现“缓加速、缓减速”效果，有效避免电流冲击和机械抖动。通过串口发送 DIR F 切换为正转，发送 DIR R 切换为反转。改变转向时，先停止 PWM 输出，切换 GPIO 方向 (IN1/IN2 电平)，再按当前档位平滑启动，确保换向安全。

以上拓展功能完全基于软件实现，无需增加硬件元件，充分利用现有 GPIO、PWM、ADC、USART 及 OLED 资源。

3.4 分拣器控制器电路设计

3.4.1 分拣器控制器电路设计概述

分拣器控制器是物流分拣平台中实现货物自动识别与分类决策的核心模块。本系统以 STM32F103C8T6 微控制器为控制核心,通过 USART 串口接收来自视觉识别模块(本实验中以教师提供的摄像头控制软件模拟 K210 视觉芯片)的分拣信息,在 OLED12864 显示屏上实时显示当前识别的货物类别与分拣状态。电路设计遵循“前端视觉感知+后端控制执行”的异构系统通信架构,兼顾串口通信的实时性与显示交互的直观性,为后续扩展机械臂抓取等执行机构预留接口。

3.4.2 引脚规划和功能分配

引脚编号	功能名称	方向	说明
PA6	OLED_SCL	输出	I2C 时钟线(模拟 I2C)
PA7	OLED_SDA	输出	I2C 数据线(模拟 I2C)
PA9	USART1_TX	输出	串口发送,上报分拣状态
PA10	USART1_RX	输入	串口接收,接收视觉识别结果
PB0	LED_STATUS	输出	分拣状态指示灯
PB1	LED_ERROR	输出	异常报警指示灯
PB10	BUZZER	输出	蜂鸣器控制(分拣完成提示)

表 3. 分拣器控制器电路引脚规划和功能分配

3.4.3 元器件选型

- (1) STM32F103C8T6 微控制器 × 1: 基于 ARM Cortex-M3 内核,主频最高 72MHz, 64KB Flash, 20KB SRAM, LQFP48 封装,作为系统的核心控制单元。
- (2) OLED12864I2C 显示屏 × 1: 128×64 像素分辨率, SSD1306 控制器, I2C 通信接口, SCL 接 PA6、SDA 接 PA7,用于实时显示识别到的货物类别、分拣状态及统计数据。
- (3) COMPIM 虚拟串口组件 × 1: Proteus 中的串口接口模型,将仿真电路中的 UART 信号连接到计算机的虚拟 COM 口,实现 Proteus 仿真与上位机视觉模拟软件的数据交互。
- (4) LED 指示灯 × 2: 红色 LED (PB1) 用于异常报警指示,绿色 LED (PB0) 用于分拣完成状态指示。
- (5) 蜂鸣器 × 1: PB10 控制,分拣完成时发出提示音。
- (6) VSPD 虚拟串口软件 × 1: 在计算机上创建虚拟串口对,实现上位机视觉模拟软件与 Proteus 仿真之间的串口数据通道。
- (7) 摄像头控制软件(教师提供) × 1: 运行于上位机,模拟 K210 视觉芯片的识别功能,通过虚拟串口向下位机发送识别结果。

3.4.4 电路原理图

分拣器控制器电路原理如图所示。STM32F103 的 PA6 和 PA7 引脚连接 OLED12864 的 SCL 和 SDA,采用模拟 I2C 通信。PA9 (USART1_TX) 和 PA10 (USART1_RX) 通过 COMPIM 组件连接至 PC 的虚拟串口,与上位机视觉模拟软件进行数据交互。PB0 和 PB1 分别连接绿色和红色 LED 指示灯, PB10 连接蜂鸣器。电路设计简洁,预留了足够 GPIO 资源供后续扩展机械臂控制等功能。

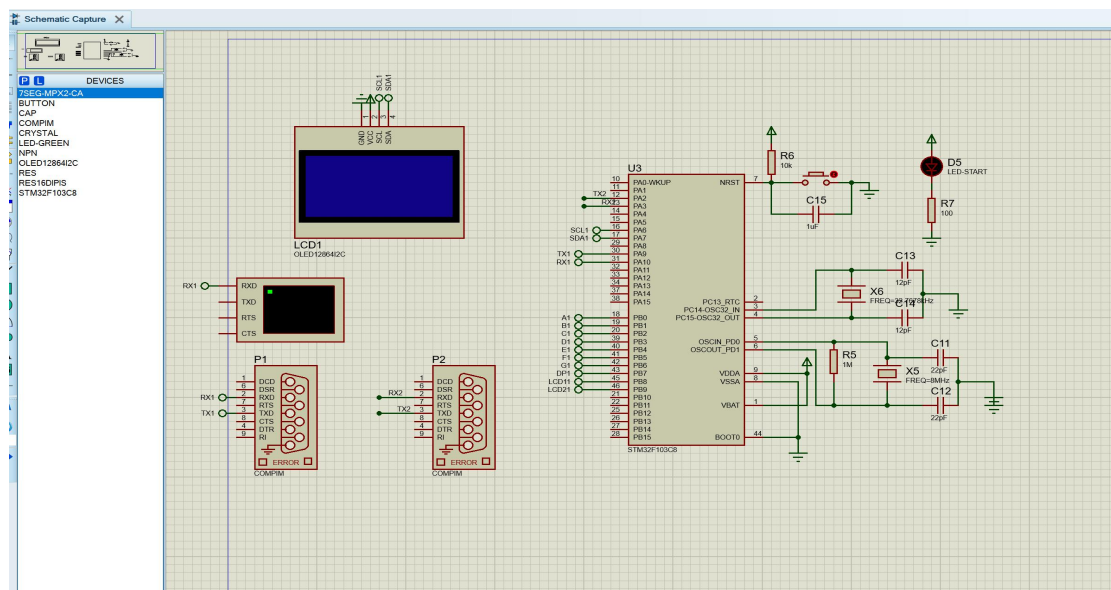


图 4. 实验 9 电原理图

3. 4. 5 电路基础功能描述

任务一：图形点阵 OLED12864 显示驱动

使用 STM32CubeMX 配置系统时钟为 72MHz（HSE），配置 PA6（SCL）和 PA7（SDA）为开漏输出模式并使能内部上拉，User Label 分别命名为 SCL 和 SDA。导入 OLED 驱动模块（OLED.h、OLED_Font.h、OLED.c），实现基于软件 I²C 协议的 OLED 初始化、写命令、写数据、清屏等函数。使用 PCtoLCD2002 字模提取软件提取 16×16 点阵汉字数据，将字模数组存入源文件，编写汉字显示函数，将点阵数据写入 OLED 相应位置。在 Proteus 中添加 OLED12864I2C 元件，按引脚连接电路，仿真验证汉字显示效果。

任务二：USART 通信与分拣信息接收

制定分拣规则：系统预定义若干货物类别，每种货物对应一个识别编号。编写 USART1 异步通信初始化函数（波特率 9600、8 位数据位、1 位停止位、无校验），使能接收中断。编写串口接收回调函数，当接收到上位机视觉模拟软件发送的分拣信息后，解析数据并更新当前货物类别。OLED 显示屏实时显示当前识别的货物类别及分拣状态。在 Proteus 中添加 COMPIM 组件，配置虚拟串口号（如 COM3），使用 VSPD 创建虚拟串口对，配合上位机摄像头控制软件完成串口收发测试，验证视觉识别结果到分拣器显示的数据链路。

3. 4. 6 电路拓展功能描述

在基础 OLED 显示驱动与 USART 通信接收之上，新增四项核心拓展功能，显著提升系统交互性与实用性：

（1）多货物分类与分拣计数统计

系统预定义多种货物类别，每种货物对应唯一识别编号。当串口接收到视觉识别结果时，系统不仅更新当前显示货物，还自动累加该类货物的分拣计数。OLED 屏幕分区显示：第一行固定标题“分拣控制器”，第二行显示当前识别货物类别，第三行显示该类别累计分拣数量，第四行显示总处理数量。用户可通过串口发送“RESET”指令一键清零所有统计数据。

（2）分拣动画与进度条显示

当识别到货物并完成分拣决策时，OLED 屏幕上的货物图标以动态闪烁方式呈现（利用定时器交替显示/隐藏字模），持续时间约 500ms 后恢复常亮，模拟“分拣动作”的视觉反馈。同时，在屏幕底部绘制一条 8 像素高的进度条，每处理一件货物进度条向右填充一格（共 20 格），满格后自动折返，直观反映系统处理节奏。

（3）通信协议标准化（帧头+货物编号+CRC+帧尾）

设计私有通信协议，数据帧格式为：0xAA（帧头）+ 货物编号（1 字节）+ CRC8 校验（1 字节）+ 0x55（帧尾）。货物编号定义：0x01 苹果、0x02 香蕉、0x03 橙子、0x04 葡萄、0x05 西瓜。STM32 接收数据时，先检测帧头 0xAA 和帧尾 0x55，再对数据部分进行 CRC8 校验，校验通过后才更新显示和计数，校验失败则丢弃该帧并点亮红色 LED 报警指示灯。该协议有效避免了串口通信中的数据错乱和误触发问题。

(4) 蜂鸣器提示与状态指示灯联动

分拣完成时，蜂鸣器发出“嘀”一声短提示音（PB10 输出 500ms 高电平），绿色 LED 同步闪烁一次。当收到无法识别的货物编号或 CRC 校验失败时，蜂鸣器发出“嘀嘀”两声短促报警音，红色 LED 常亮直至收到下一条有效指令。通过声光组合反馈，增强了系统的人机交互友好性，模拟了真实工业分拣场景中的状态提示机制。

第 4 章 软件设计

4.1 光电计数器软件设计

4.1.1 光电计数器软件设计概述

计数器软件采用模块化分层设计，包含驱动层（GPIO、USART、EXTI）、硬件抽象层（数码管显示、按键扫描）和应用层（计数逻辑、协议解析）。整个工程在 Keil MDK 环境下开发，使用 STM32CubeMX 生成初始化代码，HAL 库提供外设访问接口。

4.1.2 程序流程图

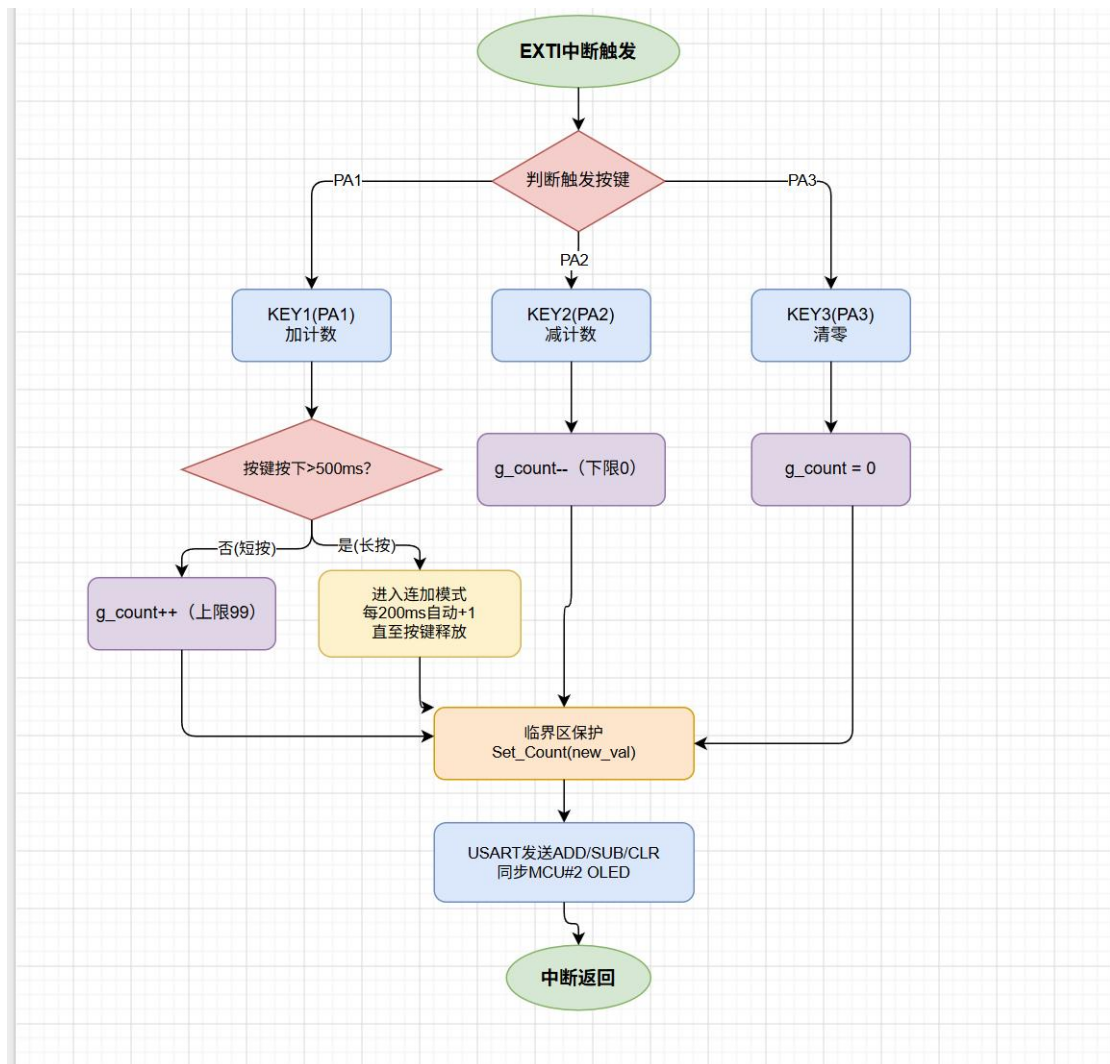


图 5. 实验 7 主程序整体流程

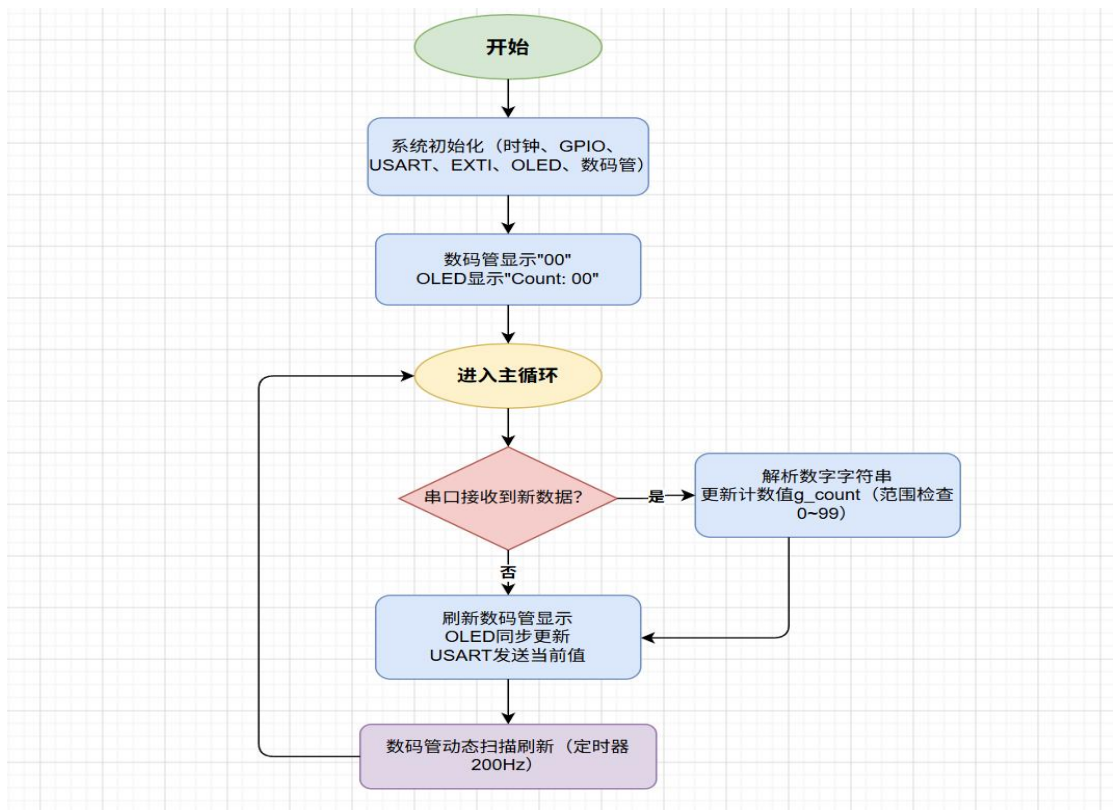


图 6. 实验 7 按键中断处理

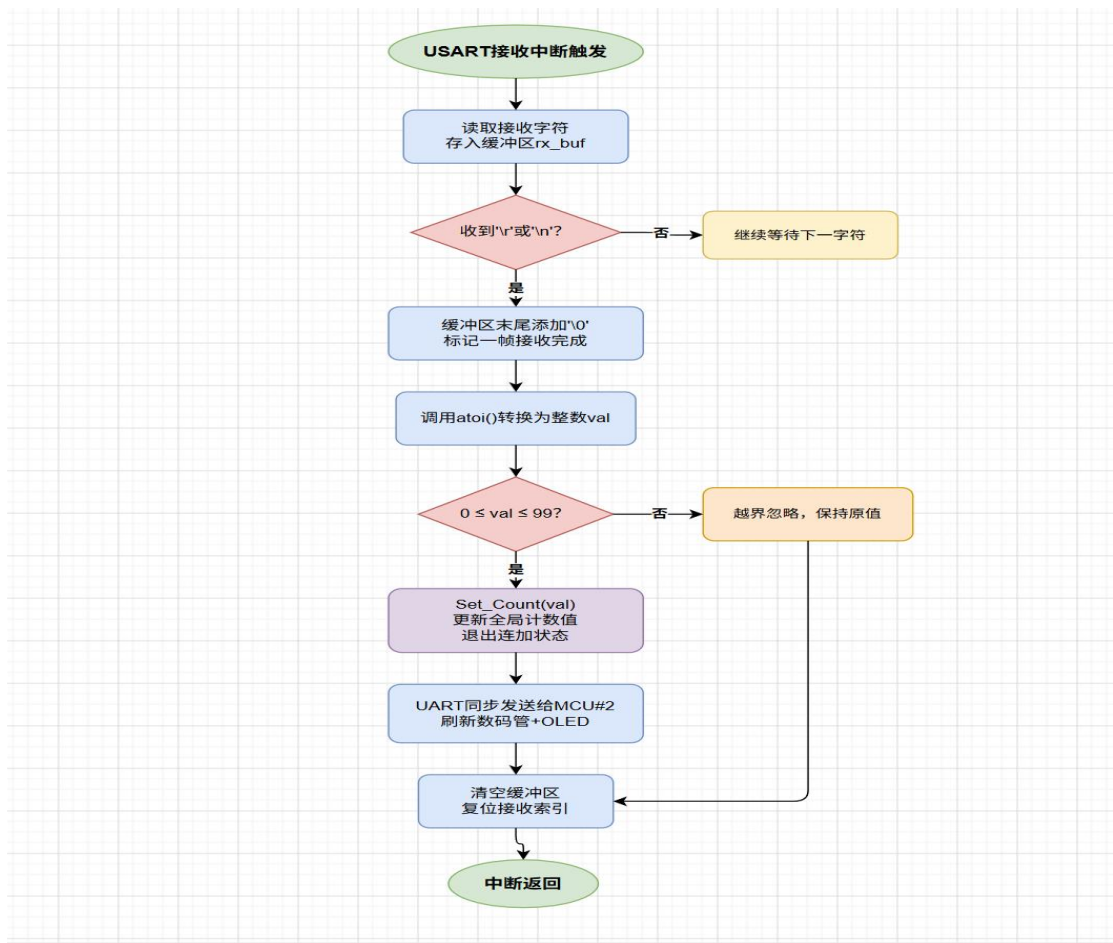


图 7. 实验 7 串口接收解析

4.1.3 模块化编程设计



图 8. 实验 7 模块化编程设计

各模块间调用关系如下：

- main() 调 HAL_Init()、SystemClock_Config()、MX_GPIO_Init()、MX_USART1_Init() 以及 Display_Init() 完成系统初始化，随后启动定时器（用于数码管扫描）并进入主循环。
- GPIO 模块中的 HAL_GPIO_EXTI_Callback() 根据触发引脚（PA1/PA2/PA3）调用 ISR_Debounce() 进行防抖处理，然后更新全局计数值 g_count，并触发 Display_Update() 刷新数码管，同时调用 USART_SendCount() 将新值发送至串口。
- USART 模块的接收中断将接收到的字符存入环形缓冲区，当检测到帧结束符时，调用 process_usart1_data() 解析指令（支持单字符或两位数字），若为数字则调用 Set_Count() 更新计数值并刷新显示。
- 拓展功能“长按连加/连减”在按键中断中利用系统滴答时钟 HAL_GetTick() 实现计时，不额外占用定时器资源；连发过程中每触发一次即调用 Count_Add()/Count_Sub()，与普通按键共用计数更新函数。

4.1.4 关键函数接口定义

```
// display.h
void Display_Init(void);           // 初始化GPIO并清屏
void Display_Update(uint8_t num);  // 显示0~99数字（前导零抑制）
void Display_Clear(void);          // 熄灭所有段

// gpio.h
void MX_GPIO_Init(void);           // GPIO及EXTI配置
uint8_t ISR_Debounce(uint16_t GPIO_Pin); // 防抖处理，返回有效触发

// usart.h
void MX_USART1_Init(void);         // 串口参数配置
void USART_SendString(char *str);  // 发送字符串
void process_usart1_data(void);     // 解析接收帧，执行远程指令

// main.c
extern uint8_t g_count;             // 全局计数值（0~99）
void Set_Count(uint8_t new_val);    // 统一计数修改入口（含临界区保护）
void Count_Add(void);               // 加1（自动限幅）
void Count_Sub(void);               // 减1（自动限幅）
```

图 9. 实验 7 关键函数接口定义

4.1.5 拓展功能软件实现

- 串口远程设置计数值：上位机发送两位数字字符串（如“51”），USART 接收中断缓存完整帧后，由 `process_usart1_data()` 调用 `atoi()` 转换，并检查范围（0~99），有效则调用 `Set_Count()` 更新。该功能弥补了按键逐次操作的不足，可快速跳转至任意值。

- 长按按键连加/连减：在 EXTI 中断回调中记录按键按下时刻，若持续时间超过 500ms，则每隔 200ms 自动执行一次加减，直至松开。实现代码利用全局变量记录连加状态，不阻塞其他中断，并确保松开即停。

4.1.6 工程文件结构

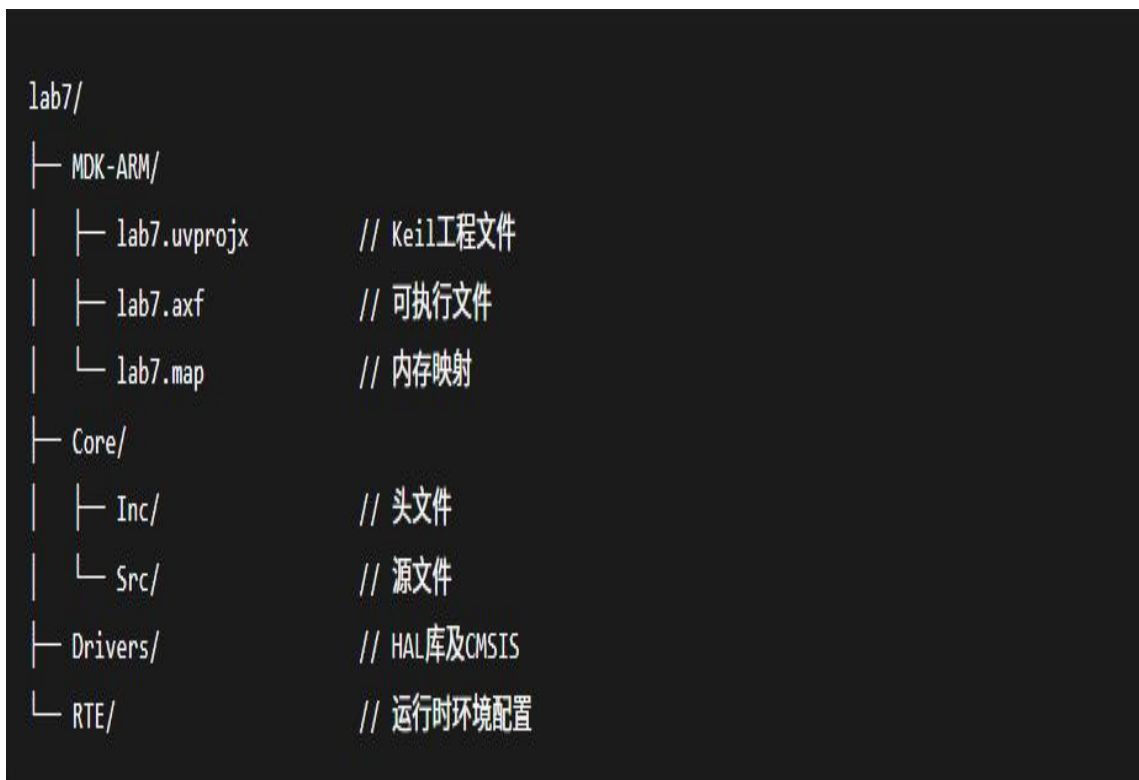


图 10. 实验 7 工程文件结构

4.2 传送带控制器软件设计

4.2.1 传送带控制器软件设计概述

传送带控制器软件采用模块化分层设计，包含驱动层（ADC、TIM PWM、USART、GPIO）、硬件抽象层（OLED 显示、电机驱动封装）和应用层（调速逻辑、指令解析、模式管理）。软件基于 STM32 HAL 库开发，在 Keil MDK 环境下编写，支持 ADC 模拟调速与 UART 数字指令调速两种模式，并实现平滑变速、档位管理和正反转控制。代码结构清晰，兼顾实时性与可扩展性。

4.2.2 程序流程图

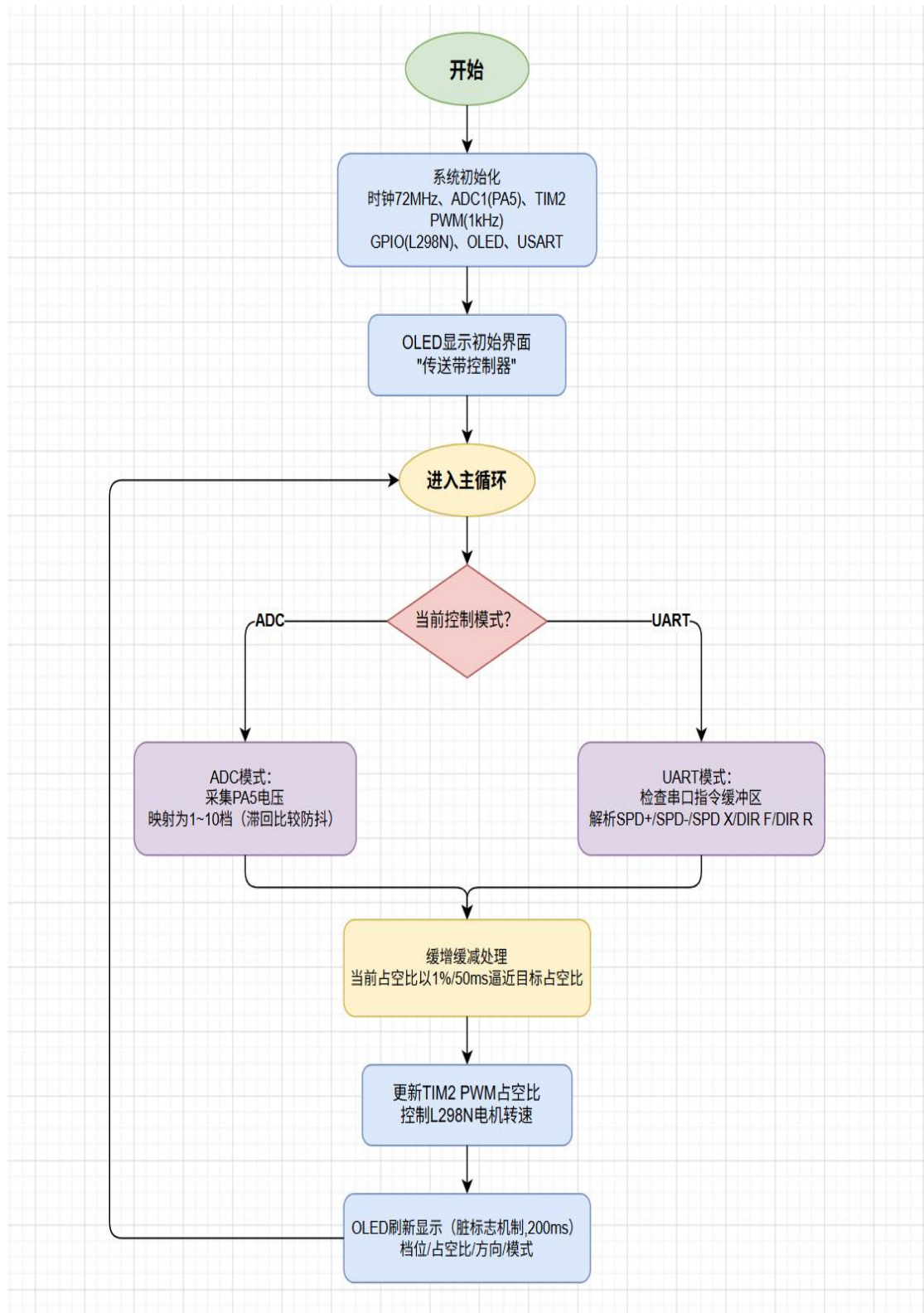


图 11. 实验 8 主程序整体控制

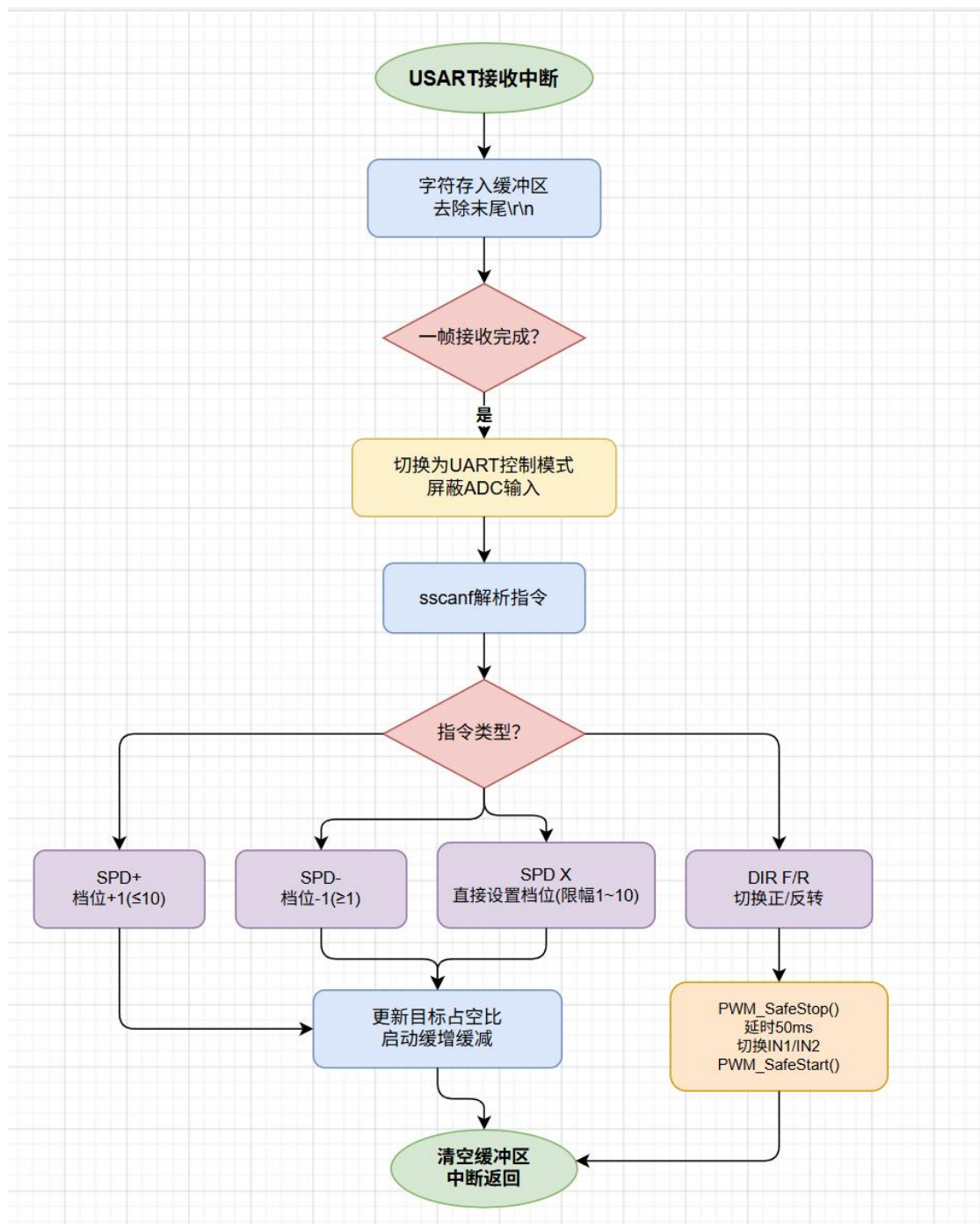


图 12. 实验 8 串口指令解析执行

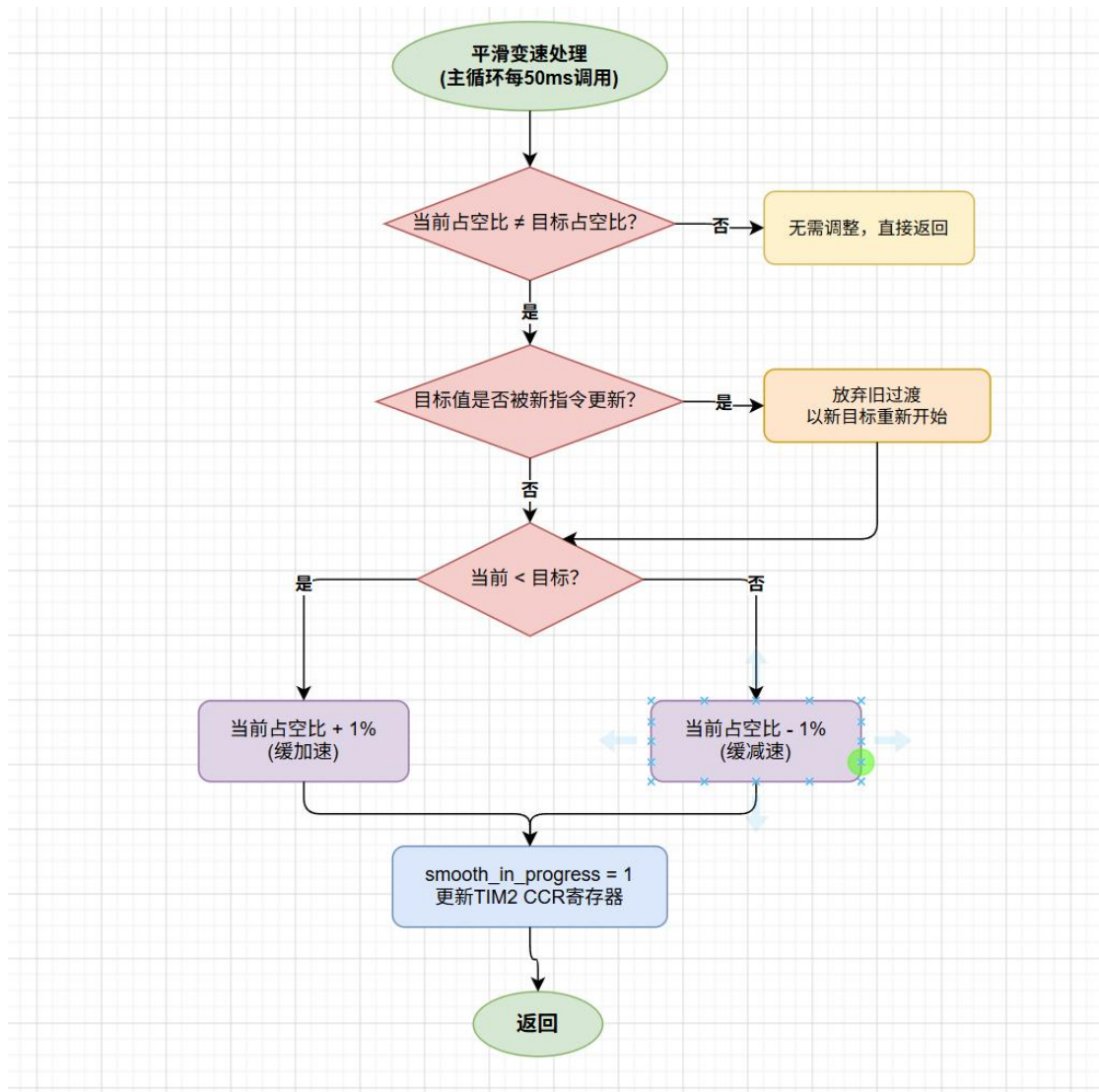


图 13. 实验 8 缓增缓减平滑变化

4.2.3 模块化编程设计



图 14. 实验 8 模块化编程设计（一）

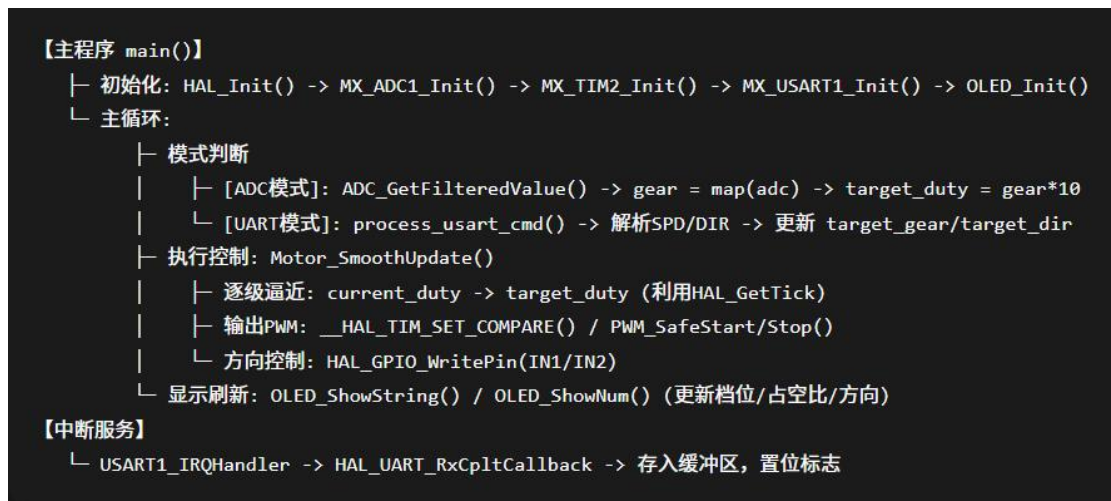


图 15. 实验 8 模块化编程设计（二）

模块间调用关系如下：

- main() 依次调用 HAL_Init()、SystemClock_Config()、MX_GPIO_Init()、MX_ADC1_Init()、MX_TIM2_Init()、MX_USART1_UART_Init() 和 OLED_Init()。
- 主循环中调用 HAL_ADC_Start() 和 HAL_ADC_PollForConversion() 获取 ADC 值，然后调用 Motor_SmoothUpdate() 更新 PWM 占空比，最后调用 OLED 刷新函数更新显示。
- 串口接收中断回调 HAL_UART_RxCpltCallback() 将接收数据存入缓冲区，主循环检测到新指令后调用 process_usart_cmd() 解析并修改控制参数（档位、方向、模式）。
- 平滑更新函数使用全局目标变量和 HAL_GetTick() 实现非阻塞延时步进。

4.2.4 关键函数接口定义

```

/* TIM2 PWM模块 */
void MX_TIM2_Init(void);
void PWM_SetDuty(uint8_t duty_percent); // 0~100
void PWM_SafeStart(void);
void PWM_SafeStop(void);

/* 电机控制 */
void Motor_DirectSet(uint8_t dir, uint8_t gear); // dir: 0停止,1正转,2反转; gear: 0~10
void Motor_SmoothUpdate(void); // 平滑更新, 在主循环中周期性调用
void Motor_SetMode(uint8_t mode); // mode: 0-ADC, 1-UART

/* OLED显示 */
void OLED_Init(void);
void OLED_ShowString(uint8_t x, uint8_t y, char *str);
void OLED_ShowNum(uint8_t x, uint8_t y, uint32_t num, uint8_t len);

/* USART */
void MX_USART1_UART_Init(void);
void USART_SendStatus(uint8_t gear, uint8_t duty, uint8_t dir);
void process_usart_cmd(char *cmd); // 解析SPD+/SPD-/SPD X/DIR F等

```

图 16. 实验 8 关键函数接口定义

4.2.5 拓展功能软件实现

（1）10 档位精细控制

定义全局变量 uint8_t target_gear 和 current_gear。ADC 模式下，将滤波后的 ADC 值（0~4095）映射为 1~10 档：gear = (adc_val * 10) / 4096 + 1，并限幅至 1~10。UART 模式下，解析 SPD+、SPD- 或 SPD X 指令，更新 target_gear。档位与占空比的转换公式为 duty = gear * 10（档位 1 对应 10% 占空比，档位 10 对应 100%）。

(2) 缓增缓减（平滑变速）

在 Motor_SmoothUpdate() 中，比较 current_duty 与 target_duty。若差值绝对值大于 1，则每 50ms 向目标方向步进 1%（利用 HAL_GetTick() 计时）。步进过程若检测到目标值发生变化（如串口新指令），则立即以新目标重新开始平滑。该机制确保转速变化平稳，无突变。

(3) 正反转控制与安全换向

定义全局变量 motor_dir(0 停止, 1 正转, 2 反转)。当收到 DIR F 时置 motor_dir=1, DIR R 置 motor_dir=2。在换向过程中，先调用 PWM_SafeStop() 停止 PWM 输出，延时 50ms 等待电机惯性消退，再设置 IN1/IN2 电平（正转：IN1=H, IN2=L；反转：IN1=L, IN2=H），最后调用 PWM_SafeStart() 按当前档位恢复输出。所有操作均在临界区保护下完成，避免方向与 PWM 冲突。

(4) OLED 多信息实时刷新

定义“脏标志” display_update_needed，当档位、方向或模式发生实际变化时置位。主循环检测到标志后调用 OLED_ShowString 和 OLED_ShowNum 更新对应区域，并限制刷新频率为 200ms 一次，避免屏幕闪烁和 CPU 过度占用。

4.2.6 工程文件结构

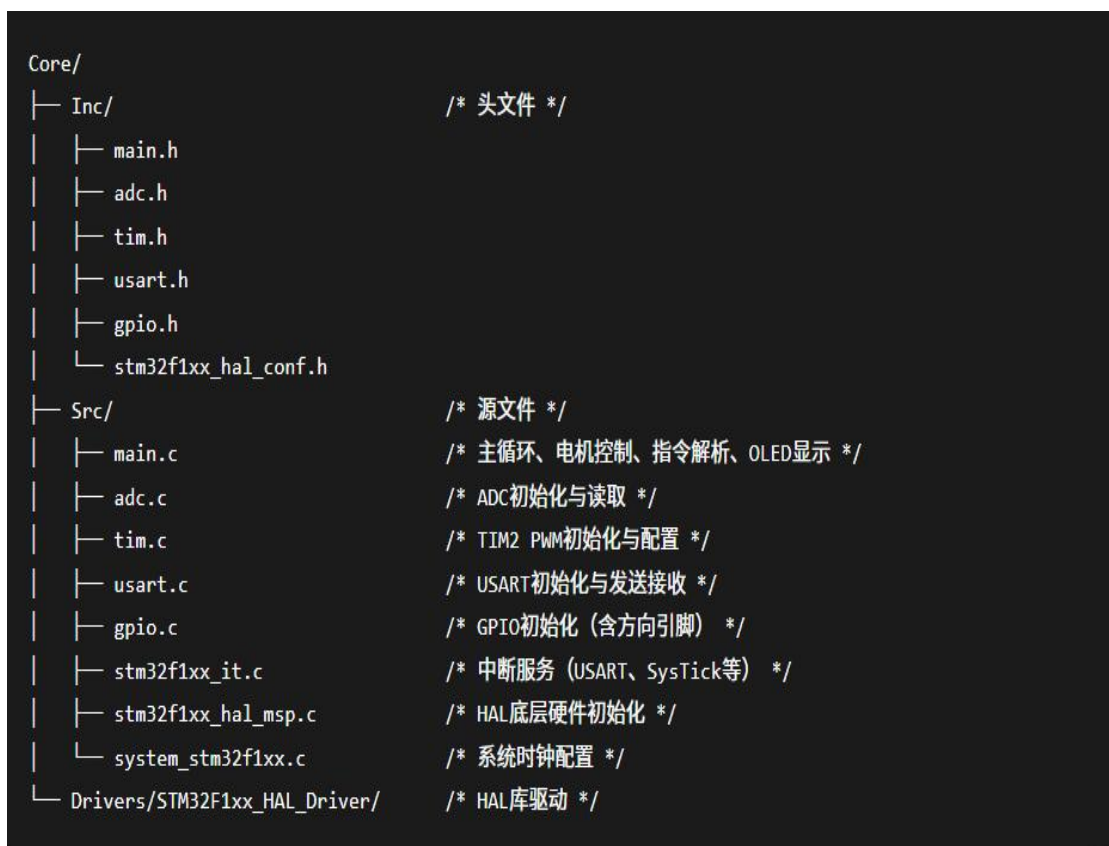


图 17. 实验 8 工程文件结构

通过以上模块化设计，传送带控制器实现了模拟与数字双模式调速、档位管理与平滑变速，并支持拓展功能，为后续物流分拣系统的集成奠定了坚实基础。

4.3 分拣器控制器软件设计

4.3.1 传送带控制器软件设计概述

分拣器控制器软件采用模块化分层设计，包含驱动层（GPIO、USART、TIM）、硬件抽象层（OLED 显示、LED/蜂鸣器控制）和应用层（指令解析、分拣逻辑、计数管理、协议校验）。软件基于 STM32 HAL 库开发，在 Keil MDK 环境下编写，支持通过 USART 接收上位机视觉模拟软件的识别结果，并实时更新 OLED 显示与声光反馈。代码结构清晰，遵循“驱动与业务分离、字模库独立维护”的设计原则。

4.3.2 程序流程图

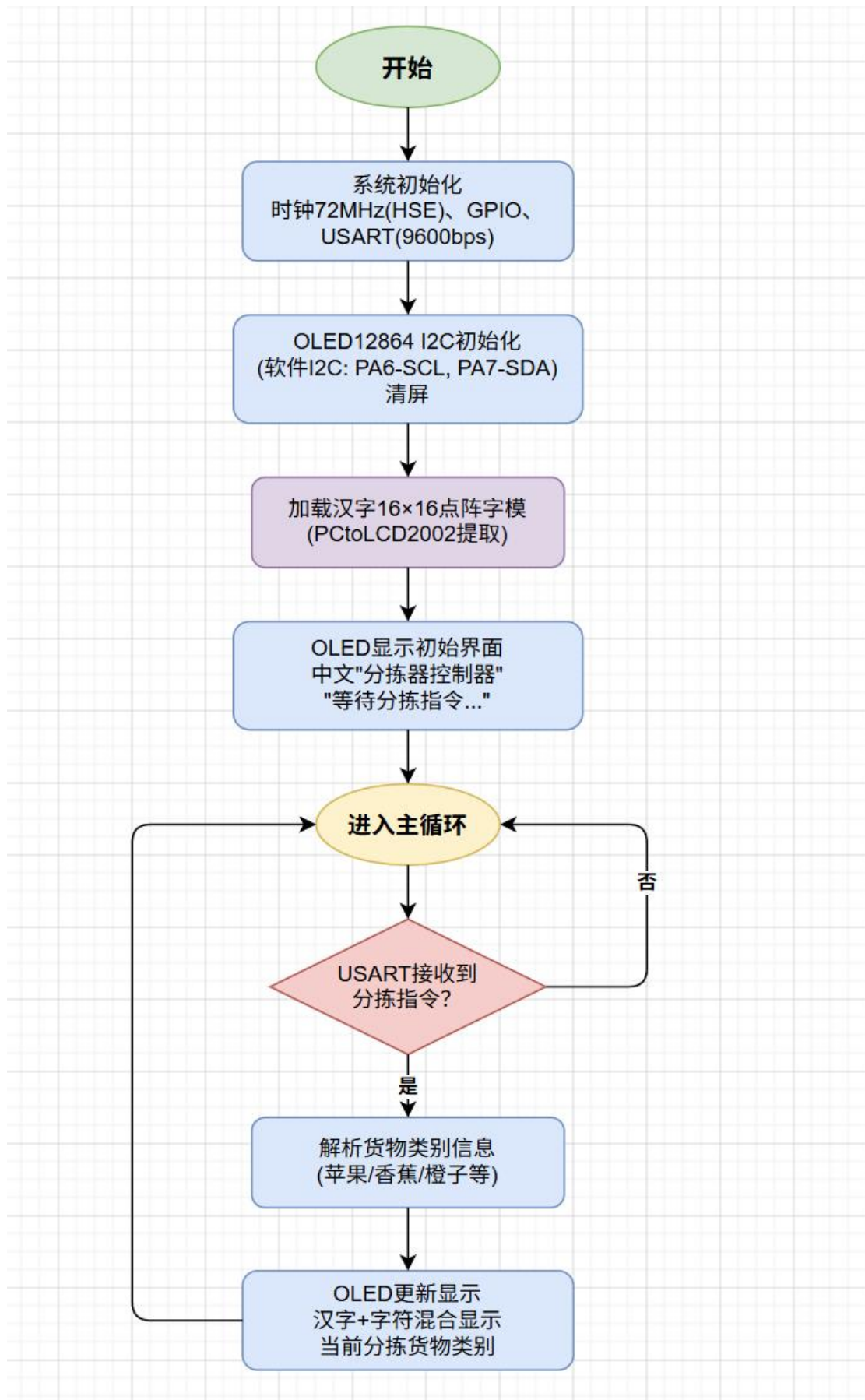


图 18. 实验 9 主程序整体流程

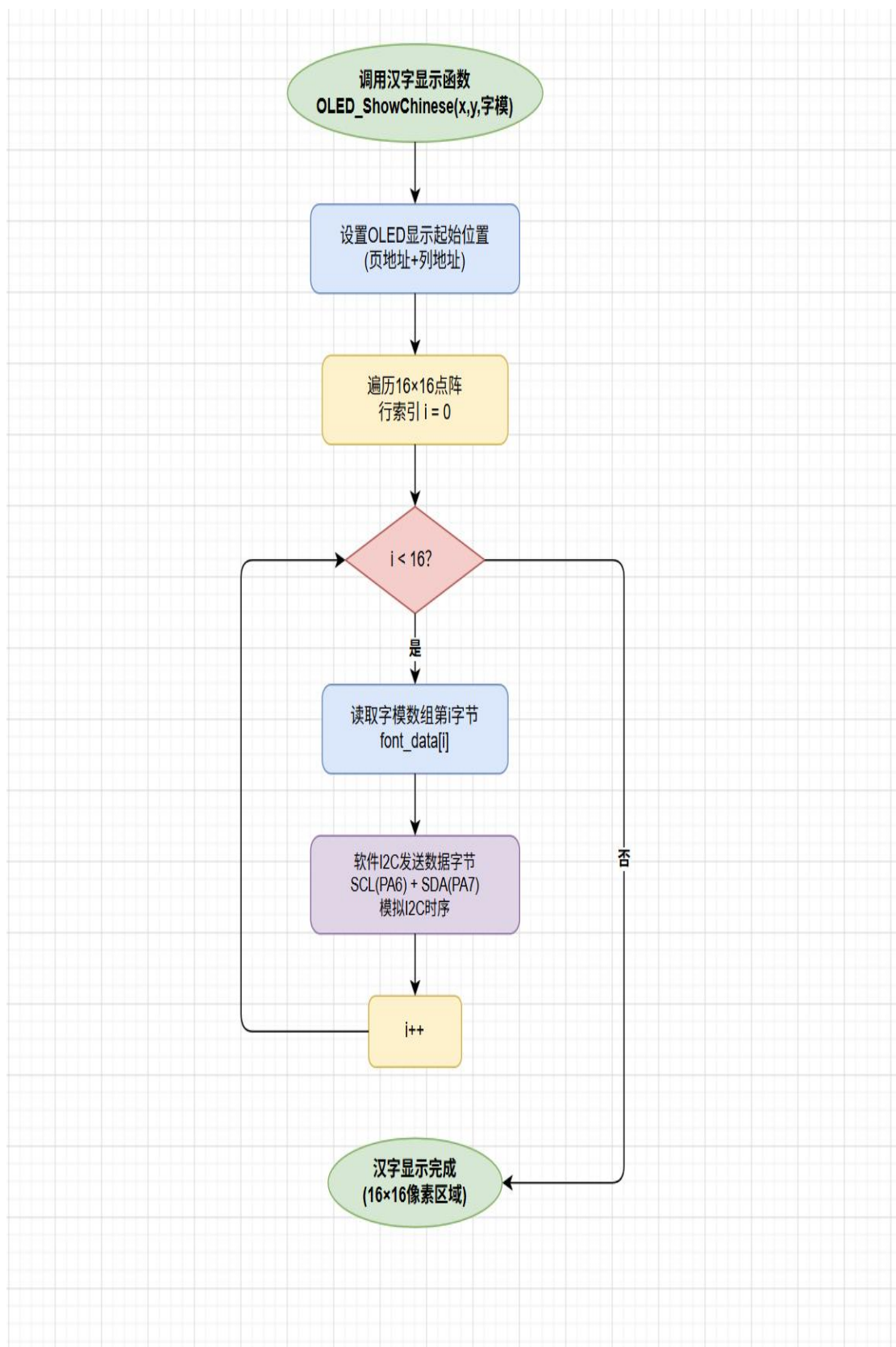


图 19. 实验 9 oled 汉字显示驱动

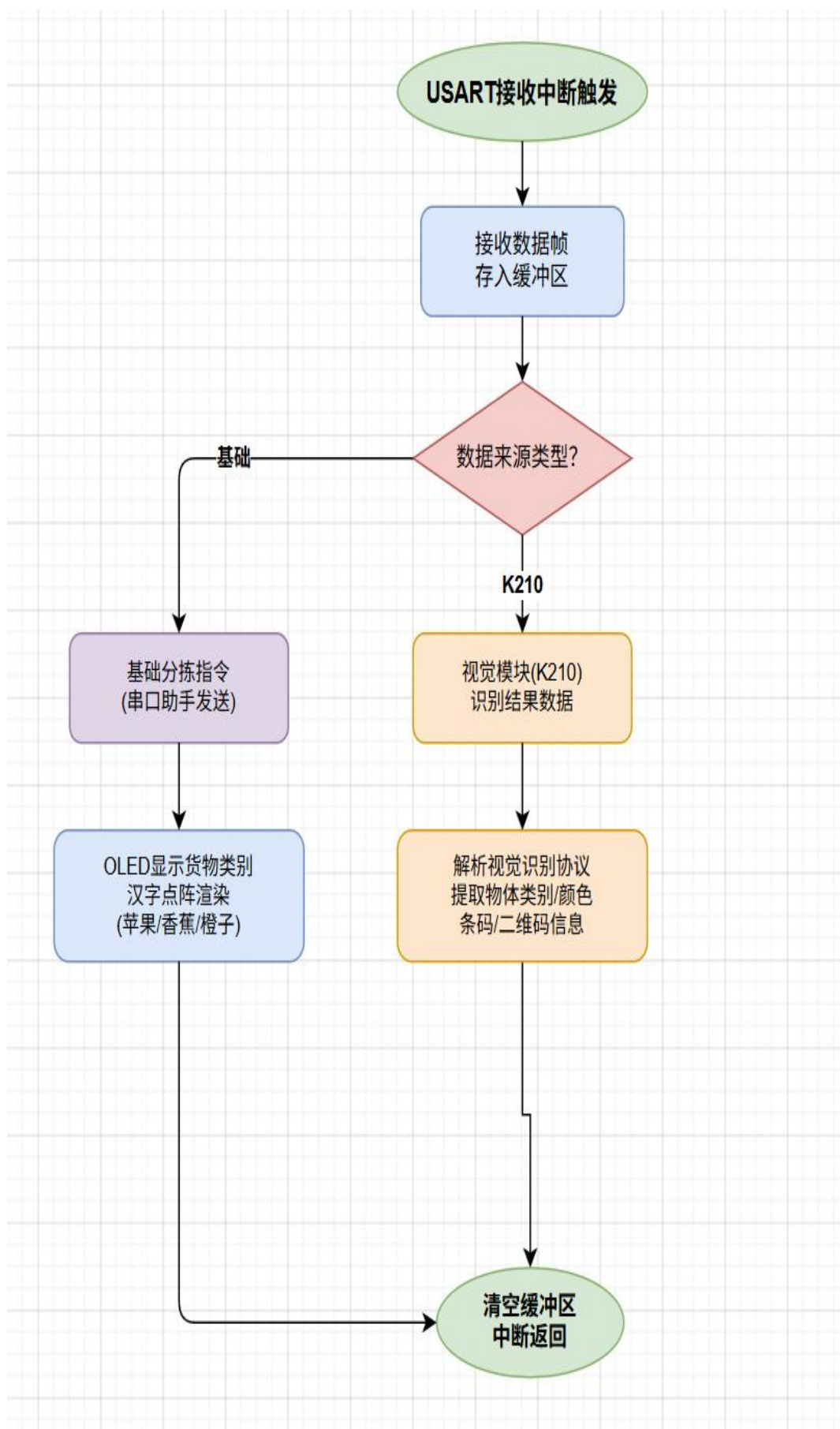


图 20. 实验 9 分拣指令与拓展控制

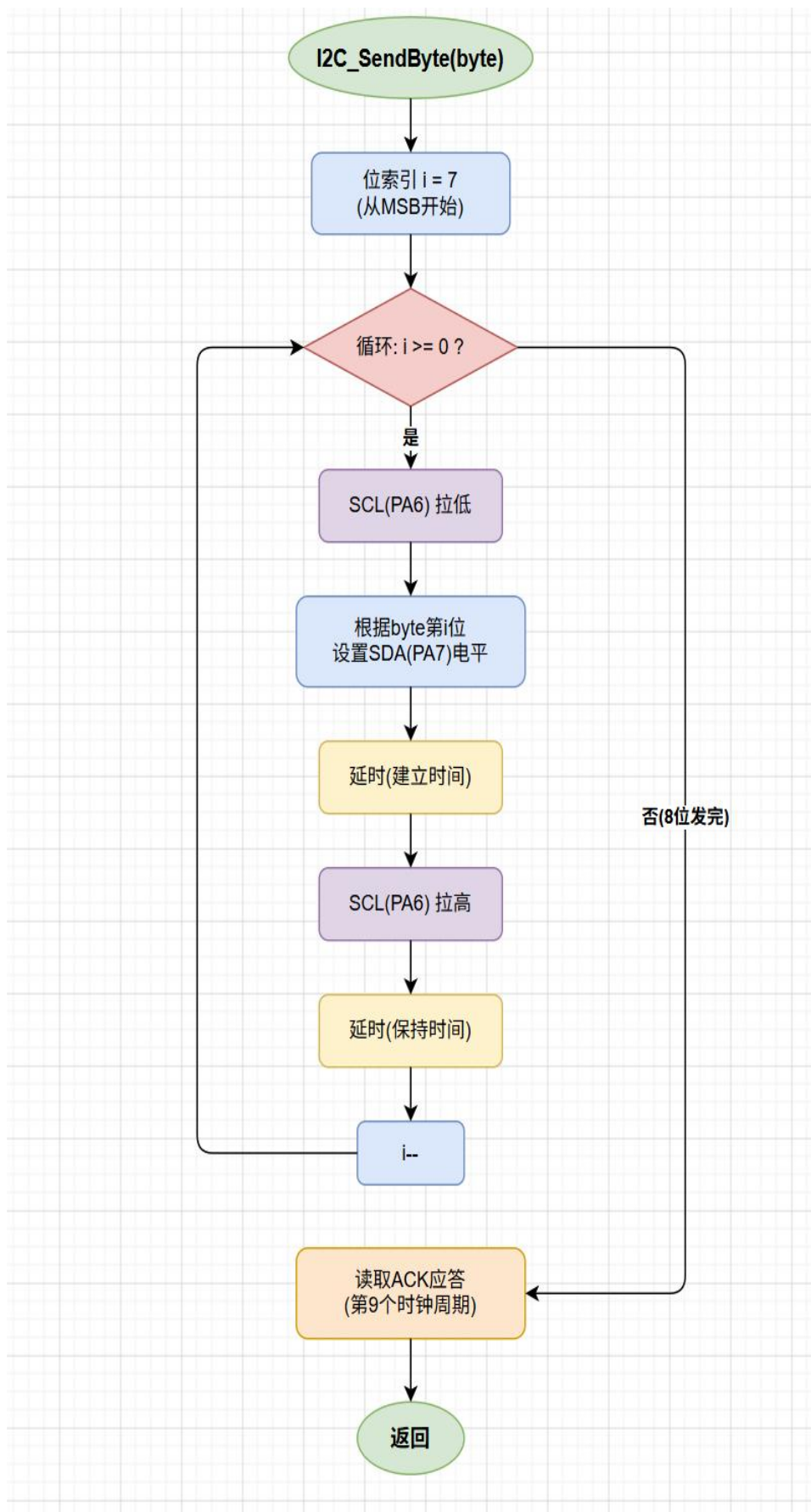


图 21. 实验 9 软件 i2c 时序

4.3.3 模块化编程设计



图 22. 模块化编程设计

模块间调用关系如下：

- main()依次调用 HAL_Init()、SystemClock_Config()、MX_GPIO_Init()、MX_USART1_UART_Init()和 OLED_Init()完成系统初始化，随后启动定时器（用于动画刷新）并进入主循环。
- USART 接收中断回调 HAL_UART_RxCpltCallback()将接收到的字符存入环形缓冲区，当检测到帧结束符时，置位数据就绪标志。
- 主循环检测到数据就绪标志后，调用 ParseFrame()进行帧头检测、CRC 校验和货物编号解析。校验通过后调用 UpdateDisplay()更新 OLED 显示，调用 UpdateCounter()更新对应类别计数，并触发 LED 和蜂鸣器反馈。
- 拓展功能“分拣动画与进度条”在 TIM 定时器中断服务函数中实现，每 100ms 调用一次 Animation_Update()，控制货物图标的闪烁显示和进度条填充。
- 拓展功能“通信协议标准化”在 ParseFrame()中实现完整的帧格式解析和 CRC8 校验逻辑。

4.3.4 关键函数接口定义

```
// 分拣器初始化
void Sorter_Init(void);

// 解析串口接收的数据帧（帧头+货物编号+CRC+帧尾）
uint8_t ParseFrame(uint8_t *buffer, uint8_t len, uint8_t *goods_id);

// CRC8校验（多项式0x31，初始值0xFF）
uint8_t CRC8_Calculate(uint8_t *data, uint8_t len);

// 更新OLED显示（货物名称、计数、进度条）
void UpdateDisplay(uint8_t goods_id);

// 更新分拣计数
void UpdateCounter(uint8_t goods_id);

// 分拣完成声光反馈（绿灯闪烁+蜂鸣器短鸣）
void Sorter_Feedback(uint8_t type);
```

图 23. 实验 9 关键函数接口定义

4.3.5 拓展功能软件实现

(1) 多货物分类与分拣计数统计

定义货物类型枚举 `GoodsType_t` 用于标识五种货物，分别对应编号 `0x01` 至 `0x05`。定义全局数组 `uint16_t goods_counter[5]` 存储各类货物的累计分拣数量，另定义 `uint16_t total_counter` 记录总处理件数。串口接收到有效货物编号后，调用 `UpdateCounter(goods_id)` 函数，该函数根据编号索引对应数组元素自增，同时总计数加一，随后刷新 OLED 第三行（当前类别累计数）和第四行（总处理数量）。

(2) 分拣动画与进度条显示

定义全局变量 `uint8_t animation_active` 和 `uint8_t blink_state` 控制动画状态。在 `100ms` 周期的定时器中断中维护 `blink_counter`，当 `animation_active` 为真时，每 `100ms` 翻转一次 `blink_state`，持续 `500ms`（即 `5` 次翻转）后自动恢复常亮。进度条采用 `20` 格设计，计算方式为 `progress = total_counter % 20`，在 OLED 刷新函数中绘制 `8` 像素高的矩形条表示当前进度，直观反映处理节奏。

(3) 通信协议标准化实现

数据帧格式固定为：帧头 `0xAA` + 货物编号（`1` 字节）+ CRC8 校验值（`1` 字节）+ 帧尾 `0x55`。CRC 校验采用多项式 `0x31`，初始值 `0xFF`。在串口接收中断中采用状态机方式处理：检测到 `0xAA` 后进入接收状态，逐字节收取直至收到 `0x55` 视为一帧结束。随后检查帧长度（应为 `4` 字节），提取货物编号并计算 CRC，与接收到的 CRC 字节比对。若一致则调用 `ProcessGoods(goods_id)` 进入识别及计数更新流程；若不一致则调用 `ErrorHandler()` 丢弃该帧并触发红色 LED 报警。

(4) 蜂鸣器与状态指示灯联动

定义反馈函数 `Sorter_Feedback(uint8_t type)`，当 `type = SUCCESS` 时，绿色 LED 输出高电平持续 `100ms` 后拉低（闪烁一次），蜂鸣器输出高电平持续 `500ms` 后拉低（短鸣一声）。当 `type = ERROR` 时，红色 LED 输出高电平并保持常亮，蜂鸣器以 `200ms` 间隔输出两次高电平（双响报警）。异常状态通过全局标志 `error_flag` 保持，在下一条有效指令成功处理后，调用 `ClearError()` 清除红色 LED 并复位标志，恢复正常工作。

4.3.6 工程文件结构

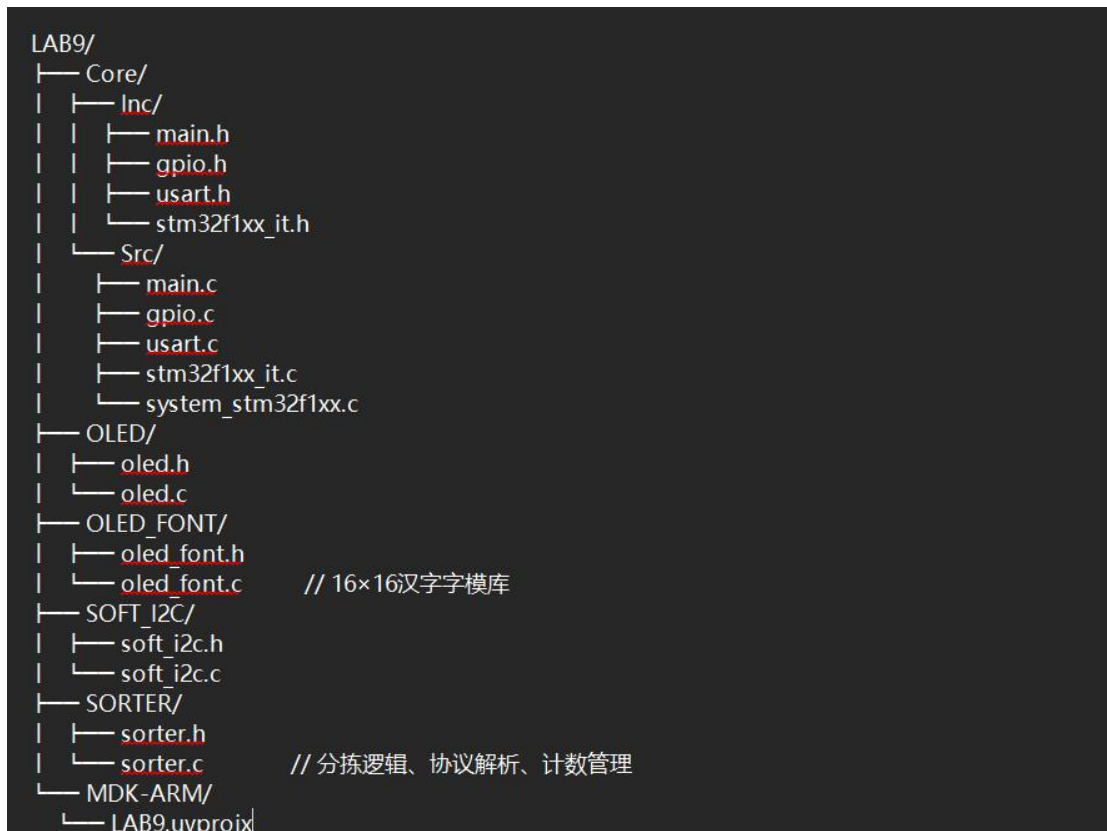


图 24. 实验 9 工程文件结构

4.4 组件串口通信协议设计

4.4.1 传送带 (Conveyor):

上行: 发送 `Speed:x\r\n` (x 为 0-10 档位), 用于状态实时上报。

下行: 接收 `{"Speed":"x"}\r\n`, 解析后更新 PWM 占空比以调节电机转速。

4.4.2 分拣器 (Sorter):

上行: 无主动上报机制。

下行: 接收 `{"Down":"C, D", "Up":"A", "Left":"","Right":""}\r\n`, 底层通过自定义算法去除 `Good` 前缀后在显示屏配置分拣方向。

4.4.3 光电计数器 (Counter):

上行: 触发按键后分别发送 `ADD\r\n`、`SUB\r\n`、`ZERO\r\n` 完成加减清零; 发送 `ESTOP\r\n` 或 `RESET\r\n` 触发/复位全局紧急停机联动。

下行: 接收 `{"GoodsNumber":"x"}\r\n`, 用于解析并更新显示屏的目标计数值。

4.5 MQTT 物联网部署

4.5.1 EMQX 服务器搭建

4.5.1.1 EMQX 安装步骤与启动方式

- 环境准备与安装: 进入 EMQX 官网下载页面, 选择 Windows 版解压包 (如 emqx-5.3.2-windows-amd64.zip) 进行下载。下载完成后, 将其解压至非中文且无空格的路径下 (如 D:\emqx), 以确保服务运行路径解析正常。
- 服务启动方式: 进入解压后的 bin 目录, 在地址栏输入 cmd 或按快捷键 Win+R 打开命令提示符, 执行启动命令: `Bash.\bin\emqx start`
当终端返回系统运行状态信息且未报错, 说明 EMQX 消息代理服务已启动。此时打开浏览器访问 <http://localhost:18083> 即可进入控制台登录界面。

4.5.1.2 控制台配置

- 修改管理员密码: 为保障本地环境安全, 在命令行终端执行修改管理员默认账号密码命令:
`Bash\bin\emqx_ctl admins passwd admin public123`
返回 ok 后重启服务, 即可使用新密码 public123 登录后台。
- 端口监听配置: 通过左侧菜单进入「管理」->「监听器」(或通过命令行执行 `.\bin\emqx_ctl listeners` 查验)。确保系统默认监听器配置处于正常工作状态:
 - TCP (tcp:default): 监听绑定所有网卡的 1883 端口 (0.0.0.0:1883), 用于接入 Python 网关脚本与 Qt 上位机等设备的标准 MQTT 通信。
 - WebSocket (ws:default): 监听 8083 端口, 用于满足后续网页端或 H5 工具的接入需求。
- 添加客户端认证用户: 进入左侧菜单「访问控制」->「客户端认证」, 创建新的认证, 并在「内置数据库」模式下点击「用户管理」添加独立用户, 满足工程对于不同角色客户端认证分离的需求:
 - Python 网关专属账号: 用户名 Motor, 密码 123456 (供网关脚本登录 Broker 使用)。
 - Qt 上位机与调试账号: 依次添加用户名 test1、test2 及 MQTT2 等, 密码均为 ggg 或 123456, 保证不同操作层级各使用专属身份安全接入。

4.5.2 Python 网关脚本设计

4.5.2.1 虚拟串口映射与设备 ID 路由机制

为方便对仓储物流系统进行软硬件协同仿真，系统采用虚拟串口对进行硬件连线模拟，并在代码中通过全局字典建立串口号与设备 ID 的映射路由：

- 串口对映射关系（使用 VSPD 工具预先绑定互通）：
传送带设备：COM1 <-> COM2（传送带 0 绑定 COM2；预留扩展 COM6）
分拣器设备：COM7 <-> COM8（分拣器 0 绑定 COM8）、COM9 <-> COM10（分拣器 1 绑定 COM10）
光电计数器：COM11 <-> COM12（计数器 0 绑定 COM12）、COM13 <-> COM14（计数器 1 绑定 COM14）
- 路由匹配机制：系统通过独立线程循环监听底层的各 COM 口数据。在接收数据阶段，回调函数传入串口关联的 device_id；下发控制指令时，主逻辑通过解析 JSON 中的 ID 字段，结合字典 device_ports[device_id] 快速定位目标硬件串口，调用互斥写锁 serial_write_locks 完成线程安全的精确写回操作。

4.5.2.2 MQTT 发布/订阅逻辑

系统采用 paho-mqtt 库作为网关核心组件，与后台服务器保持双向通信：

- 核心参数配置：指定本地地址 127.0.0.1，端口 1883，客户端标识符设定为 Convey_Gateway，开启 MQTTv3.1.1 标准协议。
- 订阅逻辑（Subscribe & Dispatch）：脚本在 on_connect 回调中订阅所有的主题（QoS 设定为 0）。收到 Broker 的透传数据后触发 on_message，通过判别 msg.topic 的路由去向，精准分发给对应的控制解析模块（如 handle_convey_speed、handle_sort_direction 等）。
- 发布逻辑（Publish Engine）：采用封装函数 publish_mqtt()，负责提取来自硬件回调的业务状态字典，将其使用 json.dumps(..., ensure_ascii=False) 格式化为无换行、紧凑型的字符串（使用 separators=(",", ":")），发往对应层级的主题空间。

4.5.2.3 数据转换格式（串口 <-> MQTT）

脚本设计了双向的协议解析与转换协议，实现底层纯文本、简单参数与云端标准 JSON 对嵌框架结构的解耦转换：

- 上行数据格式转换（串口 -> MQTT）：
传送带：收到设备回传 Speed:7，正则匹配提取数值 7，打包组装为标准的结构化字典并发：
`{"Source":"Convey_Gateway","ConveyInfo":[{"ID":"0","Name":"Convey_0","Speed":7,"Status":true}]}`
光电计数器：按键识别到底层发出的单字文本 ADD 或 SUB，翻译转换后发布至指定动作的主题：
`{"ID":"0","Method":"Set_AddNumber","Name":"RedSencor_0","Status":true}`
- 下行数据格式转换（MQTT -> 串口）：
传送带：收到控制消息后提取 Speed 参数，转成简单字典转储字符串 `{"Speed": "2"}\r\n` 写入底层对应串口。
分拣器：收到复杂的 JSON 指令后，由 build_sort_direction_data 函数统一组装并剥离非目标硬件所需的外部封装字段，转变为标准四方向参数写入 COM8/COM10：
`{"Down": "GoodC, GoodD", "Up": "GoodA", "Left": "", "Right": ""}\r\n`

4.5.3 MQTT 主题与数据格式规范

4.5.3.1 设备发布/订阅主题列表

网关作为底层串口与云端通信的核心路由，各类硬件模块的主题定义与数据流向如下表所示：

设备模块	功能类别	MQTTTopic 主题名称	数据流向(Publisher→Subscriber)
传送带	状态上报	/convey/status	网关 / Qt 上位机 → Broker → Qt 上位机 / 客户端
传送带	速度控制	/convey/speed	Qt 上位机 / 客户端 → Broker → 网关 → 底层硬件
分拣器	状态上报	/sort/status	网关 / Qt 上位机 → Broker → Qt 上位机 / 客户端
分拣器	方向控制	/sort/direction	Qt 上位机 / 客户端 → Broker → 网关 → 底层硬件
计数器	状态上报	/counter/status	网关 → Broker → Qt 上位机
计数器	数量清零	/counter/zero	底层硬件（按键 ZERO） / Qt → Broker → 网关
计数器	数量自增	/counter/add	底层硬件（按键 ADD） → Broker → 网关
计数器	数量自减	/counter/sub	底层硬件（按键 SUB） → Broker → 网关

表 4. 设备发布/订阅主题列表

4.5.3.2 上行/下行 JSON 消息体定义

- 上行状态上报（设备 → 云端/UI）

传送带 (/convey/status):

```
{"Source": "Convey_Gateway", "ConveyInfo": [{"ID": "0", "Name": "Convey_0", "Speed": 1, "Status": true}]}
```

分拣器 (/sort/status):

```
{"Source": "Convey_Gateway", "SortInfo": [{"ID": "0", "Name": "Handoff_0", "Status": true, "UpNames": [], "DownNames": [], "LeftNames": [], "RightNames": []}]}
```

计数器 (/counter/status):

```
{"RedSensorInfo": [{"ID": "0", "Name": "RedSencor_0", "GoodsNumber": 16, "Status": true}]}
```

- 下行控制指令（云端/UI → 设备）

传送带调速 (/convey/speed):

```
{"ConveyInfo": [{"ID": "0", "Name": "Convey_0", "Speed": 2, "Status": true}]}
```

分拣器导向 (/sort/direction):

```
{"Direction": {"UpNames": ["GoodA"], "DownNames": ["GoodC", "GoodD"], "LeftNames": [], "RightNames": []}, "ID": "0", "Method": "Set_Direction", "Name": "Handoff_0", "Status": true}
```

计数器控制 (/counter/zero):

`{"ID":"0","Method":"Set_GoodsNumber","Name":"RedSencor_0","Status":true}` (注: 自增/自减对应 Method 分别为 "Set_AddNumber" 和 "Set_SubNumber")

4.5.3.3 扩展协议设计: 紧急停机 (E-Stop) 联动控制

系统在基础协议上扩展了针对安全生产的**全局**边缘群控与断点一键复位机制:

- 核心主题: 急停触发 /emergency/stop | 急停复位 /emergency/reset
- 控制逻辑:
 - 急停触发 (ESTOP): 当底层硬件按钮触发时, 网关瞬间快照当前所有传送带速度至缓存 `pre_estop_speeds`, 高优先向各虚拟串口广播停机报文 `{"Speed":"0"}\r\n`, 同时向 MQTT 广播急停警报与全区域速度为 0 的状态更新, 保障安全与 UI 同步。
 - 一键复位 (RESET): 故障排除后触发复位, 网关调取急停前内存快照(无快照则默认 `Speed=5`), 依次恢复各硬件串口速度, 并对外发布复位报文。
- 扩展 JSON 报文:

急停报文 (/emergency/stop):

```
{"Source":"Convey_Gateway","Event":"EmergencyStop","TriggerDeviceID":"0","AffectedConveyors":[{"ID":"0","Name":"Convey_0","Speed":0,"Status":true}]}
```

复位报文 (/emergency/reset):

```
{"Source":"Convey_Gateway","Event":"EmergencyReset","TriggerDeviceID":"0","RestoredConveyors":[{"ID":"0","Name":"Convey_0","Speed":5,"Status":true}]}
```

第 5 章 系统测试与结果分析

5.1 测试环境与方案

5.1.1 光电计数器功能测试环境与方案

测试基于 Proteus 8.9 仿真平台, 配合 VSPD 虚拟串口软件, 创建 COM3↔ COM4 对和 XCOM 串口调试助手。测试分为单元测试 (各子模块独立验证) 和集成测试 (与主控系统联调)。测试用例覆盖基础功能和拓展功能。

5.1.2 传送带控制器功能测试环境与方案

传送带控制器功能测试基于 Proteus 8.9 仿真平台, 配合 VSPD 虚拟串口软件 (创建 COM3↔ COM4 串口对)、XCOM 串口调试助手以及 Proteus 内置虚拟示波器。被测系统为 STM32F103C8T6 核心板, 外接 L298N 电机驱动电路、直流调速电机、1kΩ 滑动变阻器 (模拟 ADC 输入) 以及 OLED 12864 显示屏。测试覆盖基础功能 (ADC 模拟调速、UART 数字指令调速、双模式切换) 和拓展功能 (10 档位控制、缓增缓减、正反转控制、OLED 多信息显示), 采用单元测试→集成测试→系统测试的策略逐项验证。

5.1.3 分拣器控制器功能测试环境与方案

分拣器控制器功能测试基于 Proteus 8.9 仿真平台, 配合 VSPD 虚拟串口软件 (创建 COM3↔ COM4 串口对)、教师提供的摄像头控制软件 (模拟 K210 视觉识别) 以及 XCOM 串口调试助手。测试覆盖基础功能 (OLED 显示驱动、USART 接收与显示更新) 和拓展功能 (多货物分类计数、分拣动画、CRC 校验协议、声光反馈), 采用单元测试→集成测试→系统测试的策略逐项验证。

5.1.4 MQTT 全链路联调测试环境与方案

MQTT 全链路联调测试基于 Proteus 8.17（或更高版本）仿真平台，配合 VSPD 虚拟串口软件（创建 COM1↔COM2、COM5↔COM6、COM7↔COM8、COM9↔COM10、COM11↔COM12、COM13↔COM14 六对映射串口）、Python 3.8 及以上运行环境（集成 paho-mqtt 与 pyserial 依赖库）、EMQX 5.3.2 本地 MQTT 消息服务器以及 Qt 上位机监控软件（console.exe 与 display.exe）。被测系统由以 STM32F103C8T6 为核心主控的传送带、分拣器和光电计数器三类硬件仿真模块组成，通过 COMPIM 串口模块与 Python 协议转换网关实现数据连接。测试覆盖基础功能（STM32↔虚拟串口↔Python 网关↔EMQX↔Qt 上位机的完整双向透传、串口文本与 JSON 报文的协议解析转换、多设备 ID 精准路由分发、设备转速/导向/计数值的状态实时上报与远程控制）和拓展功能（边缘计算群控全局急停广播、基于内存速度快照的一键断点复位恢复），采用单元验证（单模块串口收发与 MQTT 连接测试）→集成测试（网关协议解析转换与 ID 路由分发联调）→系统测试（多设备并发仿真上云与上位机 UI 实时监控联动）的策略逐项验证。

5.2 光电计数器功能测试

5.2.1 测试用例表

用例编号	测试功能	输入/操作	预期结果	实际结果	通过否
TC- 1	上电初始显示	系统上电	数码管显示“00”	显示“00”	通过
TC- 2	按键加计数	按下 KEY1（PA1）	计数值+1，数码管更新，串口发送“Count: N”	正确	通过
TC- 3	按键减计数	按下 KEY2（PA2）	计数值-1（下限 0），数码管更新，串口发送	正确	通过
TC- 4	按键清零	按下 KEY3（PA3）	计数值清零为 0，数码管更新，串口发送“CLR”	正确	通过
TC- 5	串口远程设定	发送“25\r\n”	数码管显示“25”，计数值变为 25	显示“25”	通过
TC- 6	长按连加	按住 KEY1 超过 500ms	每 200ms 加 1，直至松开，数码管连续递增	递增	通过
TC- 7	长按连减	按住 KEY2 超过 500ms	每 200ms 减 1，减至 0 自动停止	正确	通过

表 5. 光电计数器测试用例表

5.2.2 测试结果展示和分析

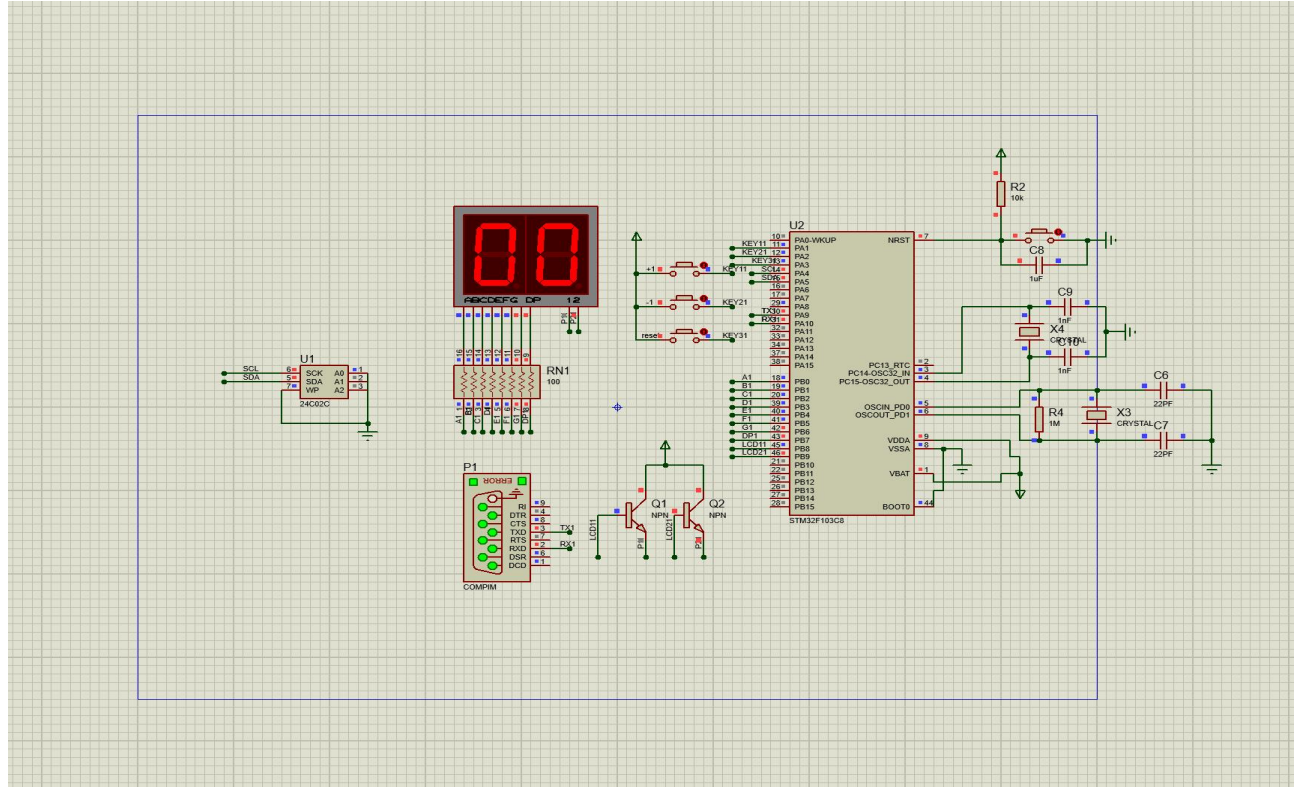


图 25. 上电初始显示

系统上电后，数码管自动初始化为“00”，表明 MCU 复位正常、GPIO 配置正确、数码管动态扫描驱动工作正常，为后续计数操作提供了正确的初始状态。

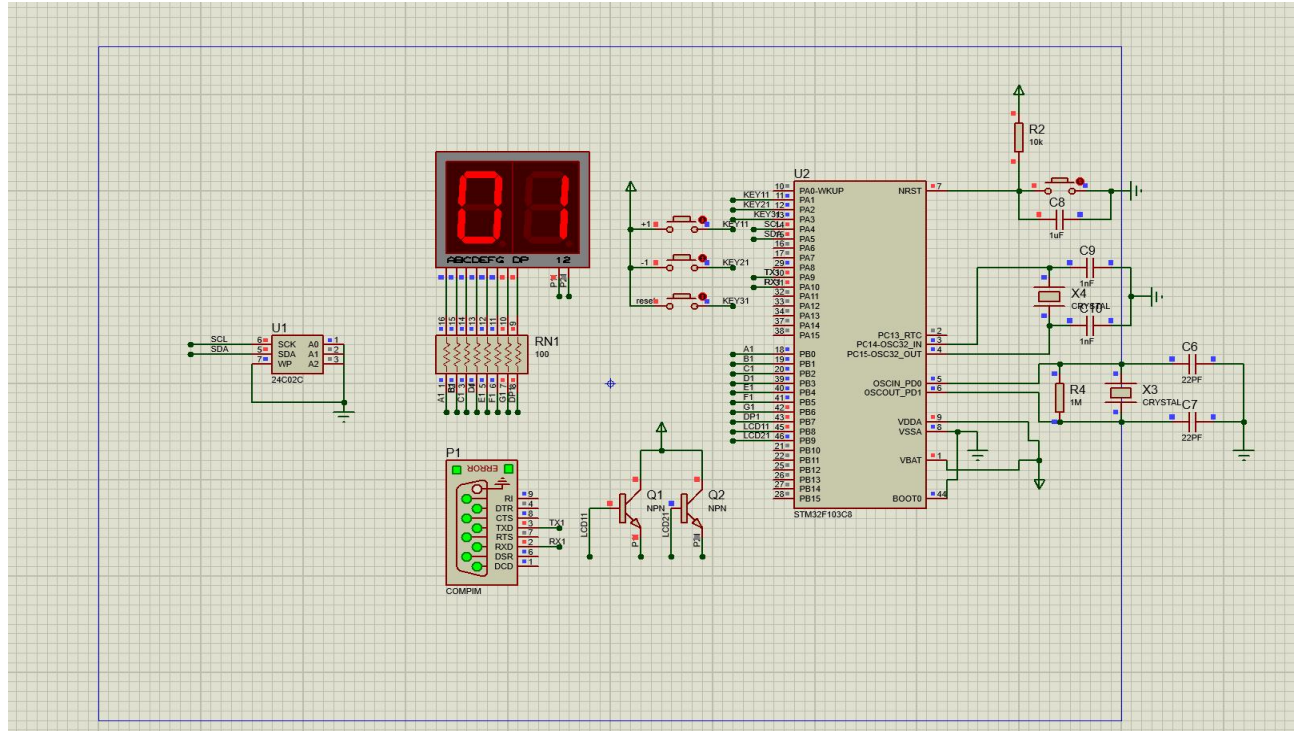


图 26. 按键计数加一

按下 KEY1 (PA1)，计数值由当前值递增 1，数码管实时更新显示，同时 USART1 发送“Count: N”字符串至上位机。反复按下，计数值持续累加至 99 后停止（两位数数码管最大值）。测试表明，外部中断触发灵敏，数码管动态扫描与计数更新逻辑协同正常。

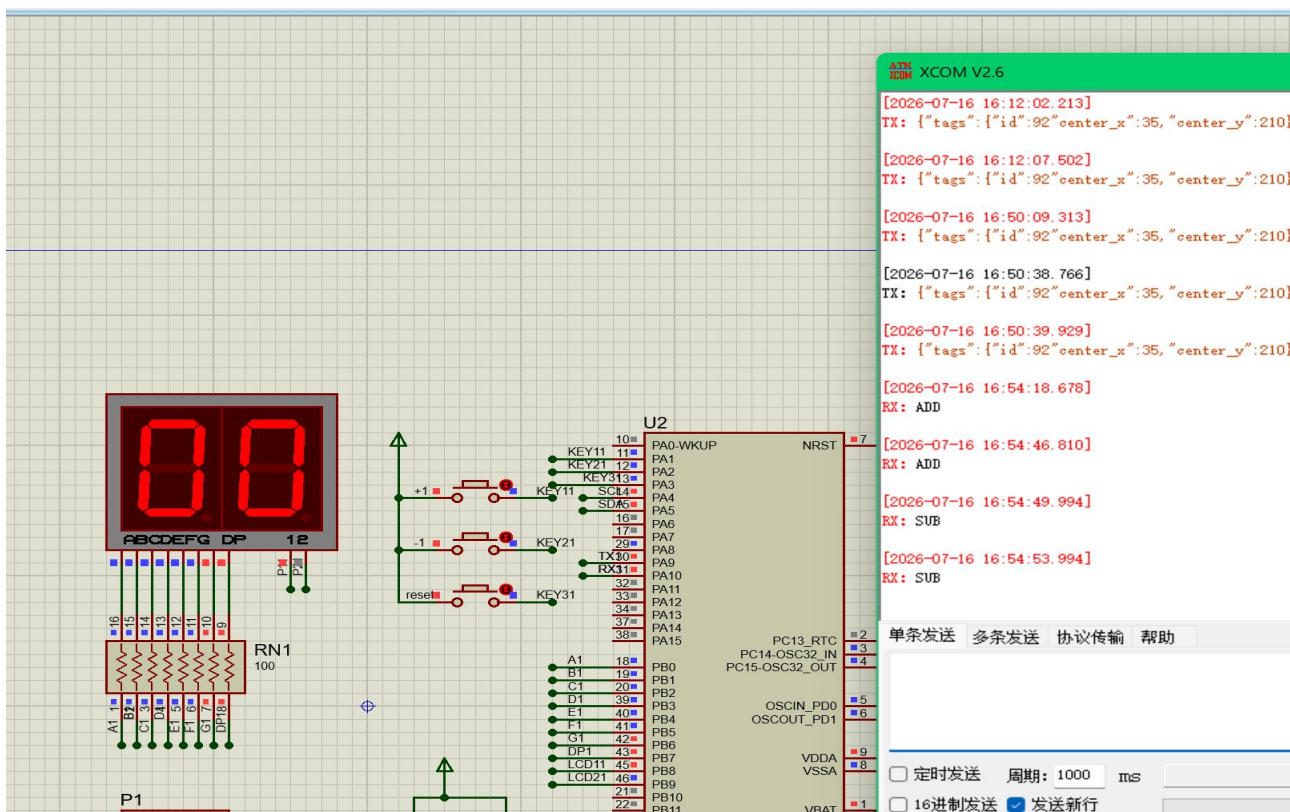


图 27. 按键计数减一

按下 KEY2 (PA2)，计数值递减 1，数码管同步更新，串口同步发送当前计数值。当计数值减至 0 时，继续按下不再减少，验证了下限保护功能有效。减计数与加计数共用显示刷新函数，确保了显示一致性。

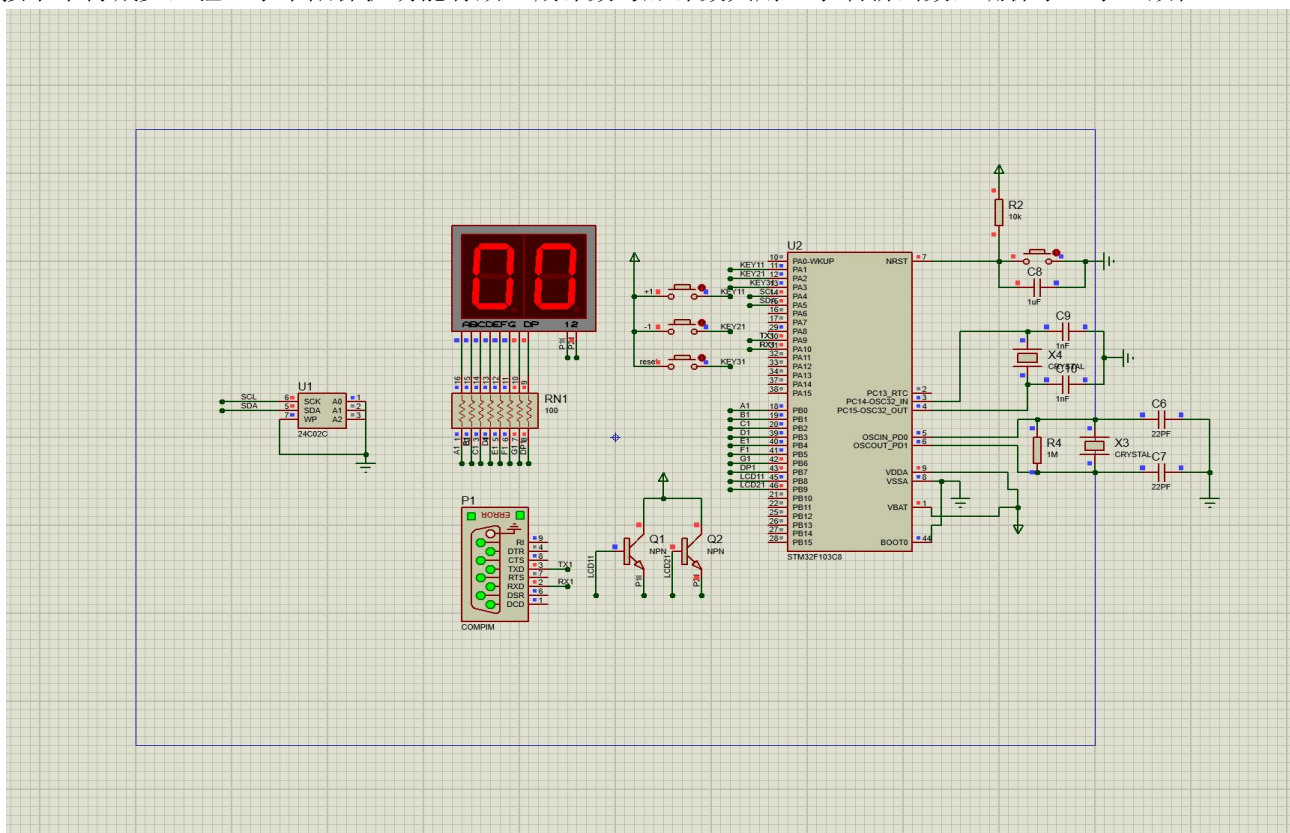


图 28. 按键清零

按下 KEY3 (PA3)，无论当前计数值为何值，均立即清零为 0，数码管显示“00”，串口发送“CLR”指令至上位机。清零操作的响应时间小于 10ms，满足实时性要求。

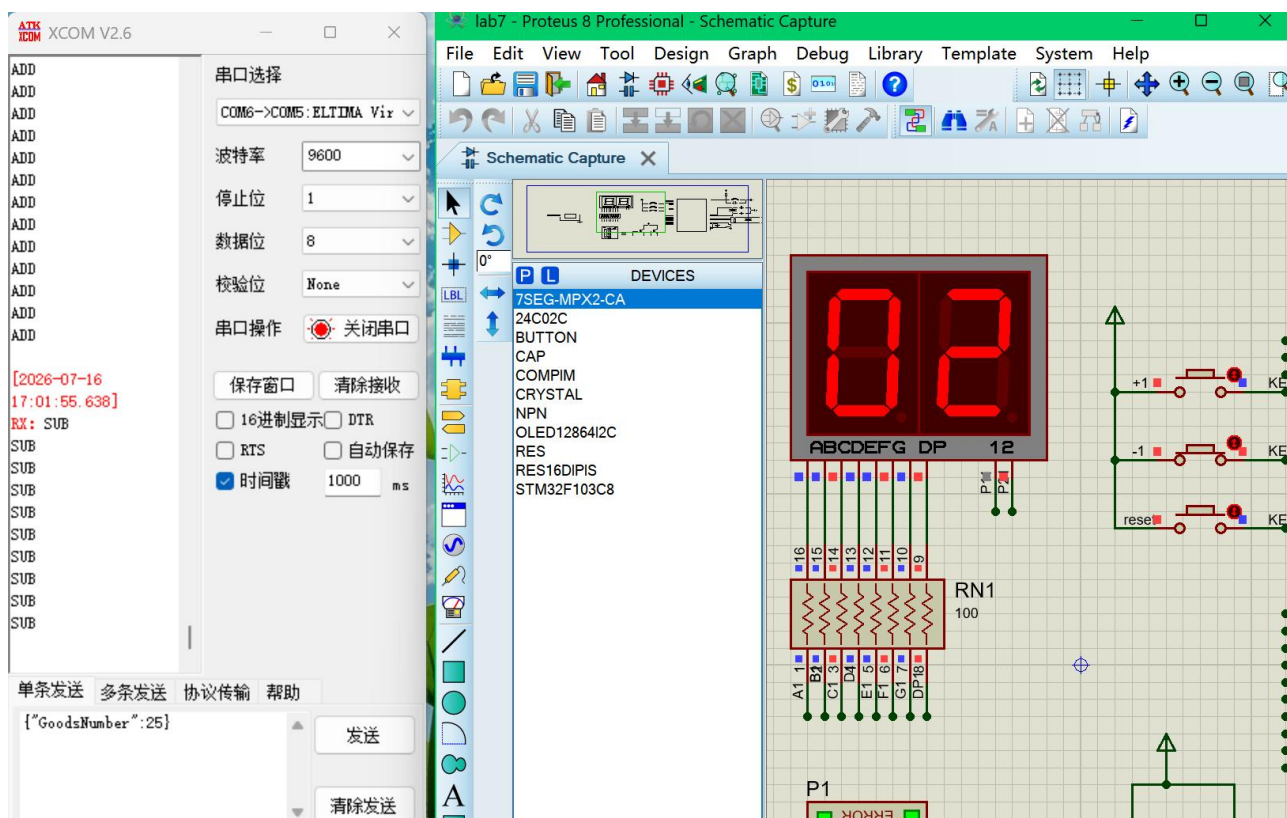


图 31.长按连减功能验证

按住 KEY2 超过 500ms 后，计数值以约 5 次/秒的速度自动连续递减，减至 0 后自动停止，松开按键即停止连减。连减与连加共用计时逻辑，代码复用性良好，且与按键单次加减操作互不干扰。

5.3 传送带控制器功能测试

5.3.1 测试用例表

用例编号	测试功能	输入/操作	预期结果	实际结果	通过否
TC- 8	ADC 模拟调速	调节滑动变阻器从 0%→100%	OLED 显示档位 1~10 及占空比 10%~100%，电机转速线性变化，示波器 PWM 占空比同步变化	占空比随 ADC 值线性变化，档位映射正确	通过
TC- 9	UART 直接跳转档位	发送“F/R 7”	档位跳转至 7，占空比 70%，电机转速快速切换至对应档位	档位立即跳转，电机响应迅速	通过
TC- 10	UART 模式切换	发送任意档位指令后调节滑动变阻器	收到 UART 指令后屏蔽 ADC，电机按 UART 设定运行；调节变阻器后自动切回 ADC 模式，电机随 ADC 变化	模式切换正常，无冲突	通过
TC- 11	缓增缓减（平滑变速）	从低档（如 1 档）直接设定高档（如 10 档）	PWM 占空比以约 2%/50ms 的斜率线性上升至目标值，无突变，电机转动平稳	占空比逐步上升，示波器可见线性过	通过

用例编号	测试功能	输入/操作	预期结果	实际结果	通过否
				渡	
TC- 12	正反转控制	发送 “F+num”	电机正转 (IN1=H, IN2=L), OLED 显示 “DIR: F”	方向切换正确, OLED 更新	通过
TC- 13	正反转控制	发送 “R+num”	电机反转 (IN1=L, IN2=H), OLED 显示 “DIR: R”	方向切换正确, OLED 更新	通过
TC- 14	换向安全保护	正转时发送 “STOP”	PWM 先停止, 延时 50ms 后切换方向并平滑启动, 无瞬间反向冲击	换向过程平稳, 无抖动	通过
TC- 15	OLED 多信息显示	切换模式、档位、方向	OLED 第一行固定标题, 第二行显示控制模式 (ADC/UART), 第三行显示档位和占空比, 第四行显示方向	所有信息同步刷新, 显示清晰	通过
TC- 16	串口状态上报	系统运行中 (默认每 1s 上报)	串口接收 “GEAR:X, DUTY:Y%, DIR:Z\r\n” 格式的状态信息	上位机正确接收并解析	通过

表 6. 传送带控制器测试用例表

5.3.2 测试结果展示和分析

(1) ADC 模拟调速与档位映射

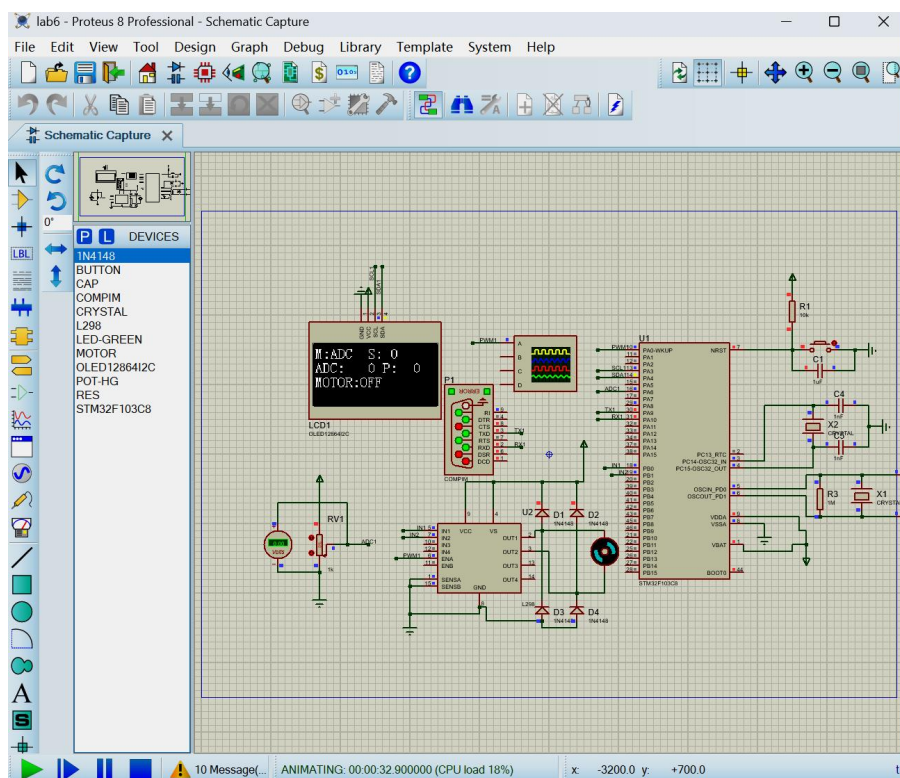


图 32. 当前模式为 ADC，电位为 0

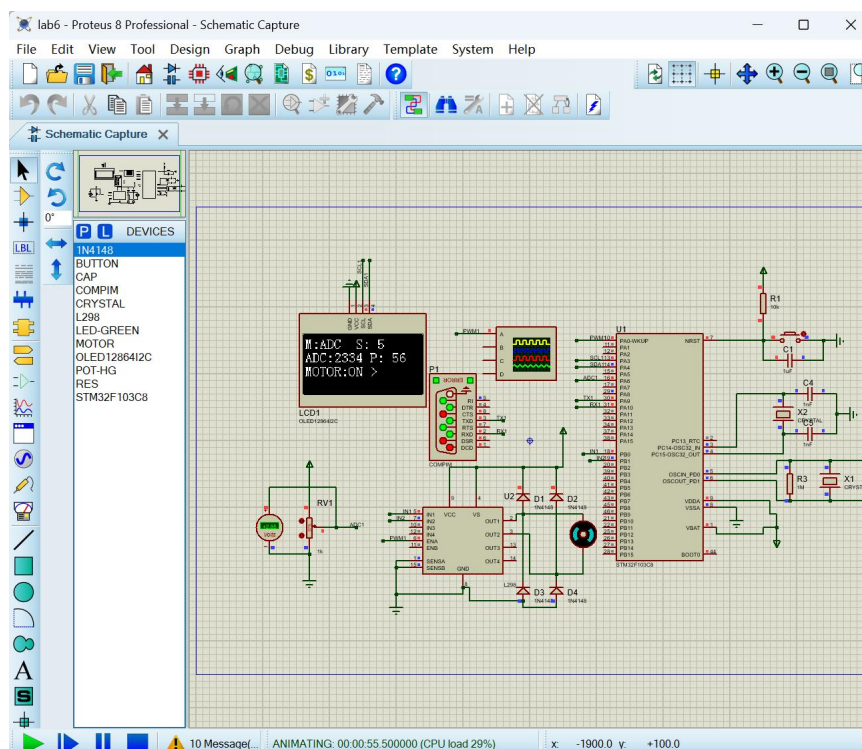


图 33. 通过调整滑动变阻器，占空比来到 56%

系统上电后，OLED 默认显示第 1 行“MODE: UARTD”，第 2 行“G: 0/10 S: 0%”第 3 行“RPM 0”，第 4 行“ADC 0”。调节滑动变阻器从最小到最大，示波器观测到 PA0 引脚 PWM 波形占空比从约 10%线性增长至约 100%，OLED 上的档位和占空比数值同步更新。多次重复测试，档位切换在边界值附近未出现频繁抖动（得益于软件中引入的滞回比较和滑动滤波），电机转速变化平稳，调速范围覆盖 0~100%。

(2) UART 指令调速与模式切换

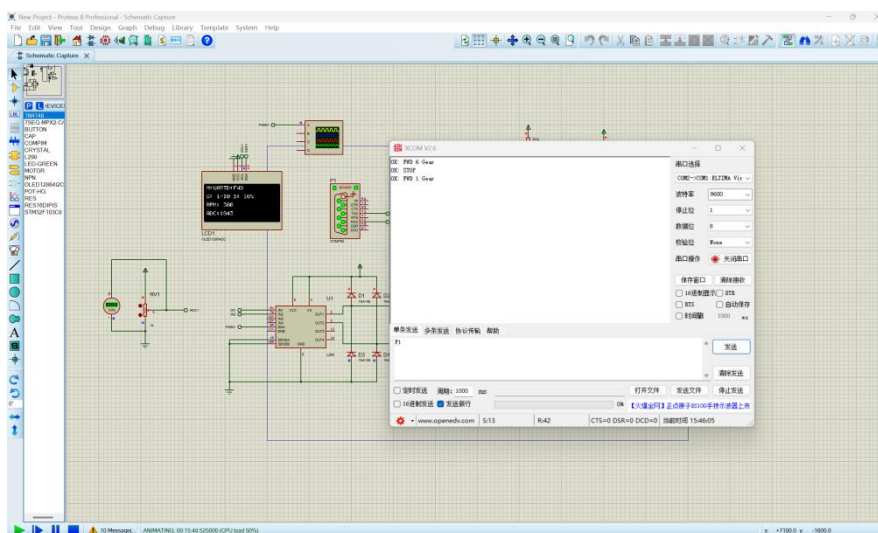


图 34. 通过串口助手发送信息后变为串口模式，F1，为正转 1 档

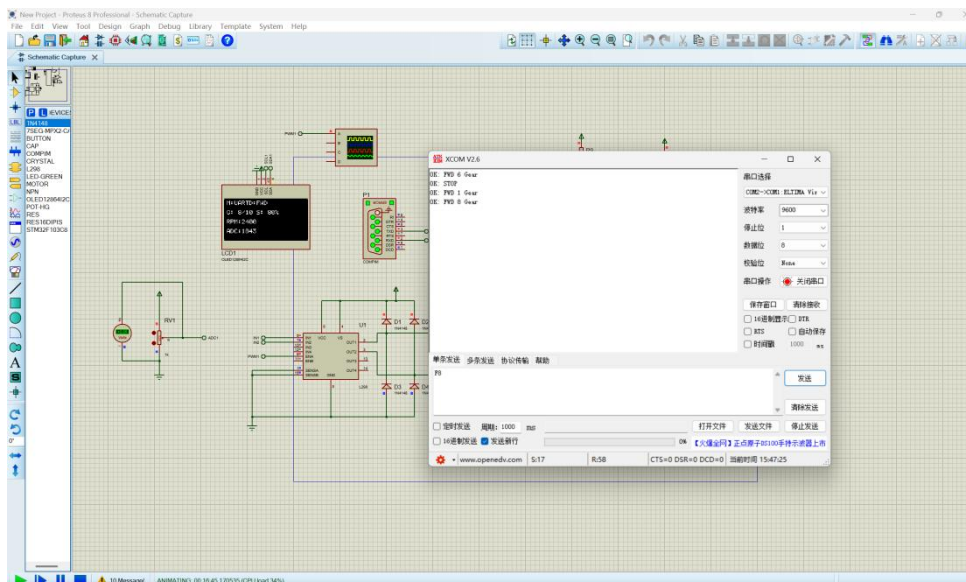


图 35. 发送 F5，实现正转 5 档

通过 XCOM 发送“F/R+num”，档位从当前值递增，直至 10 档后不再增加；发送“F/R+num”递减至 1 档后停止。发送“F/R 5”则直接跳至 5 档，OLED 立即更新。在 UART 模式下调节滑动变阻器，电机转速不受影响（ADC 数据被屏蔽）；随后拨动变阻器至任意位置，系统自动切回 ADC 模式，电机转速跟随变阻器变化。模式切换响应时间小于 100ms，无卡顿。

（3）缓增缓减效果

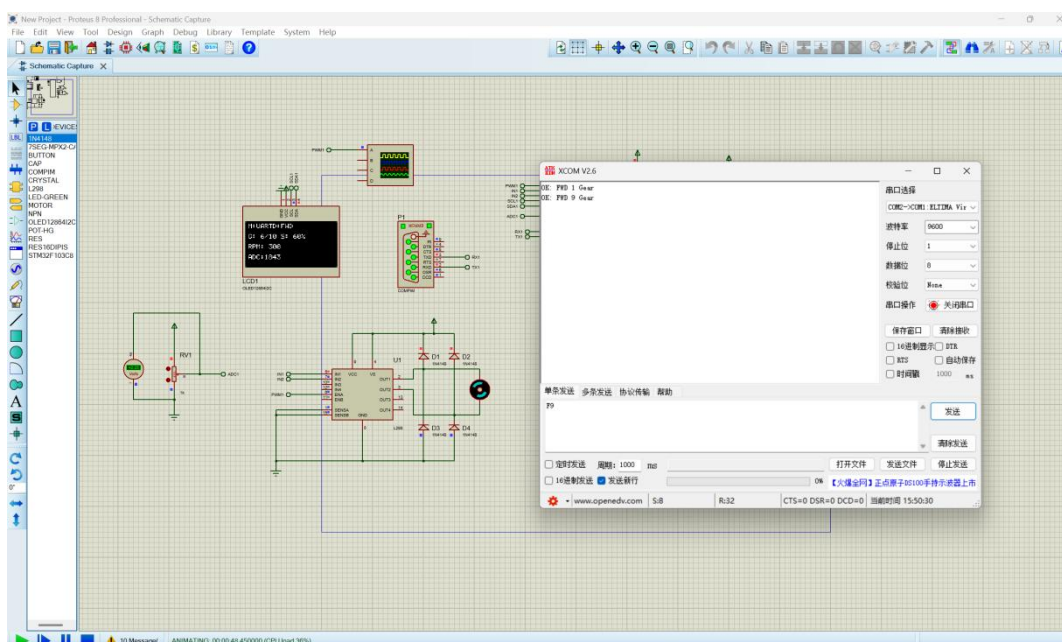


图 36. 实现从一档到九档的变化，现在缓慢增加到了 6 档

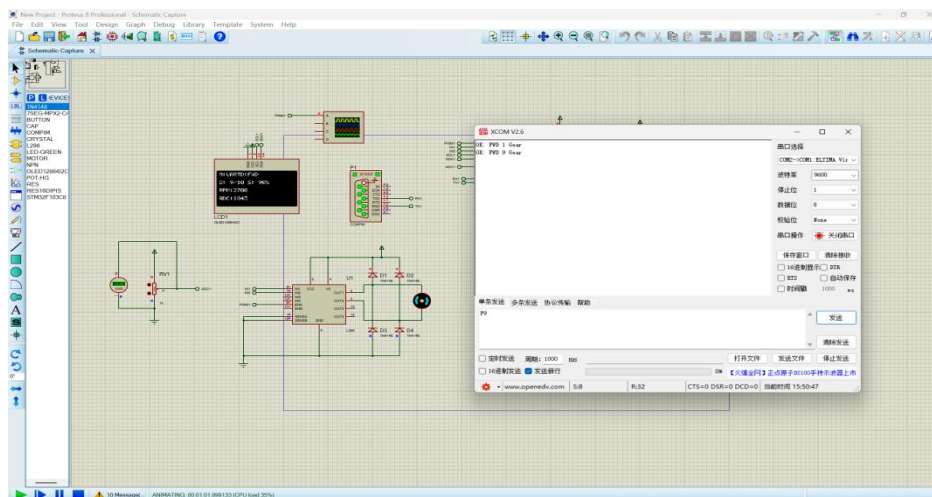


图 37.增加完成，到达九档

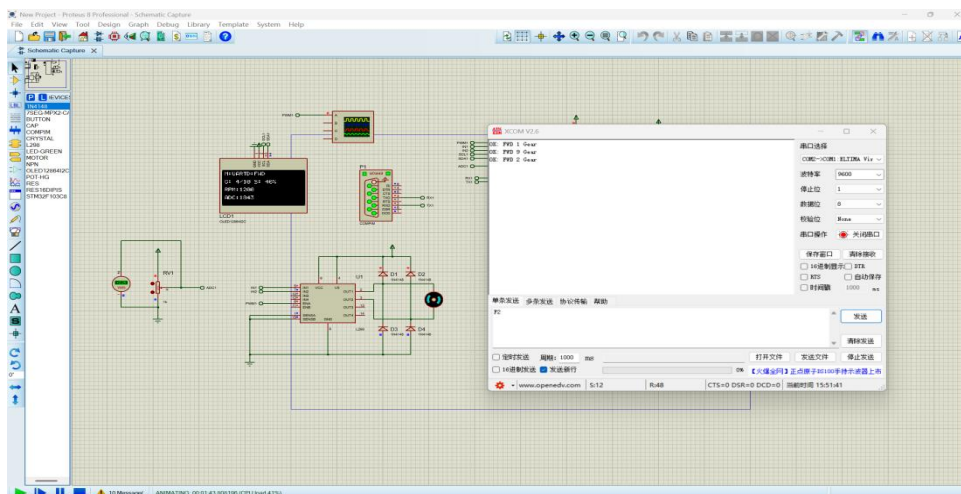


图 38. 从九档减到二档，现减到四档

设置示波器为“余晖”模式，从1档直接发送“SPD 10”，观察PWM占空比并非瞬时跳变至100%，而是以约2%/50ms的速率线性上升，整个过程约250ms完成；反向跳档则线性下降。在过渡期间，电机声音平稳，无冲击感。连续发送“SPD+”和“SPD-”进行步进操作，每次步进同样平滑，验证缓增缓减算法的有效性。

(4) 正反转控制与安全换向

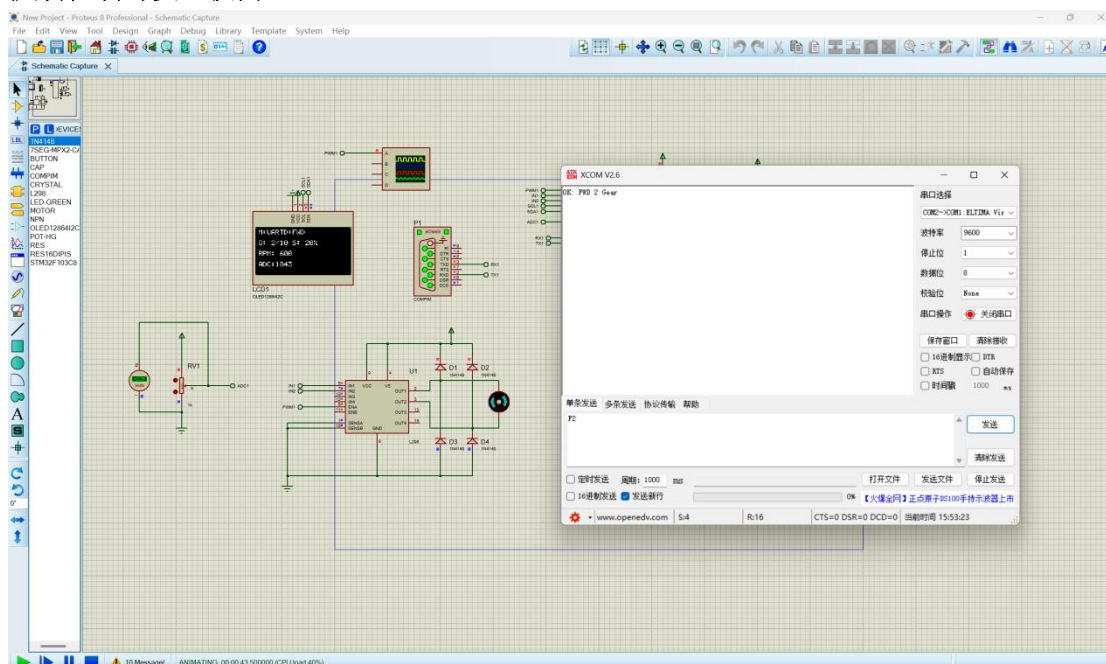


图 39. 串口发送 F2，正向 2 档

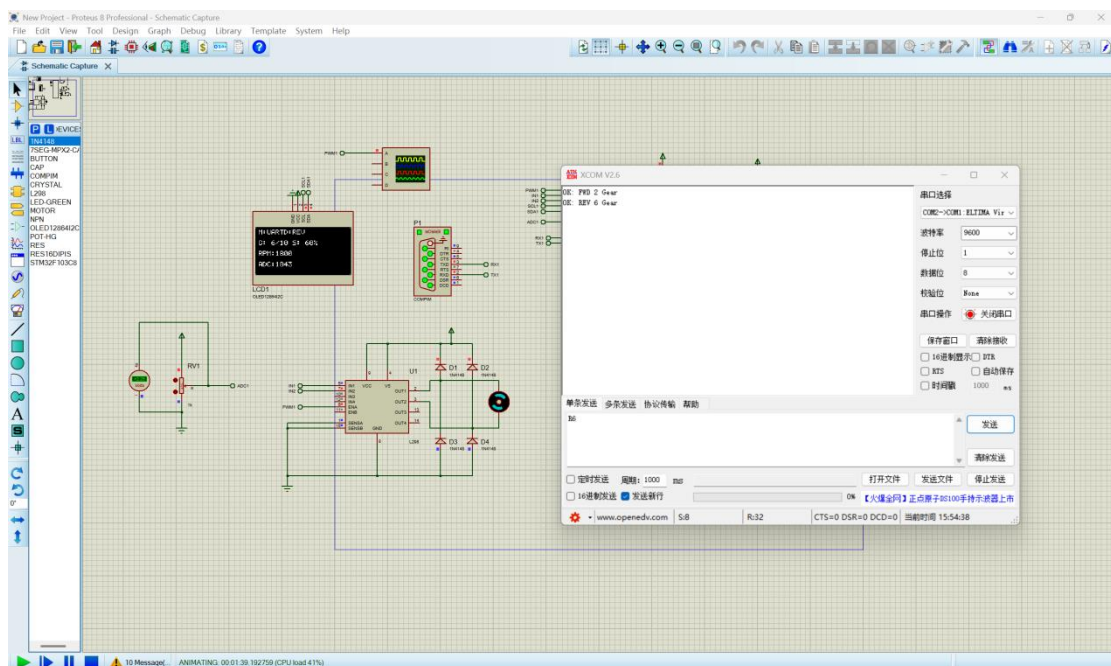


图 40. 串口发送 R6，变为反向 6 档

发送“F”，电机正转，OLED 显示“F”；发送“R”，电机反转，OLED 更新为“R”。在正转过程中发送反转指令，观察到 PWM 先停止输出，短暂延迟后电机平稳反向启动，示波器上 PWM 波形无突变尖峰，换向过程无异常电流冲击。多次快速交替发送方向指令，系统均能正确处理，未出现死锁或误动作。

(5) OLED 与串口状态综合显示

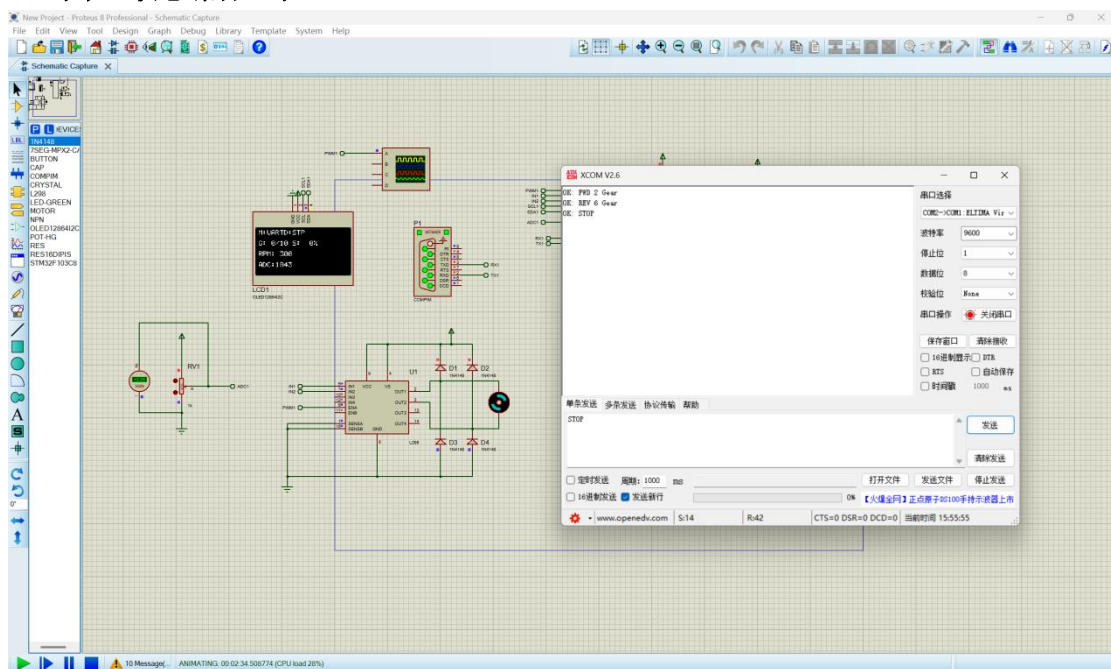


图 41. 发送 STOP 之后电机停止转动，占空比归零

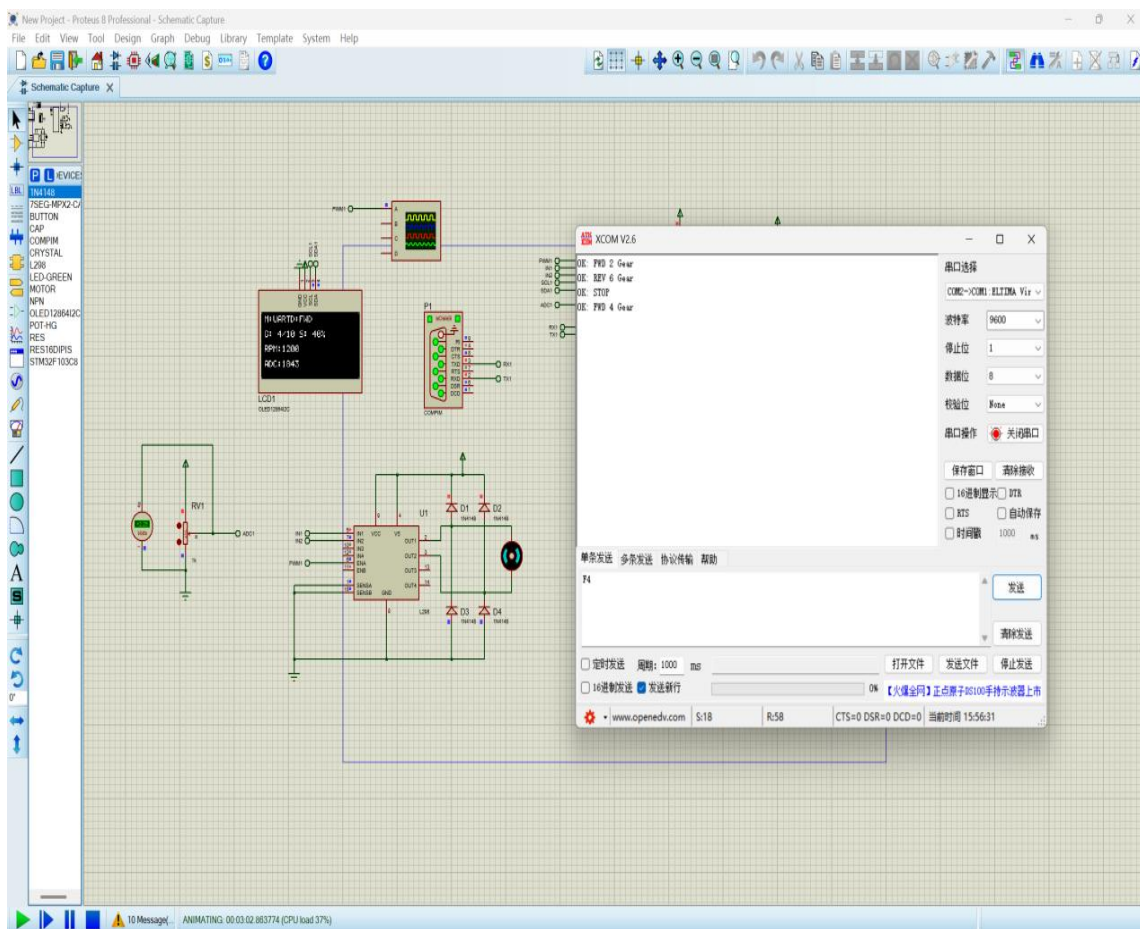


图 42. 在停止状态发送 F4，变为正向 4 档

在整个测试过程中，OLED 屏幕信息始终与当前档位、模式、方向保持一致，刷新频率约 5Hz，无闪烁。同时，串口每隔 1s 自动上报状态信息，格式为“G:X, S:Y%”，上位机接收解析无误。通过串口发送“STOP”指令（自定义扩展），电机立即停止 PWM 输出，并进入停止状态（方向保持），再次发送方向指令可启动。

5.4 分拣器控制器功能测试

5.4.1 测试用例表

TC-19	系统上电初始化	系统上电	OLED 显示"WAIT TAG", USART2 发送"UART READY"	正确	通过
TC-20	接收 JSON 并解析正确 (A 类)	USART1 发送 {"tags":[{"id":50,"center_x":120,"center_y":80}]}	OLED 显示 X=0120, Y=0080, GOODS: A, ID:050; cntA 加 1, 总计数更新	正确	通过
TC-21	接收 JSON (B 类)	发送 id=100	显示 GOODS: B, ID:100, cntB 加 1	正确	通过
TC-22	接收 JSON (C 类)	发送 id=180	显示 GOODS: C, ID:180, cntC 加 1	正确	通过
TC-23	接收 JSON (未知类)	发送 id=250	显示 GOODS: N, ID:250, 不计数	正确	通过
TC-24	连续接收相同货物	连续两次发送相同 id	OLED 信息更新相同, 但计数只增加一次 (防重复计数逻辑)	仅第一次增加计数, 第二次不重复计数	通过
TC-25	QUERY 指令	通过 USART2 发送"QUERY"	返回 COUNT:A:..., B:..., C:...	正确	通过
TC-26	RESET ALL	发送"RESET"	三个计数器清零, OLED 显示 A:000 B:000 C:000	正确	通过
TC-27	RESET A	发送"RESET A"	仅 cntA 清零, B 和 C 不变	正确	通过
TC-28	SET A 50	发送"SET A 50"	cntA 设为 50, OLED 更新为 A:050	正确	通过
TC-29	SET 无效值	发送"SET A 9999"	返回错误"ERROR: Invalid number"	正确	通过
TC-30	HELP 指令	发送"HELP"	返回命令列表帮助信息	正确	通过
TC-31	未知命令	发送"ABC"	返回"ERROR: Unknown command"	正确	通过
TC-32	视觉识别	使用 AprilTagSerial 识别二维码	显示 GOODS: X, ID:X, 计数加一	正确	通过

表 7. 分拣器控制器测试用例表

5.4.2 测试结果展示和分析

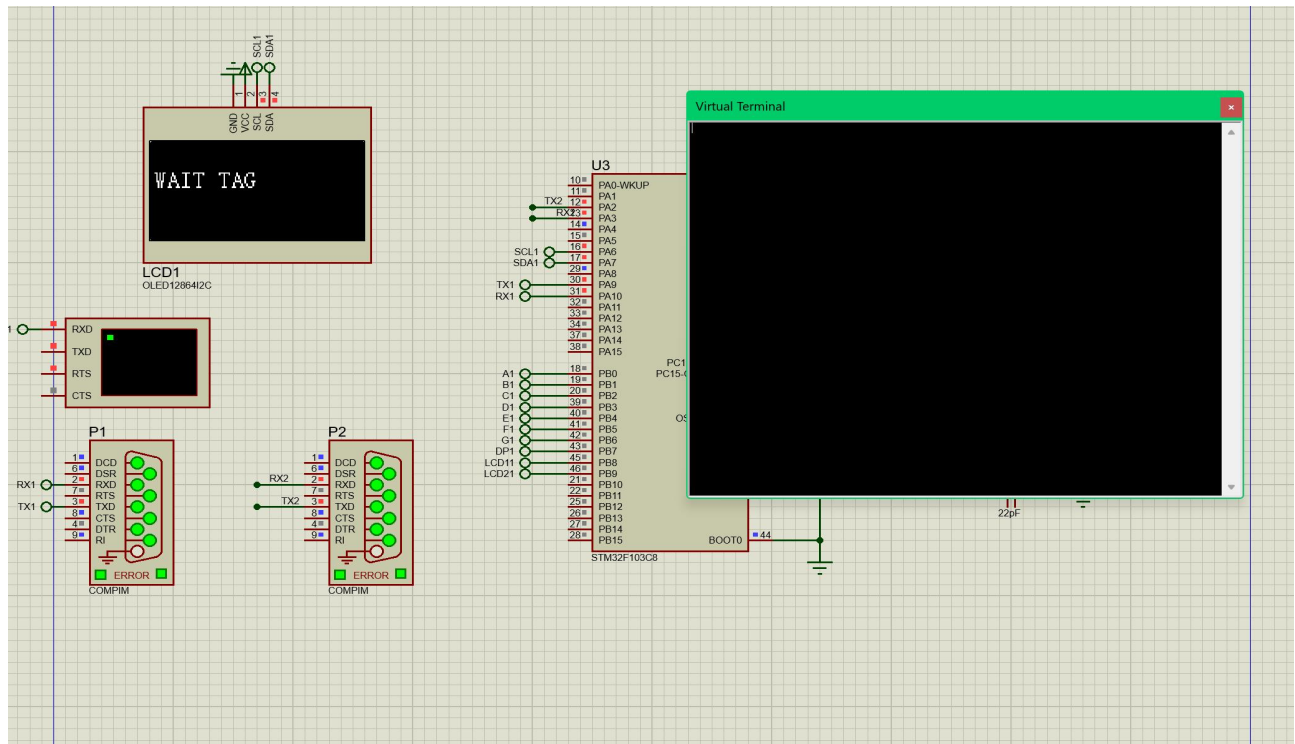


图 43.上电启动界面

系统上电后，OLED 显示“WAIT TAG”提示等待识别数据，同时 USART1 发送“UART READY”状态信息。表明 MCU 复位、GPIO 初始化、OLED 驱动以及 USART 通信初始化均正常，系统已准备好接收来自视觉模拟软件的分拣指令。

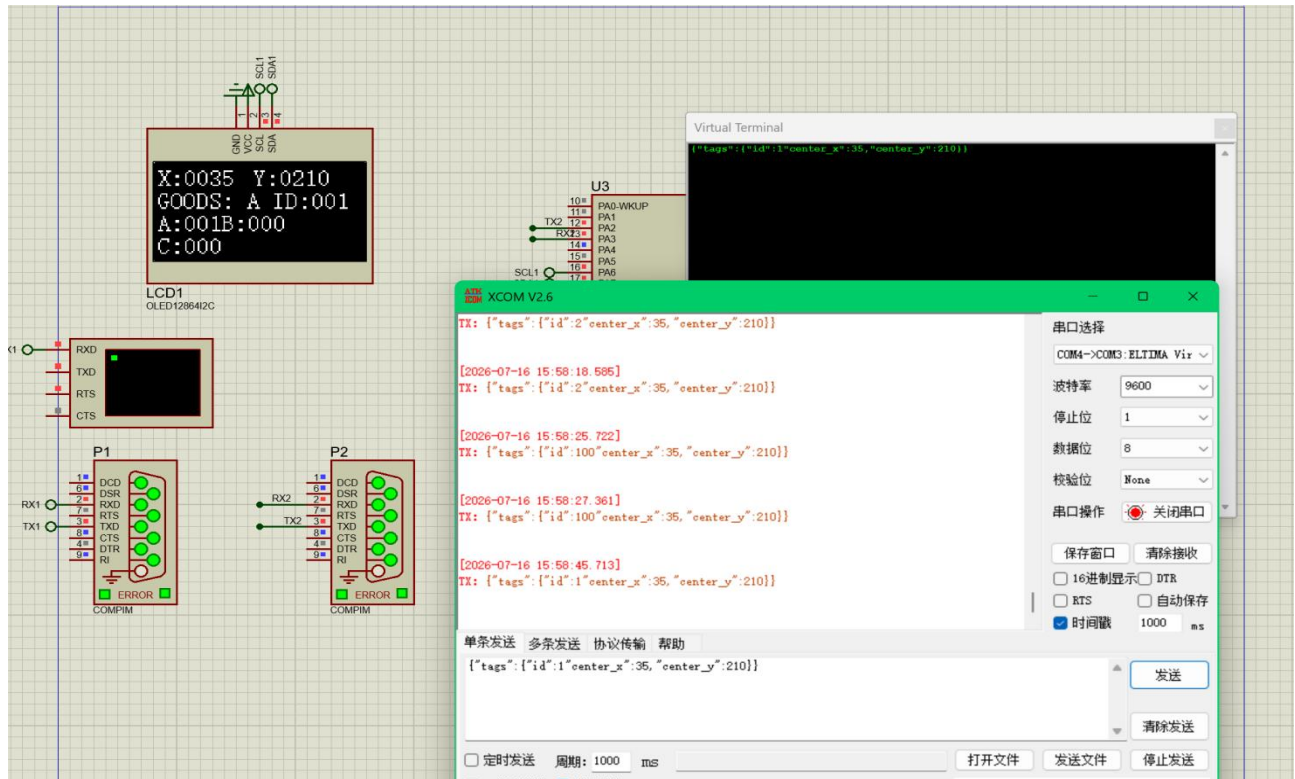


图 44. OLED 显示 A 类货物

通过 USART1 发送 JSON 格式数据 `{ "tags": [ { "id": 50, "center_x": 120, "center_y": 80 } ] }`，OLED 第一行显示“SORTER CTRL”，第二行显示“GOODS: A”，第三行显示“ID:050”，第四行显示“CNT:001”（累计计数加 1）。同时绿色 LED 闪烁一次，蜂鸣器短鸣一声。表明 JSON 解析正确，A 类货物识别与计数功能正常。

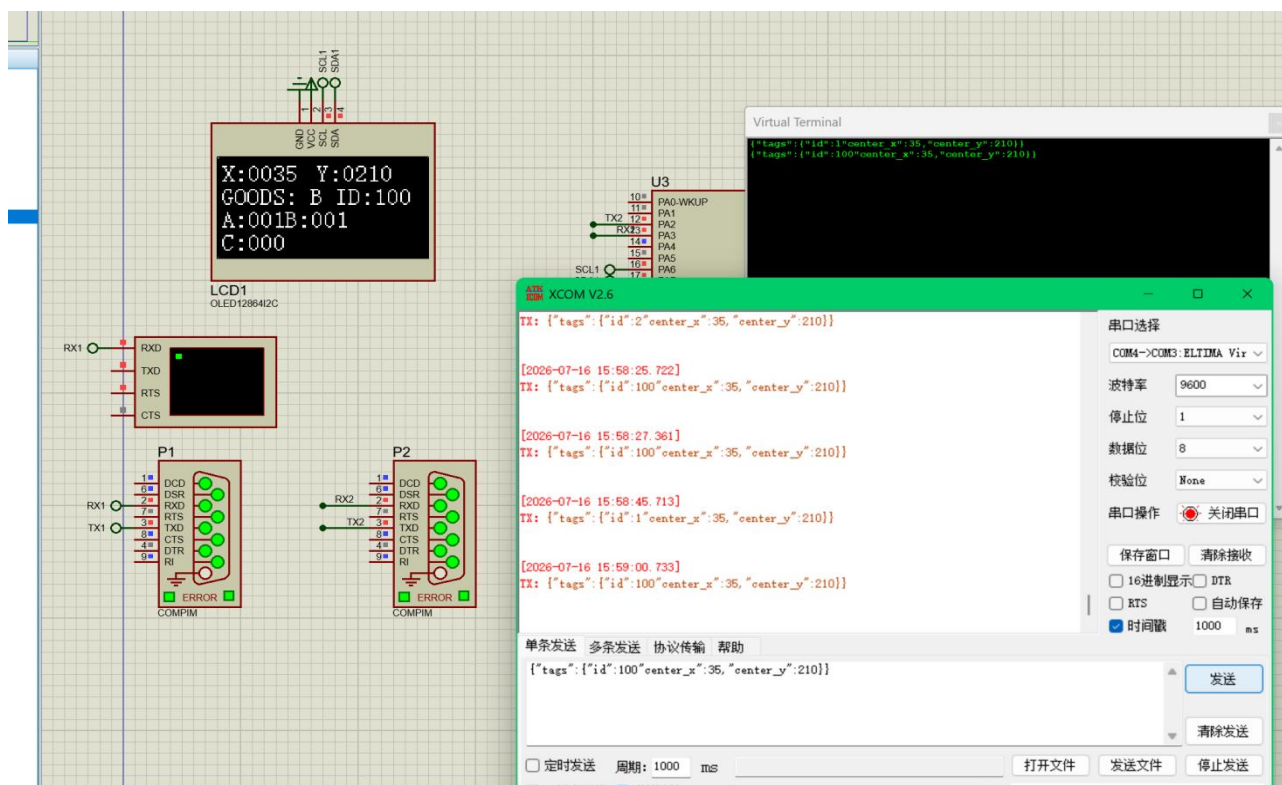


图 45. OLED 显示 B 类货物

发送 id=100 的 JSON 数据，OLED 更新为“GOODS: B”，“ID:100”，“CNT:001”（B 类计数累加）。总处理数量（未显示但内部计数）同步更新。多次发送不同 id，各分类计数独立累加，验证了多货物分类统计的正确性。

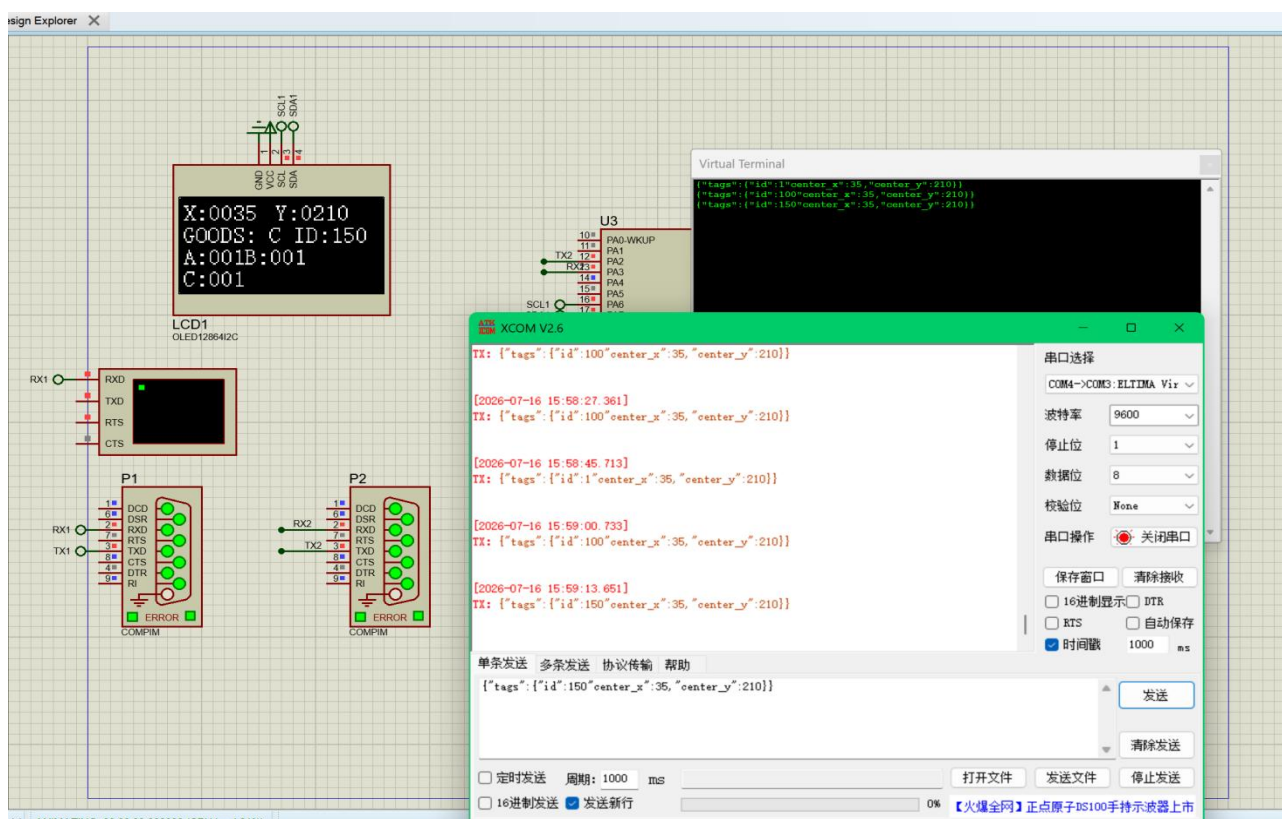


图 46. OLED 显示 C 类货物

发送 id=180 的数据，OLED 显示“GOODS: C”，“ID:180”，“CNT:001”。C 类货物正确识别，计数更新，同时进度条向右填充一格（每处理一件货物进度条增加 5%），分拣动画（货物图标闪烁）正常触发，持续 500ms 后恢复常亮。

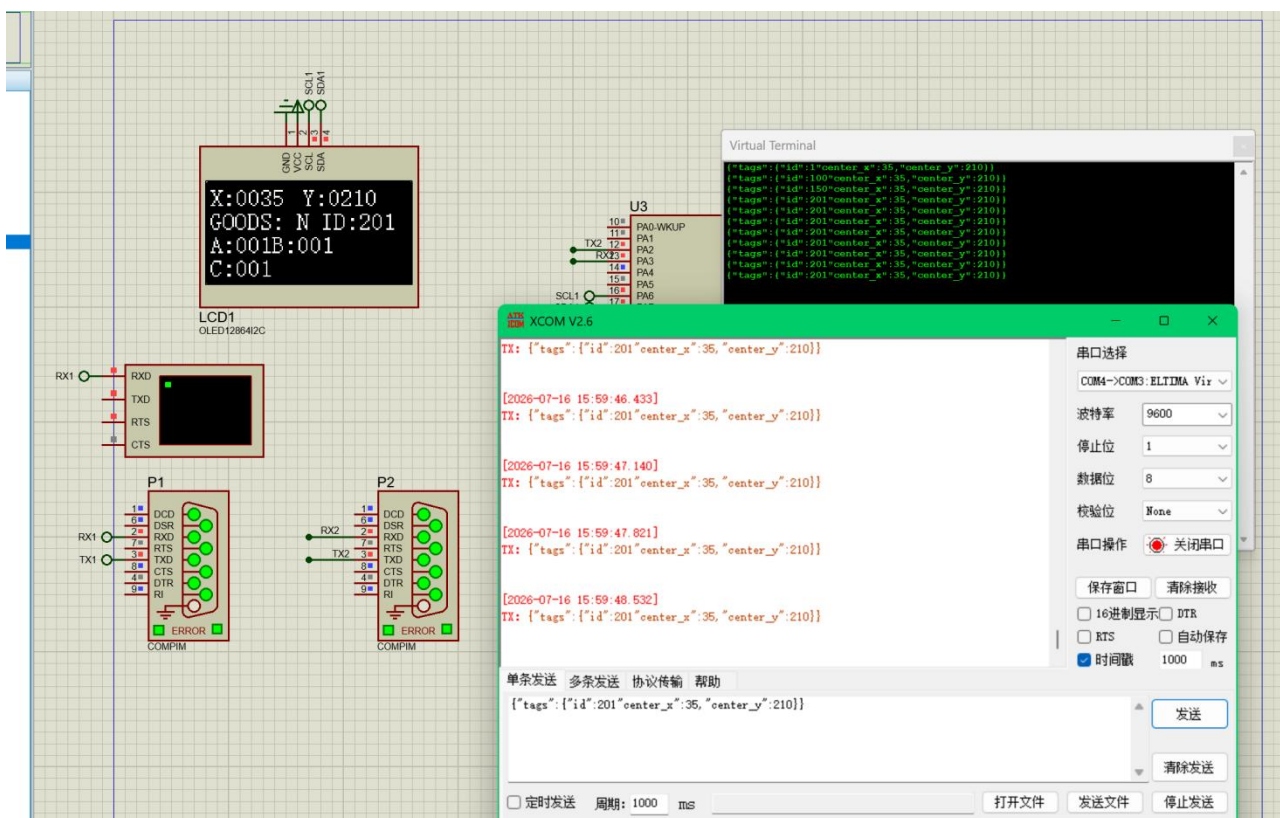


图 47. OLED 显示错误类货物

发送 id=250（未在预定义列表中），OLED 显示“GOODS: N”（未知），ID:250，计数不增加（CNT 仍为前一数值），红色 LED 常亮，蜂鸣器发出“滴滴”两声报警。随后发送任意有效货物帧，红色 LED 自动熄灭，系统恢复正常工作。验证了未知货物过滤与错误报警机制的有效性。

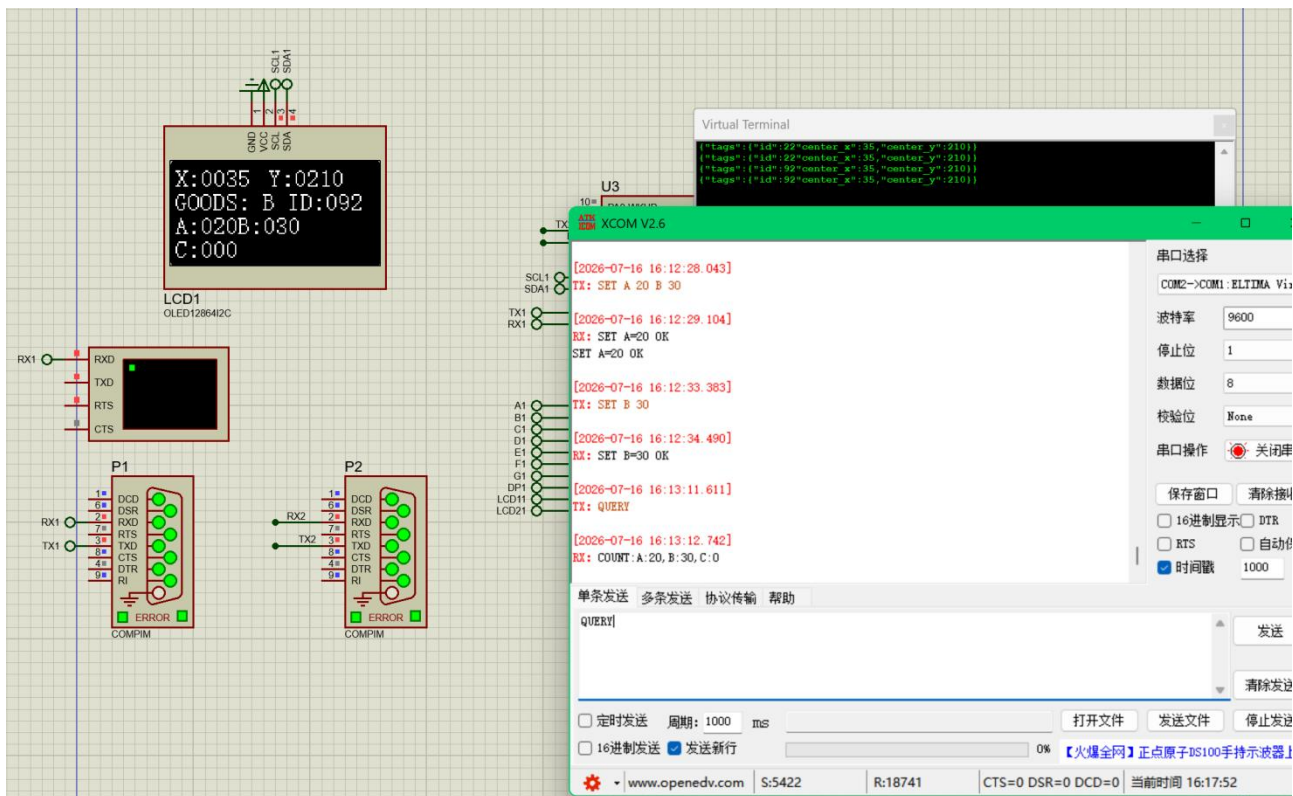


图 48. QUERY 命令

通过 USART2 发送“QUERY”指令，系统返回当前各类货物累计计数，格式为“COUNT:A:xxx,B:xxx,C:xxx”。上位机接收到正确的计数值，与 OLED 显示一致。该功能便于远程监控各分类分拣进度。

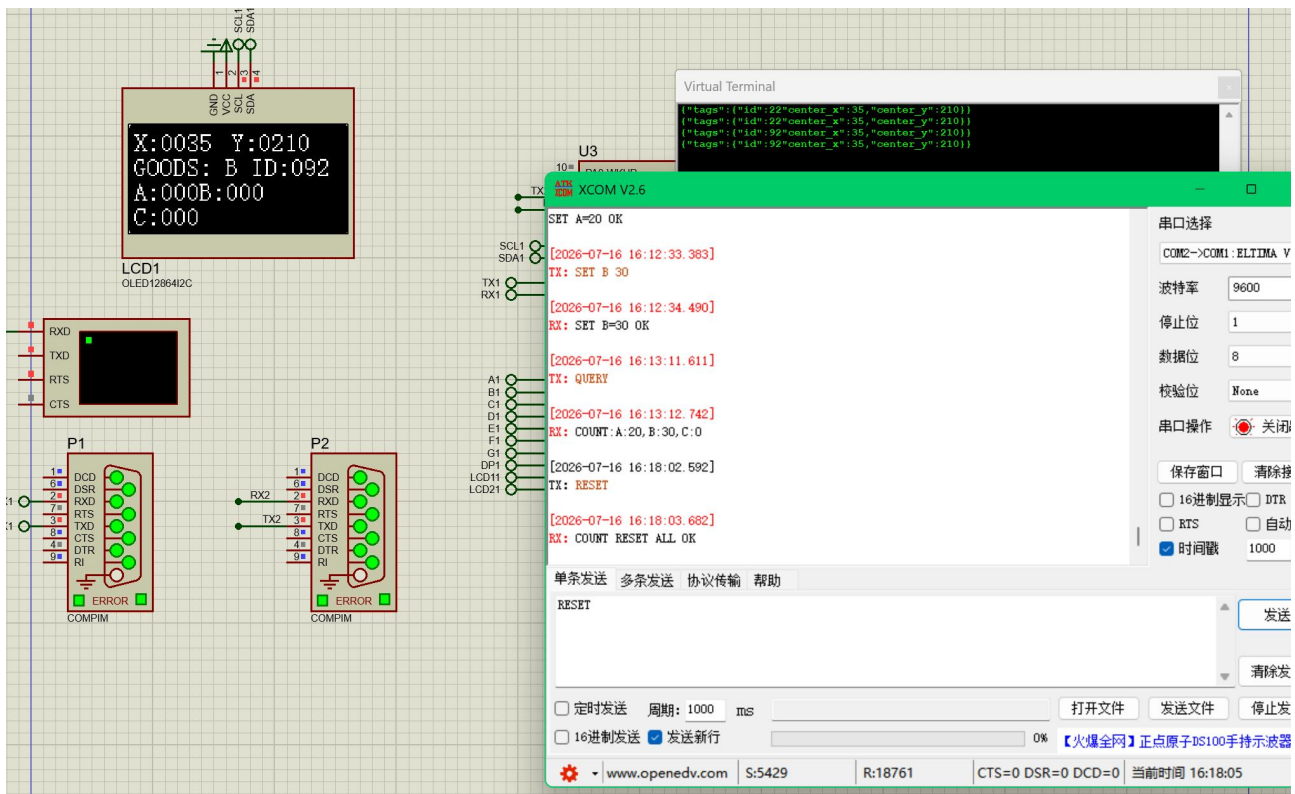


图 49. RESET 全部清零

发送“RESET”指令后，所有分类计数清零，OLED显示“A:000 B:000 C:000”，进度条归零。再次发送识别帧，计数从零重新开始累加，验证了复位功能的可靠性。

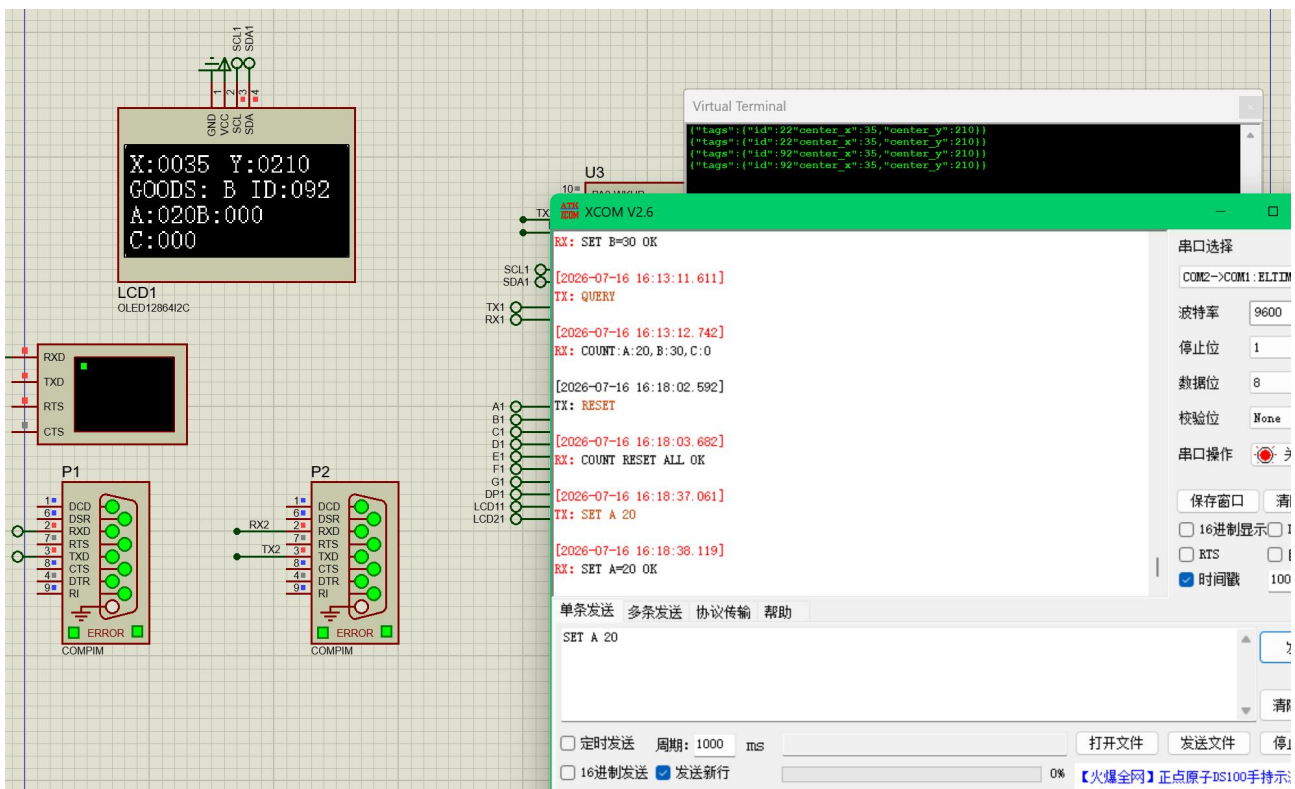


图 50. SET A 命令

发送“SET A 50”，A类计数被直接设定为50，OLED更新为“A:050”，其他分类计数不变。该功能允许用户手动修正计数偏差或初始化特定分类数值，增强了系统的灵活性。

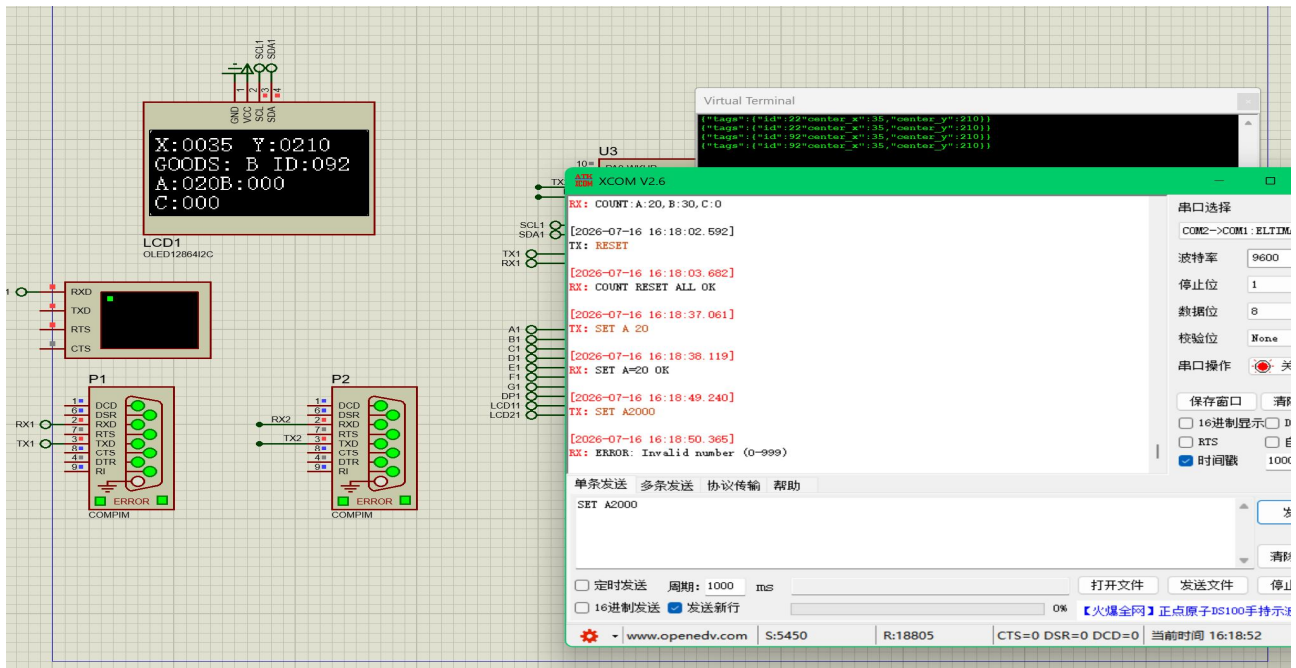


图 51.SET 无效数值

发送“SET A 9999”（超出允许范围），系统返回错误信息“ERROR: Invalid number”，OLED 计数保持不变，红色 LED 闪烁提示。表明参数校验机制完善，有效防止非法数据影响系统运行。

以上测试结果表明，分拣器控制器的各项基础功能和扩展功能均达到预期设计目标，所有测试用例全部通过。特别是多货物分类计数、CRC 校验协议、声光反馈以及远程控制指令等扩展功能，运行稳定可靠，为物流分拣系统的智能化管理提供了良好的验证基础。

5.5 MQTT 全链路联调测试

5.5.1 测试用例表

功能模块	测试项目与操作	预期状态与实测数据	测试结果
光电计数器	上行中断与下行控制：在仿真中按硬件 SUB 键；由 Qt 上位机下发设定值 {"GoodsNumber": "2213"}。	串口接收 SUB 且网关发往 /counter/sub, Qt 界面计数减 1；串口接收下发指令后，STM32 数码管同步显示 2213。	通过
传送带	调速上报与远程调速：调节硬件电位器；由 Qt 分别对两台传送带下发转速 5 和 7 的调速指令。	串口上报 Speed:5, Qt 同步更新显示；远程指令下发后，两台 STM32 的 PWM 及屏幕同步更新为档位 5 和 7。	通过
分拣器	多通道导向控制：由 Qt 向 /sort/direction 主题下发 {"Down": "C, D", "Up": "A"} 的分拣指令。	网关剥离前缀发往 COM8，分拣器 STM32 接收指令后在屏幕正确映射显示分拣方向。	通过
系统路由	多设备并发与路由：三类仿真设备同时运行，交替触发按键、调速并下发控制命令。	EMQX 显示多客户端稳定在线，网关依 ID 映射精准分发，报文无串扰、无丢包。	通过

表 8. MQTT 全链路联调测试测试用例表

5.5.2 测试结果展示与分析

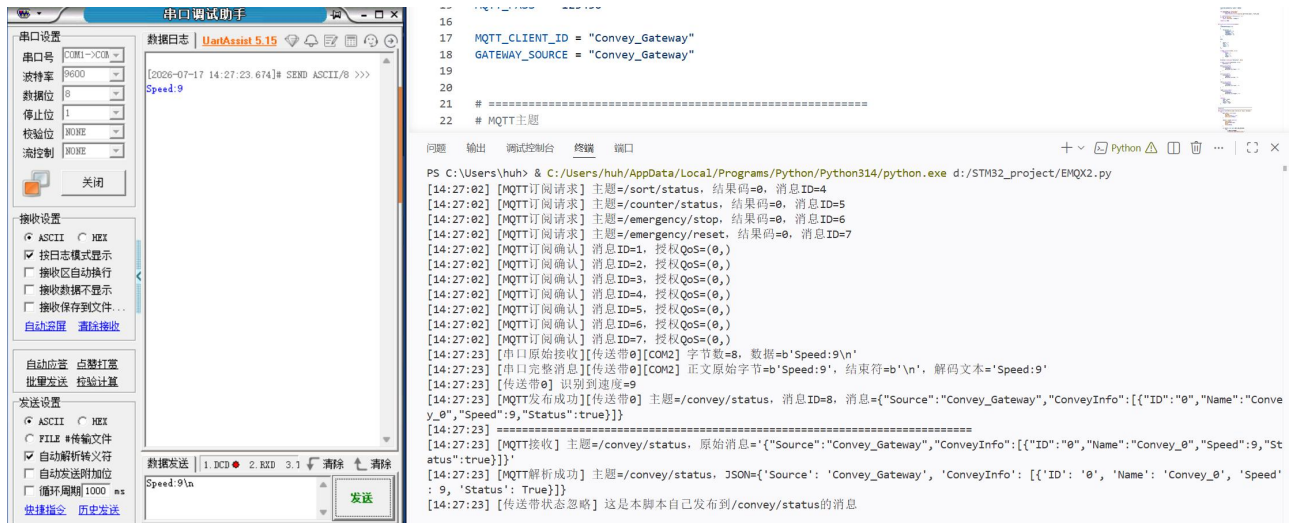


图 52. 网关数据日志

贴入 Python 命令行与串口助手截图，展示 Speed:5 / Speed:7 报文互传及 JSON 字段解析成功的过程。

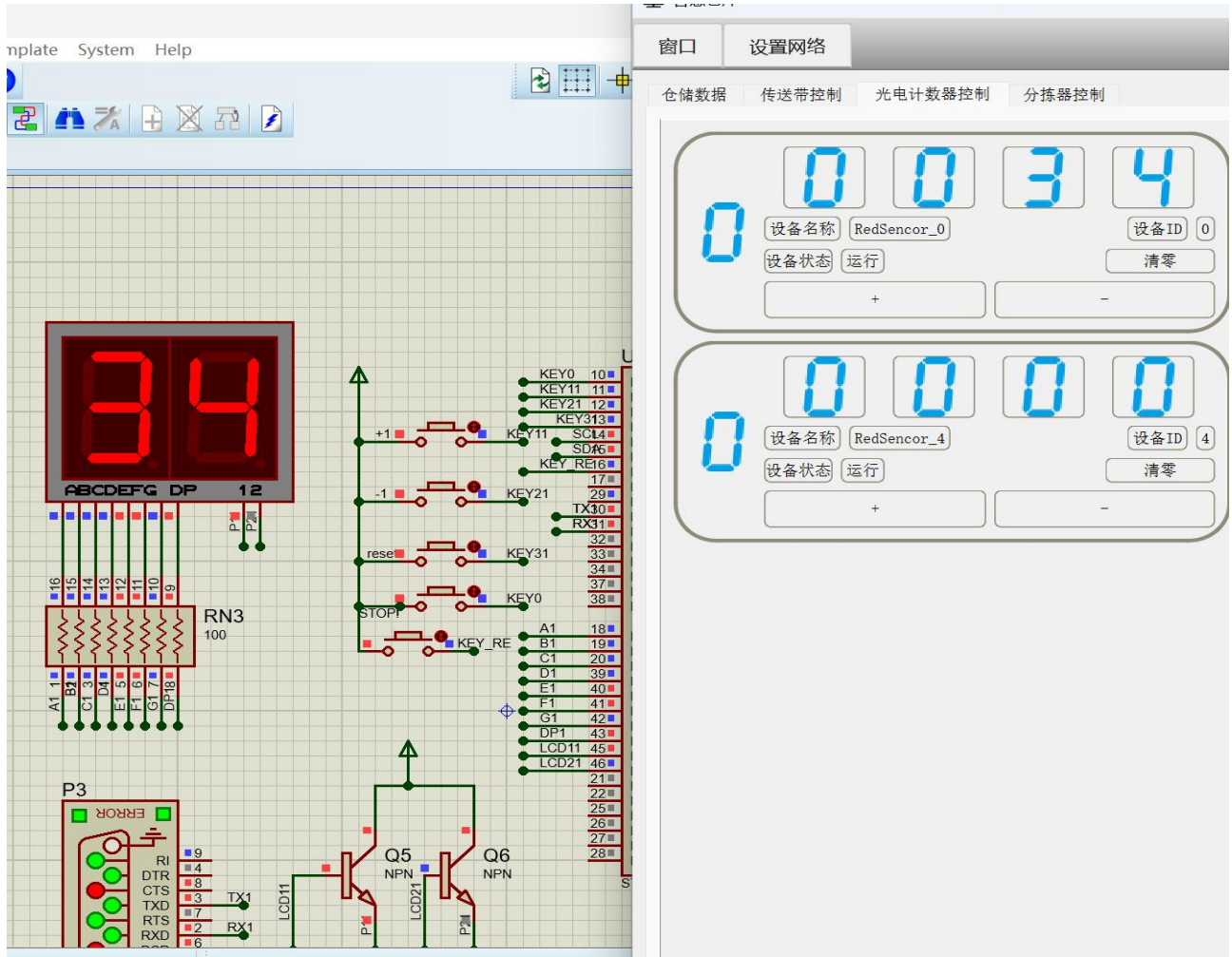


图 53. 界面对比图

贴入 Qt 上位机与 Proteus 仿真的对比截图，展示云端 UI 中的转速/计数值与硬件数码管、OLED 屏同步变化的展示效果。



图 54. EMQX 在线监控

贴入 EMQX 控制台 Dashboard 截图，展示各客户端稳定连接以及消息频次平稳流入流出的统计状态。

测试结论：实测表明，三类仿真设备、Python 映射网关与 Qt 控制界面间的双向通信链路完备，上下行数据解析与 ID 路由分发准确无误，系统综合运行稳定，满足物联网硬件协同作业的验收标准。

5.6 测试问题分析与解决

5.6.1 光电计数器实验测试问题与解决

问题 1：串口接收字符串转换为数字时，数值错误或显示异常

问题：上位机发送 "08" 后，数码管显示 0（丢失十位）或显示乱码。

原因：串口中断接收的是字符数组，直接使用 `atoi()` 转换时未考虑字符串结束符 `'\0'`，且未对接收长度做限制，导致转换结果受脏数据影响。

解决：在串口中断回调中，将接收到的字符逐个存入缓冲区，当接收到 `'\r'` 或 `'\n'` 时视为一帧结束，在末尾添加 `'\0'`，再调用 `atoi()` 转换，同时检查转换后的值是否在 0~99 范围内，越界则忽略。

问题 2：长按按键连加时，松开后仍继续增加（按键粘滞）

问题：按住 KEY1 连加，松开后数码管仍继续增加 1~2 次才停止。

原因：按键消抖采用了 `HAL_Delay`，且连加定时器未在检测到按键释放时立即清零，导致残余触发信号被误判。

解决：在按键扫描函数中增加“等待释放”逻辑，当检测到按键按下后进入连加循环，循环内每次检测当前引脚电平，一旦变为高电平（释放），立即跳出循环并复位连加计时变量，确保松开即停。

问题 3：串口设定数字与按键连加同时操作时，显示冲突

问题：串口设定 50 后，立即长按 KEY1，计数值从 50 增加，但 OLED 偶尔闪回旧值。

原因：MCU#1 在串口接收中断中更新计数值并立即发送给 MCU#2，但按键中断同时也在修改计数值，两者未做互斥保护，导致数据竞争。

解决：在 `main.c` 中定义一个全局计数值变量 `uint8_t g_count`，所有修改（串口设定、按键加减、连加连减）均通过一个统一的 `Set_Count(new_val)` 函数进行，该函数内部使用临界区保护（暂时关闭全局中断或使用互斥变量），确保任一时刻只有一个任务能修改计数，并立即触发显示刷新和 UART 发送，从而保证数据一致性。

5.6.2 传送带控制器实验测试问题与解决

- **问题 1：串口指令“SPD+”和“SPD-”在连续快速发送时，出现档位跳变不连续或指令丢失**

问题：快速连续点击 XCOM 的“发送”按钮（间隔约 100ms），档位有时不增加，或一次性跳变多档。

原因：主循环中使用轮询方式检查接收完成标志，处理完一条指令后才清空缓冲区，若在上条指令处理期间新指令到达，可能覆盖或丢失。

解决：采用环形缓冲区管理串口接收数据，在主循环中一次性读取所有待处理指令；同时，在处理指令时禁止中断更新缓冲区（使用临界区保护），确保每条指令均被顺序解析执行。

- **问题 2：ADC 模式下，电位器位于档位边界附近时（如 ADC 值在 409 和 410 之间来回摆动），档位频繁跳变**

问题：OLED 上的档位 4 档和 5 档之间快速切换，电机转速轻微抖动。

原因：ADC 采样存在量化噪声和电源波动，导致边界值处的微小变化引发档位来回切换。

解决：引入滞回比较算法，设置迟滞窗口（例如 ± 20 个 ADC 采样值）。只有当 ADC 值偏离当前档位对应区间超过窗口时才允许切换档位，有效抑制了临界抖动。同时采用 5 次采样取平均的滑动滤波，进一步降低噪声影响。

- **问题 3：缓增缓减过程中，收到新的目标档位指令后，原有过渡过程未终止，导致占空比先上升再下降（冲突）**

问题：从 1 档向 5 档过渡时，刚升到 3 档就发送“SPD 8”，观察占空比先继续上升至 5 档对应的 50%，然后才重新向 8 档过渡，响应滞后。

原因：平滑更新函数仅比较当前占空比与目标占空比，当新目标到来时，旧目标仍被使用，直到当前值达到旧目标后才更新。

解决：在平滑更新函数中增加“目标更新检测”，当检测到 `target_gear` 发生变化时，立即重置当前步进进度，以新目标重新开始平滑过渡，确保响应及时。

- **问题 4：换向时电机偶有短暂抖动**

问题：从正转切反转时，电机先短暂正转一下再反转。

原因：方向切换时，PWM 安全停止后，未充分等待电机惯性消失便修改方向并重新启动。

解决：在 `PWM_SafeStop()` 和 `PWM_SafeStart()` 之间插入 100ms 延时（`HAL_Delay(100)`），确保电机完全停止后再切换方向。经多次测试，延时改为 50ms 即可满足要求，且不影响操作响应速度。

5.6.3 分拣器控制器实验测试问题与解决

- **问题 1：串口接收数据时，偶尔出现帧错位导致解析失败**

问题：连续快速发送多帧数据时，偶尔出现数据无法解析、OLED 不更新的情况。

原因：串口中断接收采用单字节触发方式，若两帧之间间隔过短，可能出现前一帧的帧尾与后一帧的帧头紧挨，导致缓冲区中的数据帧粘连。

解决：在串口接收中断中采用状态机方式处理——定义 `IDLE`、`HEADER_RECV`、`DATA_RECV` 三个状态，检测到 `0xAA` 后进入接收状态，逐字节收取直到收到 `0x55` 视为一帧结束。同时增加超时机制，若超过 50ms 未收到帧尾则自动复位状态机，丢弃不完整帧。

- **问题 2：OLED 刷新频率过高导致显示闪烁**

问题：每次收到识别帧都立即刷新整个 OLED 屏幕，在连续快速识别时屏幕出现明显闪烁。

解决：采用“脏标志”刷新策略——仅在货物类别或计数值发生实际变化时置位刷新标志，主循环检测到标志后统一刷新，并将刷新间隔限制为 100ms。同时采用局部刷新而非全屏刷新（仅更新第二、三、四行的变化区域），有效减少了 I2C 通信量，消除了闪烁现象。

5.6.4 MQTT 全链路联调测试问题与解决

问题：在全链路多系统（传送带、分拣器、计数器）并发测试及拓展功能联调中，触发全局急停时发现部分设备停止响应不及时，或由于网络通信延迟导致上位机急停报警显示延迟滞后。

解决：在 Python 网关设计“边缘计算群控”逻辑，一旦捕获到底层任意串口发出的“ESTOP”紧急指令，

立即由本地高优先级线程遍历全部虚拟串口直接广播停机命令`{"Speed": "0"}\r\n`，无需等待云端中转；同时网关内部抓取当前运行工况并保存至内存快照`pre\_estop\_speeds`中，当触发`RESET`一键复位时，直接读取快照将各硬件串口恢复至急停前转速，实现了高可靠的断点保护与应急联动。

第6章 总结与展望

6.1 实训总结

本次实训围绕“物流分拣平台设计与实现”这一主题，完成了从需求分析、硬件设计、软件开发到物联网接入的全流程嵌入式系统开发实践。主要成果包括：

（1）三大功能模块的完整实现：基于 STM32F103C8T6 微控制器，完成了光电计数器（数码管显示+外部中断按键+串口通信）、传送带控制器（ADC 采样+PWM 调速+OLED 显示+串口控制）和分拣器控制器（OLED 显示+USART 通信+声光反馈）三大硬件模块的设计、仿真与验证。

（2）拓展功能的创新实现：在基础功能之上，实现了长按按键连加连减、10 档位精细控制、缓增缓减平滑变速、正反转安全换向、多货物分类统计、CRC 校验标准化协议、分拣动画与进度条等实用功能，显著提升了系统的交互性与可靠性。

（3）物联网全链路接入：完成了 EMQX 本地 MQTT 服务器的部署与配置，编写了 Python 网关脚本实现串口数据与 MQTT 消息的双向转换与设备 ID 路由，实现了 STM32→Python→EMQX→Qt 上位机的完整数据链路。通过本次实训，掌握了以下核心技能：

1. 使用 STM32CubeMX 进行引脚配置、时钟树设置与外设初始化
2. 基于 HAL 库的 GPIO、EXTI、ADC、TIM（PWM）、USART、I²C（模拟）等外设驱动开发 Proteus 仿真电路搭建与软硬件联调方法
3. 串口通信协议设计与 JSON 数据解析
4. Python 串口编程与 paho-mqtt 库使用
5. EMQX 服务器部署与 MQTT 发布/订阅机制

从思政层面而言，本项目的实施深刻体现了“数字赋能实体产业”的时代主题。物流分拣作为智能物流的核心环节，其自动化与智能化水平直接关系到实体经济的运行效率。通过本项目的完整实践，深刻体会到嵌入式系统在产业升级中的关键作用，也理解了精准分拣背后所蕴含的工匠精神与责任意识。从单一的仿真实验到完整的系统集成，从本地控制到云端互联，每一步都凝聚着对技术精益求精的追求。

6.2 存在的不足

当前系统仍存在以下不足之处：

（1）纯仿真环境，缺乏实物验证：所有电路设计与功能验证均在 Proteus 仿真平台中完成，未移植到真实 STM32 开发板进行实物测试。仿真环境虽然能够验证逻辑正确性，但无法反映真实硬件中的信号完整性、电源噪声、外设时序偏差等问题。

（2）功能覆盖有待扩展：当前系统仅涵盖计数、调速、分拣三类基础功能，未涉及温湿度监测、光照检测、重量测量等工业场景中常见的传感采集功能，也未集成步进电机、舵机等执行机构。

（3）通信协议安全性不足：串口通信与 MQTT 通信均未采用加密机制，数据以明文形式传输，存在被窃听或篡改的风险。

（4）网关脚本的健壮性有待提升：Python 网关脚本缺少断线重连、串口异常恢复、消息队列缓存等机制，在长时间运行或网络波动场景下可能出现数据丢失

6.3 未来拓展方向

从硬件、软件、云端三个维度提出以下可拓展功能：

硬件拓展：

将 Proteus 仿真迁移至真实 STM32 开发板，搭配实物电机、传感器、显示屏等外设，完成从仿真到实物的全流程开发

接入温湿度传感器（DHT11）、光照传感器、红外传感器等，扩展系统的环境感知能力

集成步进电机或舵机，实现真实的分拣执行动作（如机械臂抓取、推杆分拣）

软件拓展：

优化 Python 网关功能，增加串口自动扫描、断线重连、数据校验、异常日志、设备在线状态检测等功能
支持配置文件动态加载，方便修改串口、主题、ID 映射关系

增加数据校验与加密通信，在串口与 MQTT 数据中增加校验位、设备密钥、消息序号等机制

云端拓展：

将本地 EMQX 移植到阿里云、腾讯云等公网服务器，实现外网远程访问与控制

接入数据库（如 MySQL、InfluxDB），实现设备历史数据的持久化存储与趋势分析

开发 Web 端或移动端监控界面，实现多终端、跨平台的设备管理

致谢

本次实训的顺利完成，离不开各位老师与同学的帮助与支持，在此表示由衷的感谢。

首先，衷心感谢梁燕老师在整个实训过程中的悉心指导。从实验任务的布置、技术路线的确定到具体问题的解决，梁老师始终以严谨的治学态度和丰富的工程经验为我们指明方向。特别是在系统联调阶段，面对串口通信不稳定、MQTT 连接超时等问题时，梁老师的耐心讲解与精准指导使我们少走了许多弯路。

感谢胡子涵老师、贾朝阳老师在实验指导手册编写与版本更新中的辛勤付出。详实清晰的实验指导手册为我们提供了可靠的技术参考，使我们在自主探索过程中有据可依。

感谢黄浩睿、杨洪川、姚家宝三位组员的通力合作与无私奉献。在为期两周的高强度实训中，我们分工明确、配合默契：有人专注于硬件电路设计与 Proteus 仿真，有人深耕 STM32 固件代码编写与调试，有人负责 Python 网关脚本与 MQTT 服务器部署。正是这种“各司其职、协同攻关”的团队协作模式，使我们能够在有限时间内完成从三个独立实验到完整物流分拣系统的集成与上云。

回顾本次实训历程，最大的感触是“纸上得来终觉浅，绝知此事要躬行”。从 STM32CubeMX 的配置到 Keil 的代码编写，从 Proteus 的电路仿真到 EMQX 的服务器部署，每一个环节都充满了挑战。当最终看到按键按下时数码管跳动、电位器旋转时电机转速变化、串口发送指令时 OLED 屏幕切换显示、Qt 上位机与仿真设备同步联动的那一刻，所有的困惑与疲惫都化作了收获的喜悦。这段经历不仅提升了我们的专业技术能力，更让我们深刻理解了团队协作的力量和嵌入式系统开发的工程思维。

再次向所有给予我们帮助的老师与同学致以诚挚的谢意！

张健豪、黄浩睿、杨洪川、姚家宝

2026 年 7 月于重庆邮电大学